

Contents

Every line is a link. The page numbers are read back from the rendered book, never typed by hand.

How to use this manual	3
The exam at a glance	5
MODULE ONE · SYSTEMS, ANALYSTS AND THE LIFE CYCLE	10
Unit 1 · Systems Analysis and the Systems Analyst	11
Unit 2 · The System Development Life Cycle	15
Unit 3 · Methodologies, Feasibility and Approaches	18
MODULE TWO · REQUIREMENTS: DISCOVERY AND MODELLING	23
Unit 1 · Fact-Finding and Requirements	24
Unit 2 · Use Case Modelling	28
MODULE THREE · PROCESS AND DATA FLOW MODELLING	32
Unit 1 · Data Flow Diagrams	33
MODULE FOUR · DATA MODELLING WITH ENTITY-RELATIONSHIP DIAGRAMS	38
Unit 1 · The Entity-Relationship Model	39
MODULE FIVE · OBJECT-ORIENTED MODELLING WITH UML	44
Unit 1 · UML and Class Diagrams	45
Unit 2 · Object-Oriented Design Concepts	48
Unit 3 · Activity and Sequence Diagrams	50
MODULE SIX · DESIGN, ARCHITECTURE AND DELIVERY	53
Unit 1 · System Design and Architecture	54
Unit 2 · Project Management, Installation and Support	57
REFERENCE · THE NOTATION LEGEND AND MUST-MEMORISE LIST	60
The Notation Legend	61
The Must-Memorise List	62
WORKED PAPER 2025/2026	65
Section A: Questions One and Two	66
Section B: Questions Three to Six	70
WORKED PAPER 2024/2025	79
Section A: Questions One and Two	80
Section B: Questions Three to Six	82
OBJECTIVE SECTION · BOTH CLASS TESTS, EVERY OPTION EXPLAINED	89
How to Attack an Objective Question	90
Module One, the Analyst, the Life Cycle and Methodologies	91
Module Two, Fact-Finding and Use Cases	108
Module Three, Data Flow Diagrams	116
Module Four, the Entity-Relationship Model	122
Module Five, Object Orientation and UML	125
Module Six, Design, Architecture and Delivery	129
MOCK EXAMINATION ONE	133
Question Paper	134

Answers	137
MOCK EXAMINATION TWO	140
Question Paper	141
Answers	144
MOCK EXAMINATION THREE	146
Question Paper	147
Answers	150

How to use this manual

Read this page once. It is short, and it decides whether the rest of the manual works for you.

TEACH

THE BARGAIN

This manual assumes you have never analysed a system, never drawn a use case or an entity-relationship diagram, and never heard of a class diagram. It carries you from there to a full mark in the examination. It is also meant to be **enough on its own**: you should never need a lecturer, a textbook or a website to follow it. Anything it uses, it teaches first, including every symbol of its own notation.

In exchange it asks one thing: **draw the diagrams yourself**. Reading a finished use case diagram or a solved data flow diagram creates a feeling of understanding that evaporates in the hall, where you face a blank page and a paragraph describing a business. Every diagram in this book was drawn by hand first from the words of a real question. Do the same, and the gap between your diagram and the one printed here is exactly the information you were missing.

The boxes

Every page is built from a small set of boxes. Each has one job, and the colour tells you which before you read a word.

TEACH

New ideas, explained from nothing. Read these properly.

● MUST MEMORISE

Emerald green means **learn this by heart**. Definitions the examiner asks for word for word, the phases of the life cycle, the four use case relationships, the crow's-foot symbols, the four object-oriented concepts. It is the only thing this colour is ever used for. If a page has green on it, there is something on that page you are expected to know cold.

WORKED

SOLVED RIGHT WHERE IT IS TAUGHT

A real question, solved in numbered steps, then a **now you try** at the end. Every worked box carries a small chip saying where the question comes from, so you know it is a question that was actually asked, not one invented to be easy:

PQ 25/26 Q4 a past paper (session and question number); **TEST 2** a class test; **TEXT** the course text's own exercise. Do the "now you try" before you move on.

⚠ TRAP

A mistake that is easy to make and costly in marks. Most are taken from the exact places the examiner sets a snare: a use case joined to another with the wrong arrow, a data flow drawn straight from an entity to a data store, a many-to-many relationship left with an attribute stranded on the line, an aggregation drawn where the question describes a composition.

RECALL

Questions with the answers printed underneath, below a dashed line. **Cover the answers.** Say your answer out loud, or write it, and only then look. Checking whether you knew something is not the same as knowing it, and the difference is the whole point of this box.

EXAM QUESTION

A question reproduced **as it appeared on the paper**, in a dark **As printed** bar, plain and uncoloured, so you meet it the way the examiner set it. A violet **Break it down** box then reads it in your shoes, and a teal **Answer** box works it in full.

Reading a figure

● MUST MEMORISE

THE NOTATION IS THE EXAM

More than half of this course is drawn, not written. An entity is a rectangle, a relationship a line with a verb on it, and the small marks at the ends of that line, the **crow's foot**, a circle, a single bar, carry the whole meaning of a business rule. A hollow diamond and a filled diamond look almost the same and mean opposite things. Get a symbol wrong and the answer is wrong, however good the prose around it. Every symbol used in this manual is defined the first time it appears, and the full legend sits in the Reference sheet. Where a figure is a diagram you must be able to draw, redraw it yourself until it comes out without looking.

One promise, and one label

TEACH

The promise. Every diagram, every solved question, and every past question quoted in this manual was worked out and transcribed independently, not copied from a model answer. The course's own materials and the papers themselves contain slips (you will meet a question that says "answer parts i to vii" and then prints eight, and a paper that misspells a word inside its own scenario), so nothing here is trusted just because a source printed it. Where a paper is wrong, this manual quotes it exactly and flags the error underneath, because in the hall you must answer the paper as printed.

The label. The course text covers most of what the examiner asks: the life cycle, requirements, use cases, data flow diagrams, entity-relationship diagrams and the UML diagrams are all in it. A few topics are examined but are not in the text, chiefly system architecture and delivery: the client-server tiers, hardware architecture, system acquisition, interface design and relational database design in Module Six. Where a topic is not in the course text, it carries a **NOT IN THE COURSE TEXT** label. It is still taught and solved in full, because it is worth real marks, and the label simply tells you that you will not find it if you go back to the text alone. This manual is where you find it. One more thing to know: the text draws entity-relationship diagrams in the **Chen** style (diamonds and ovals), while the exam asks for the **Information Engineering** style (the crow's foot). This manual teaches both and tells you which the paper wants.

BEFORE YOU START

The Exam at a Glance

What the paper looks like, how long you have, and which topics earn the marks. Learn this before you learn anything else: it tells you where to aim.

⚠ TRAP

THE SHAPE HAS CHANGED. READ THIS BOX TWICE.

Everything printed on the past papers in this book describes a paper with **Section A and Section B**, where you answered one of two and then any three of four. That is genuinely what the 2025/2026 and 2024/2025 papers printed, and both are reproduced here exactly as they were set.

The paper you are about to sit is not that paper. The lecturer confirmed the shape on 9 August 2026, and it differs in the two ways that matter most:

- ▶ There is now an **objective section** as well as the written questions, on the same paper, inside the same three hours.
- ▶ The written half is **three questions with NO CHOICE**: one from Module One, one from Module Two, one from Module Three.

No choice is a harder rubric than one of two plus three of four, not an easier one. Under the old shape a weak topic could be traded away. Under this one it cannot.

This matters more than it sounds, so it gets its own box. When the lecturer says Module One, Two and Three he means the **course text's** three modules, because that is the document he sets from. This book is organised into six parts of its own, and the two numberings are **not the same**. Reading his rule with this book's numbering in mind would send you into the hall having prepared the wrong third of the syllabus.

Here is the course text's own structure, read from its contents and its unit headings, with what each unit becomes in this book:

THE COURSE TEXT	ITS UNITS	WHERE IT IS TAUGHT HERE
Module One Introduction to Information System Development Planning	1 The System Analyst and Information System Development 2 System Development Life Cycle 3 System Development Methodology	Module One of this book, all three units
Module Two Requirements Modeling	1 Fact Gathering Techniques 2 Modelling System Requirement with Use Cases 3 Process Modelling 4 Data Modelling and Entity-Relationship Diagramming	Modules Two, Three and Four of this book: requirements and use cases, data flow diagrams, and the entity-relationship model
Module Three Object orientation and UML	1 Object orientation, ob- jects and classes 2 Structural and Use- Case Modelling in UML 3 Behavioural UML Diagrams	Module Five of this book: UML and class diagrams, the object-oriented concepts, and activity and sequence diagrams

A note on the text, since you will meet it. The course text heads its Module Three Unit 1 as "Data Modelling and Entity-Relationship Diagramming", which is the heading it already used for Module Two Unit 4. The heading is a slip: that unit's own first paragraph is about a Contacts app, classes and objects, and it goes on to teach object orientation. It is recorded here rather than tied away, because a reader looking for the ER material at that heading needs to know it is somewhere else.

Three hours, one paper, two halves.

- ▶ **The objective section.** Its format is the one used in the second class test, which is the multi-select form: questions asking **which TWO** of the options are correct, alongside single-answer questions and matching questions. Both class tests are reproduced and answered in the Objective Section of this book, all sixty questions, every option explained.
- ▶ **Question One, compulsory, from the text's Module One.** The systems analyst and what the job actually involves, the system development life cycle and its phases with their deliverables, methodologies (predictive against adaptive, waterfall, prototyping, rapid application development, spiral, agile) and feasibility.
- ▶ **Question Two, compulsory, from the text's Module Two.** Fact-finding techniques and their trade-offs, functional against non-functional requirements, use case modelling, data flow diagrams, and entity-relationship diagrams. This is the widest of the three, and it is where most of the drawing lives.
- ▶ **Question Three, compulsory, from the text's Module Three.** Object orientation, classes and objects, UML class diagrams and their relationships, and the behavioural diagrams, that is activity and sequence.

What this means for revision. Modules One to Five of this book must all be ready to WRITE and to DRAW, not merely to recognise. The one part that sits outside the three compulsory questions is **Module Six**, system design and architecture, acquisition, installation, support and project management, which the course text does not carry either. Revise Module Six for the objective section, and give the drawing time to Modules Three, Four and Five.

◆ KEY POINT

A rubric offering a choice lets a candidate trade a weak topic for a strong one. A rubric with no choice removes that trade, so a mark for a topic you cannot do is simply lost, and the cheapest marks available are the ones gained by making your worst compulsory topic merely adequate.

BLUNT VERSION: no choice means no hiding. Spend the revision on your worst diagram, not your best.

Two halves, and the objective section is the one that quietly eats the clock.

- ▶ **Five minutes: read the whole paper.** Count the objective questions and check the rubric with your own eyes. The shape described here is what the lecturer confirmed; the instruction printed at the top of your paper is what governs.
- ▶ **The objective section first, at pace.** About 45 seconds a question. A multi-select question needs BOTH of its choices to score, so read every option before deciding and never stop at the first one that looks right.
- ▶ **Then the three written questions, about forty minutes each.** Take the one you are most confident about first. An early complete answer is worth real marks in confidence alone.
- ▶ **Read for nouns and verbs before you draw.** The nouns that matter become entities, classes, processes or data stores; the verbs become relationships, flows or use cases. Underline them on the question paper before a single box is drawn.
- ▶ **Answer the verb in the instruction.** Draw, identify, differentiate and explain want different things. Drawing where it says explain loses the marks even when what you drew is correct.
- ▶ **Leave ten minutes** for the objective questions you flagged and for any part left blank. Something written always beats nothing.

Every worked question in this book opens with a **Break it down** box that does the reading step for you, so that by the exam it is automatic.

The same bank of topics carries every paper. What the new rubric changes is which half of the paper each one lands in, so revise them for different purposes.

THIS BOOK	TOPIC	REVISE IT TO
Module One	The analyst, the life cycle and its phases and deliverables, methodologies, feasibility	WRITE. Question One, compulsory
Module Two	Fact-finding techniques with their trade-offs, functional against non-functional requirements, use case actors, the relationships, the diagram and the narrative	WRITE and DRAW. Question Two
Module Three	Data flow diagrams: the four elements, the context diagram, Process 0 and Level 1, levelling, balancing, and the mechanical errors	WRITE and DRAW. Question Two
Module Four	Entity-relationship diagrams: entities, keys, cardinality, crow's foot against Chen notation, reading an existing diagram	WRITE and DRAW. Question Two
Module Five	UML class diagrams, the object-oriented concepts (encapsulation, inheritance, polymorphism, composition, aggregation), activity and sequence diagrams	WRITE and DRAW. Question Three, compulsory
Module Six	System design and architecture, acquisition strategy, installation, support, project management	RECOGNISE. Objective section

Recognise is a lower bar than write, and it is not a low bar. A multi-select question on aggregation against composition needs the distinction held precisely enough to pick two correct statements and reject two plausible ones.

MODULE ONE

Systems, Analysts and the Life Cycle

Before you can model a system you must know what a system is, what the person building it actually does, and the orderly path a project follows from a vague business complaint to a working, supported product. This module is the whole of Section A of the paper.



Systems Analysis and the Systems Analyst

By the end of this unit you can say what systems analysis and design is, describe what a systems analyst does and the skills the role needs, define an information system, and explain functional decomposition.

ANCHOR

A restaurant is a system. It has parts doing different jobs, the kitchen, the waiters, the till, and none of them is the restaurant on its own. It has a boundary: the supplier who delivers the fish is outside it. It takes input, orders and ingredients, turns them into output, meals and receipts, and it has feedback, because complaints change the next service. Every one of those words is used in its technical sense in this unit.

TEACH

WHAT THE COURSE IS ABOUT

Systems analysis is the process of understanding a business problem well enough to say exactly what a new system must do about it. **Systems design** is the process of deciding how the system will be built to do it. Analysis asks *what*; design asks *how*. Together they are the front half of building any information system, and they are what this course examines.

Analysis comes first and design follows, because you cannot sensibly decide how to build something until you know what it has to do. Analysis is a **discovery** process: its two watchwords are discovery and understanding. Design is a **decision** process: it turns the understanding into a blueprint of program structure, data, screens and hardware that a programmer can build from.

TEACH

WHO DOES IT

The person who does this work is the **systems analyst**. The single most important idea about the analyst, and one the examiner asks almost every year, is this: **a systems analyst is a business problem solver**. The analyst is not primarily a programmer. The analyst studies a business, finds where it is losing time or money or accuracy, works out whether technology can help, and if so specifies the system that will help. The solution to the problem is usually, but not always, an information system.

Because the analyst sits between the people with the problem and the people who will build the solution, the analyst is a **translator**: turning business objectives into functional requirements, user needs into system specifications, and technical jargon into terms an ordinary manager can understand. Everything else the analyst does grows out of that go-between position.

A systems analyst is a business problem solver. Roles and responsibilities: study the existing system and business processes to find problems; gather and analyse user requirements; carry out feasibility studies; document requirements and prepare system models and specifications; guide design, testing, implementation and user training; and act as the link between users and the technical team.

Memory hook: the analyst is a **bridge**. A bridge touches both banks (users and builders) and carries traffic both ways (needs across to the builders, working systems back to the users).

TEACH

HOW AN ANALYST SOLVES A PROBLEM

"How does an analyst solve problems?" is a recurring exam question, and it wants a definite sequence, not a vague answer. The analyst first **researches and understands the problem**, then follows these steps:

1. Verify that the benefits of solving the problem outweigh the costs.
2. Define the requirements for solving the problem.
3. Develop a set of possible solutions (alternatives).
4. Decide which solution is best, and make a recommendation.
5. Define the details of the chosen solution.
6. Implement the solution.
7. Monitor to make sure the desired results are obtained.

Notice the shape: understand, justify, explore options, choose, specify, build, check. It is the same shape as the life cycle you meet in Unit 2, because the life cycle is this problem-solving sequence scaled up to a whole project.

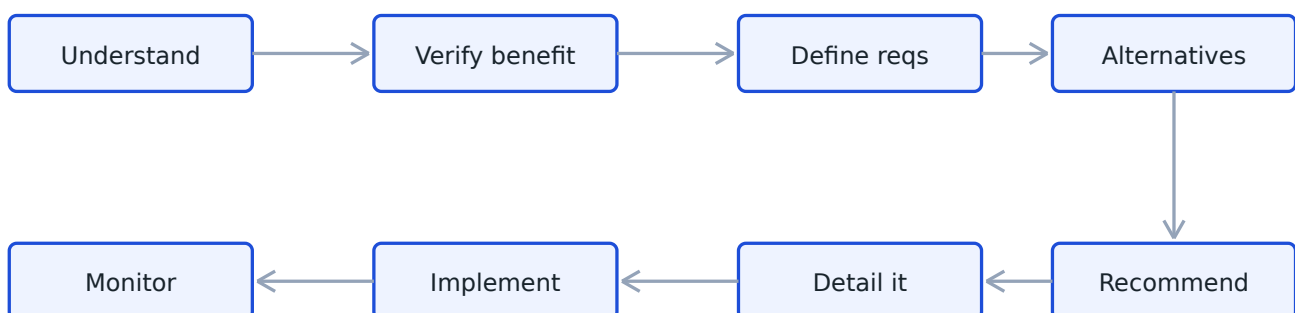


Figure 1.1 · The analyst's problem-solving sequence. It reads left to right along the top, then right to left along the bottom.

When the analyst does solve the problem with technology, the solution is usually an **information system**: a collection of interrelated components (people, data, processes and information technology) that work together to collect, process, store and output the information an organisation needs. Its job is to process input, keep data, produce reports, answer queries and handle transactions.

Any system can be broken into **subsystems**: smaller systems that are part of a larger one. A customer relationship management (CRM) system, for instance, has a sales management subsystem, an account management subsystem and a product information subsystem. Splitting a system into its components this way is called **functional decomposition**, and it is the analyst's single most useful habit of mind: it lets you focus on one area of a business, or the interface between two areas, without having to hold the whole thing in your head at once. Every diagram in this course is a form of functional decomposition.

Because the role is a bridge, the analyst needs three kinds of knowledge and skill. "Describe the skills of a systems analyst" is a standard question; answer it in these three groups and give an example in each.

- ▶ **Technical knowledge and skills.** Understanding of computers, storage, input and output, networks, programming languages and operating systems, and of the tools and techniques used to build systems, so the analyst can understand and communicate technical requirements even without doing the programming.
- ▶ **Business knowledge and skills.** Understanding of how organisations work, how they are structured and managed, and what the specific business does and what makes it succeed, so the system is aligned with real organisational goals.
- ▶ **People (interpersonal) knowledge and skills.** The most important of the three, because the analyst relies on managers, users, programmers and vendors to take a system from idea to delivery. The analyst must communicate well in interviews, reviews and presentations, manage expectations and resolve conflict.

The analyst works with both tools and techniques, and the examiner likes you to tell them apart:

- ▶ A **tool** is a software product used to develop analysis and design specifications or completed components. Examples: a modelling tool such as a computer-aided visual modelling package, a software package such as a database product, and integrated development environments (IDEs) such as Visual Studio.
- ▶ A **technique** is a strategy for completing a specific development activity. Examples: project planning techniques, cost-benefit analysis, interviewing techniques, requirements modelling, and database design techniques.

The one-line distinction: a **tool is a thing you use**; a **technique is a way of doing something**. You will meet a third and fourth term, model and methodology, in Unit 3, and the four-way distinction between model, tool, technique and methodology is itself an exam question.

1. In one sentence, what is a systems analyst?
2. Name the three kinds of skill a systems analyst needs.
3. What is functional decomposition?

ANSWERS

1. A business problem solver who studies a business, finds problems, and specifies the information system that solves them, acting as the bridge between users and the technical team.
2. Technical, business, and people (interpersonal) skills.
3. Breaking a system into smaller subsystems (its components) so each can be studied on its own.

Asked "what does a systems analyst do?", weak answers say "writes the program" or "codes the system". That is the programmer. The analyst's product is **understanding and specification**: models, requirements and recommendations. Say "problem solver" and "bridge between users and technical team", and you have the marks; say "programmer" and you have missed the whole point of the course.

The analyst is not primarily a programmer. The role is to understand a business problem well enough to specify a solution, and then to hold that specification steady between people who know the business and people who can build software. Both technical and non-technical skills are required, and the paper asks for both.

BLUNT VERSION: the analyst is a translator who can be held responsible for the translation.

The System Development Life Cycle

By the end of this unit you can name the phases of the life cycle, state the objective, the main activities and the deliverables of each, and list the principles of a successful systems development project.

TEACH

THE SINGLE MOST EXAMINED TOPIC

The entire process of building, deploying, using and updating an information system is called the systems development life cycle (SDLC). Businesses use information systems to support the work they have to do, and the SDLC is the orderly, phased path a project follows so that the system is built in a disciplined way, on schedule, within budget and to a standard, instead of by trial and error. Without a methodology, projects run over cost, deliver late, or produce a system that does not do what users needed.

"Discuss the major phases of the SDLC and identify the deliverables in each phase" is worth nine or ten marks and has appeared on almost every paper. It is the single most reliable question on the exam. Learn the five phases, their objectives and their deliverables cold.

MUST MEMORISE

THE FIVE PHASES

The SDLC has **five phases: Planning, Analysis, Design, Implementation, and Support**. The first four are the development project; Support is what happens for years afterwards and is not usually counted as part of the original project.

1. **Planning:** identify the need for the system and decide whether it is feasible and worthwhile. Objective: identify the scope, confirm feasibility, and produce a schedule, resource plan and budget.
2. **Analysis:** study the current system and business to determine detailed requirements. Objective: understand and document what the system must do.
3. **Design:** turn requirements into detailed specifications. Objective: design the solution system (architecture, network, interfaces, database, controls).
4. **Implementation:** build, test and deploy the system. Objective: produce a reliable, fully functional system and train the users.
5. **Support:** keep the system running productively after deployment. Objective: maintain, enhance, and support the users for years.

Memory hook: P A D I S: "Please Analyse, Design, Implement, Support." Five letters, five phases, in order.

Each phase is a group of activities. You do not have to recite every one, but knowing the shape lets you write a full answer.

- ▶ **Planning:** define the problem (business problem and scope), produce the project schedule, confirm project feasibility, staff the project, and launch it.
- ▶ **Analysis:** gather information; define system requirements; prioritise requirements; build prototypes to discover requirements; generate and evaluate alternatives; review recommendations with management.
- ▶ **Design:** design the application architecture; design and integrate the network; design the user and system interfaces; design and integrate the database; design the system controls.
- ▶ **Implementation:** construct software components; verify and test; convert data; train users and document the system; install the system.
- ▶ **Support:** maintain the system (fix bugs, make minor adjustments); enhance the system (add capability as the business changes); and support the users, which includes running a **help desk**.

A **deliverable** is the concrete output a phase hands to the next. This is the part of the question most candidates forget, so it is the part that separates a six-mark answer from a ten-mark one.

PHASE	DELIVERABLES
Planning	Project proposal and recommendation; investigation report; analysis of alternatives; cost-benefit analysis; schedule.
Analysis	Detailed system requirements; revised cost-benefit analysis; revised schedule.
Design	Logical design; technical design; test plans; revised cost-benefit analysis; revised schedule.
Implementation	System elements; implementation plans; contingency plans; end-user training materials; maintenance plan; operations plans; cutover or phase-in.
Support	Maintenance reports; updated system documentation.

Installation, turning off the old system and turning on the new, sits in the Implementation phase, and there are **four conversion strategies**. They come up again in Module Six, but meet them here:

1. **Direct cutover:** the new system immediately replaces the old one. Cheapest and fastest, but riskiest: if the new system fails there is nothing to fall back on.
2. **Parallel:** both systems run together for a month or two until it is clear the new one has no bugs. Safest, but the most expensive, because everything is done twice.
3. **Phased:** small parts of the new system gradually replace small parts of the old. Lower risk, but drawn out.
4. **Pilot:** one small group of users runs the new system before everyone else. Limits the blast radius of a failure to one group.

Now you try: a hospital with a live patient-safety system is replacing it. Which conversion strategy would you recommend, and why? (Think about what "the new system fails" costs a hospital, then re-read the four options.)

● MUST MEMORISE

PRINCIPLES OF A SUCCESSFUL PROJECT

"What are the principles of a successful systems development?" is a Section A question in its own right. The principles are:

- ▶ Get the user involved.
- ▶ Use a problem-solving approach.
- ▶ Establish phases and activities.
- ▶ Establish standards for development and documentation.
- ▶ Justify systems as capital investments (cost-benefit).
- ▶ Do not be afraid to cancel or revise scope.
- ▶ Divide and conquer (break the system into manageable parts).
- ▶ Design systems for growth and change.
- ▶ Apply proper planning and project management.

Memory hook: if you remember nothing else, remember **get the user involved** and **use a problem-solving approach**.

RECALL

COVER THE ANSWERS

1. List the five phases of the SDLC in order.
2. Which phase produces the cost-benefit analysis and the schedule as first deliverables?
3. In which phase does installation happen, and name its four strategies.

ANSWERS

1. Planning, Analysis, Design, Implementation, Support (PADIS).
2. Planning.
3. Implementation; direct cutover, parallel, phased, pilot.

◆ KEY POINT

Every phase of the life cycle produces a named deliverable, and the deliverable is what makes the phase examinable. A phase named without its output is half an answer: planning yields a project plan and a feasibility report, analysis yields a requirements specification, design yields the design specification.

BLUNT VERSION: name the phase and its deliverable together, always. One without the other is half a mark.

MODULE ONE · UNIT 3

Methodologies, Feasibility and Approaches

By the end of this unit you can distinguish the predictive and adaptive approaches, describe the main methodologies and say when to use each, and carry out the feasibility checks a project must pass in the Planning phase.

TEACH

TWO FAMILIES OF APPROACH

How a project handles requirements, change and uncertainty puts it in one of two families:

- ▶ A **predictive** (traditional) approach assumes the project can be planned in full in advance and built to that plan, progressing through a fixed sequence of phases each completed before the next begins. It suits systems that are well-understood and unlikely to change.
- ▶ An **adaptive** approach assumes the project cannot be fully planned in advance and must be modified as it goes. Development runs in short cycles (iterations or sprints), each producing a working piece of the system, with continuous user feedback. It suits changing requirements and high uncertainty.

TEACH

PREDICTIVE VERSUS ADAPTIVE, SIDE BY SIDE

PREDICTIVE APPROACH	ADAPTIVE APPROACH
Requirements are defined upfront	Requirements evolve over time
Development is linear and structured	Development is iterative and incremental
Emphasis on detailed documentation	Strong user involvement and collaboration
Limited flexibility to accommodate change	Working systems valued over extensive documentation

A **methodology** is a complete, named way of running a project: it bundles a life-cycle model with the tools, techniques and rules for using them. These are the methodologies the examiner names, with the one thing you must be able to say about each and when to choose it.

- ▶ **Waterfall:** the classic predictive model. Phases flow strictly downward, each finished before the next starts. Choose it when requirements are stable and well understood. Weakness: change late in the project is very costly.
- ▶ **Prototyping:** build a quick working model, show it to users, refine it. Choose it when requirements are unclear and need to be discovered. Weakness: the prototype can be mistaken for the finished system, and scope can creep.
- ▶ **Rapid application development (RAD):** fast delivery through prototyping, reusable components and heavy user involvement. Choose it when the schedule is short and requirements are unclear. Weakness: less suited to large, complex systems.
- ▶ **Spiral:** an adaptive model that repeats a loop of plan, analyse and design, build a prototype, then test and integrate, adding risk analysis each turn. Choose it for large, high-risk projects.
- ▶ **Incremental:** deliver the system in working increments, each adding capability. Choose it when you want value in users' hands early.
- ▶ **Agile:** the modern adaptive family; short iterations, continuous feedback, working software over documentation. Choose it when requirements change often.
- ▶ **V-shape:** waterfall bent into a V, pairing each development stage with the test that validates it. Choose it when verification and traceability matter.
- ▶ **Object-oriented design (OOD):** analyses and designs the system as cooperating objects rather than as separate data and processes. It underlies the UML diagrams of Module Five.

A recurring question: "You are a systems analyst on a project with unclear user requirements, a short time schedule and schedule visibility. What methodology do you recommend? Give its advantages and disadvantages."

1 Match the clues to a family.

"Unclear requirements" rules out a predictive model like waterfall, which needs requirements fixed upfront. "Short schedule" points to fast, adaptive delivery.

2 Name the methodology.

Rapid application development (RAD), or prototyping, is the textbook answer: both discover requirements as they go and deliver quickly.

3 Give advantages.

Fast delivery; requirements discovered with the user, so the result fits real needs; strong user involvement raises acceptance.

4 Give disadvantages.

Less discipline and documentation; can struggle on large complex systems; scope creep if the prototype keeps growing.

Now you try: your manager wants to replace waterfall with prototyping on a **large and complex** project. Do you agree? (Re-read RAD's weakness: what does "large and complex" do to an adaptive method?)

Now the full four-way distinction, an exam question by itself. Keep the one-line version of each:

- ▶ A **model** is a representation of an important aspect of the real world (a diagram of the system). It leaves out detail so you can reason about the whole.
- ▶ A **tool** is a software product used to help do the work (a modelling package, an IDE).
- ▶ A **technique** is a strategy for completing one activity (interviewing, cost- benefit analysis).
- ▶ A **methodology** is the complete package: a life-cycle model plus the tools, techniques and rules that say how to run the whole project (waterfall, agile).

Reading order of size: a methodology contains techniques, techniques use tools, and tools produce models.

TEACH**FEASIBILITY IN THE PLANNING PHASE**

Before a project is allowed to proceed, the Planning phase runs a **feasibility study** to decide whether the proposed system can be built and is worth building. A feasibility study reduces risk: by finding constraints early, management can proceed, modify or abandon before money is wasted. It checks a project from several angles, and each angle carries a matching risk:

- ▶ **Technical feasibility:** is the technology and skill available? Risk: the team cannot build what is proposed.
- ▶ **Economic feasibility:** do benefits outweigh costs (cost-benefit analysis)? Risk: the system costs more than it saves.
- ▶ **Operational feasibility:** will users accept and use it, and does it fit the organisation? Risk: a working system nobody adopts.
- ▶ **Schedule feasibility:** can it be finished in the time required? Risk: delivered too late to be useful.
- ▶ **Legal feasibility:** does it comply with laws and regulations? Risk: fines or forced shutdown.

● MUST MEMORISE**FIVE FEASIBILITIES**

The five feasibility types: **Technical, Economic, Operational, Schedule, Legal.**

Memory hook: TEOSL: or read it as "The Exam Only Suits the Legal", any sentence that fixes the five initials. Note that "confirming project feasibility" is one of the five Planning activities from Unit 2, so feasibility links the two units.

RECALL**COVER THE ANSWERS**

1. Give the one-line difference between the predictive and adaptive approaches.
2. Which methodology suits unclear requirements and a short schedule?
3. Name the five feasibility types.

ANSWERS

1. Predictive plans the whole project upfront and builds to the plan; adaptive plans as it goes, in short iterations with user feedback.
2. RAD (rapid application development), or prototyping.
3. Technical, economic, operational, schedule, legal.

⚠ TRAP**NAME THE METHODOLOGY, THEN JUDGE**

When a question describes a situation and asks which methodology, weak answers list every methodology they know. The marks are for **choosing one** that fits the clues in the scenario and **justifying it from those clues**, then giving its advantages and disadvantages. Read the scenario for the tell-tale words, "unclear requirements", "short schedule", "large and complex", "high risk", "stable requirements", and let them pick the methodology for you.

◆ KEY POINT

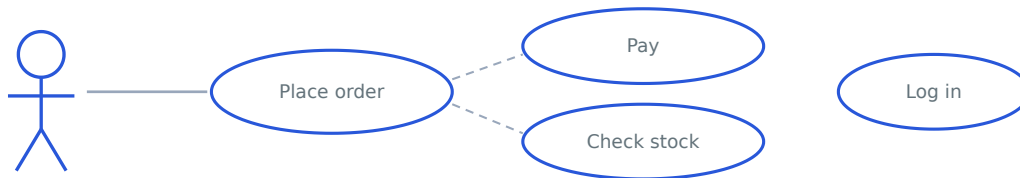
Methodologies divide on ONE question: how much is decided up front. Predictive approaches, waterfall above all, fix the requirements first and are right when those requirements are well understood and stable. Adaptive approaches, prototyping, rapid application development, agile, expect the requirements to move and are right when they will.

BLUNT VERSION: predictive plans the whole road before setting off. Adaptive drives and looks ahead. Choose by how well you know the destination.

MODULE TWO

Requirements: Discovery and Modelling

A system can only be as good as the understanding it is built on. This module is the Analysis phase in detail: how to find out what a business really needs, how to write those needs down so nothing is lost, and how to draw the first model of the system, the use case diagram.



Fact-Finding and Requirements

By the end of this unit you can tell a functional requirement from a non-functional one, name and justify the requirement elicitation techniques with their stakeholders and limitations, and explain the as-is and to-be view of a system.

ANCHOR

Ask a friend what they want for dinner and they say "anything". Ask whether they want rice, and suddenly there are conditions: not rice again, not too late, nothing with fish. The requirement was always there; the open question simply failed to reach it. That is the whole difficulty of fact-finding, and it is why there are seven techniques rather than one.

TEACH

WHAT A REQUIREMENT IS

A **requirement** is a statement of something the system must do or a quality it must have. The whole of the Analysis phase exists to discover, agree and record requirements, because everything designed and built afterwards is built to satisfy them. Get the requirements wrong and every later phase faithfully builds the wrong system. Requirements are divided two ways, and the examiner asks about both.

By **what they describe**:

- ▶ A **functional requirement** is something the system must *do*: a function or feature. "The system shall let a customer place an order." "The system shall calculate the total of an order." They map directly onto use cases and processes.
- ▶ A **non-functional requirement** is a *quality* the system must have: how well it does things, not what it does. Performance, security, usability, reliability, availability. "The system shall respond within two seconds." "The system shall encrypt all passwords."

● MUST MEMORISE

FUNCTIONAL VS NON-FUNCTIONAL

Functional = what it does (a verb). Non-functional = how well it does it (a quality). The paper asks you to formulate three functional and two non-functional requirements from a scenario. Functional ones start "the system shall [verb]"; non-functional ones name a quality (fast, secure, reliable, usable, available).

Memory hook: non-functional requirements are the "-ilities": security, reliability, usability, scalability, availability.

By **whose language they are in**, requirements come at three levels. The 25/26 paper asks you to formulate all three from one scenario, so keep them separate:

- ▶ **Business requirements:** the high-level goals of the organisation. "Reduce the time to process an admission by half." Written in the language of the business.
- ▶ **User requirements:** what a particular user needs to do with the system. "A nurse needs to record a patient's vitals at the bedside." Written from the user's point of view, often as user stories.
- ▶ **System requirements:** the detailed, technical statements the developers build to. "The system shall store vitals as time-stamped records linked to a patient ID." These are the functional and non-functional requirements above.

They nest: business goals drive user needs, which drive system requirements.

Analysis studies two versions of the system. The **as-is** system is the current one, with all its weaknesses; the **to-be** system is the proposed one, with the improvements. A common exam task is to "identify the weaknesses of the as-is system, the expected improvements in the to-be system, and the assumptions that must be validated." Answer it in exactly those three parts: list what is wrong now, list what the new system will fix, and list the things you are assuming that had better be true (for example, that users have smartphones, or that the network reaches every ward).

Discovering requirements is called **fact-finding**, **requirements elicitation**, or **gathering information**. The examiner asks you to choose two or three techniques and, for each, justify **why** it suits this project, say **what** information it collects and from **whom**, and give a limitation. Learn the techniques with those hooks:

TECHNIQUE	WHAT / WHO	A LIMITATION
Interview	One-to-one questions with a key user or manager; rich, deep information, can follow up.	Time-consuming; quality depends on the interviewer; only a few people.
Questionnaire	Written questions to many people at once; cheap way to reach a large, scattered group.	No follow-up; low response rate; answers can be shallow.
Observation	Watching users actually do the work; reveals what people really do, not what they say.	People behave differently when watched; time-consuming; a snapshot.
Document review	Reading existing forms, reports, procedures and business rules; grounds you in facts.	Documents may be out of date or not match real practice.
JAD	Joint application development (JAD): a facilitated workshop bringing users and analysts together; fast consensus.	Hard to schedule; needs a skilled facilitator; dominant voices can skew it.
Prototyping	Building a quick model for users to react to; surfaces requirements people cannot state in the abstract.	Users may think it is finished; can encourage scope creep.

The 25/26 paper asks for a full **requirements elicitation plan**: for each technique, the stakeholders involved, the information collected, and why it suits the project. Suppose the project is a new library management system.

- 1 Pick techniques that cover different sources.
One for depth (interview), one for breadth (questionnaire), one for reality (observation), one for facts (document review).
- 2 Interview the head librarian.
Information: the rules for borrowing, reservations and fines. Why: they hold the business rules and can be probed in depth.
- 3 Questionnaire to student borrowers.
Information: what they want from self-service. Why: cheaply reaches a large, scattered user group.
- 4 Observe the issue desk at a busy hour.
Information: the real check-out workflow and its bottlenecks. Why: shows what staff actually do, not what they report.
- 5 Review existing forms and the current catalogue.
Information: the data already held on books and members. Why: grounds the new system in real records.

Now you try: write the same four-line plan for a hospital appointment system. Who do you interview, who gets the questionnaire, what do you observe, and which documents do you review?

Two smaller questions round the unit off. The **goals of requirement analysis** are to understand the business need, to define what the system must do, and to produce a clear, agreed, prioritised set of requirements. The **overall strategy for fact-finding** is the sensible order of work: start with document review and background reading to get oriented, then observe, then interview key people, then broaden with questionnaires, and confirm findings with the users before you model. Start broad and cheap, then go deep and targeted.

1. Classify: "the system shall email a receipt" and "the system shall be available 99.9% of the time".
2. Which technique reaches many scattered people cheaply, and what is its main weakness?
3. What are the as-is and to-be systems?

ANSWERS

1. The first is functional (a thing it does); the second is non-functional (a quality, availability).
2. A questionnaire; its weakness is no follow-up and often a low response rate.
3. The as-is is the current system with its weaknesses; the to-be is the proposed improved system.

◆ KEY POINT

A functional requirement says what the system must DO; a non-functional requirement says how well, how fast, how safely or how reliably it must do it. The distinction earns marks on its own and is tested by a single question: could you write a test that passes or fails on this sentence, and would that test be about behaviour or about quality?

BLUNT VERSION: functional is what it does. Non-functional is how well it does it.

MODULE TWO · UNIT 2

Use Case Modelling

By the end of this unit you can draw a use case diagram from a scenario, use the four relationships correctly, and write a use case narrative. This is the first Section B diagram, and it appears almost every year.

TEACH

ACTORS AND USE CASES

A **use case** is a single thing a user wants to achieve with the system, named as a verb phrase: "Place order", "Cancel appointment", "Print report". A **use case diagram** shows all the use cases of a system and who uses them. It has three pieces:

- ▶ An **actor**, drawn as a stick figure: a role outside the system that interacts with it (a customer, a store keeper, an insurance carrier). An actor is a role, not a person, and can be another system.
- ▶ A **use case**, drawn as an oval with the verb phrase inside.
- ▶ The **system boundary**, a box drawn around all the use cases, with the actors outside it.

A plain line (an **association**) joins an actor to each use case it takes part in. That is the fourth relationship, and the simplest.

"Identify the types of relationships in use cases" is worth marks every year. There are four, and the two dashed ones are where marks are won and lost:

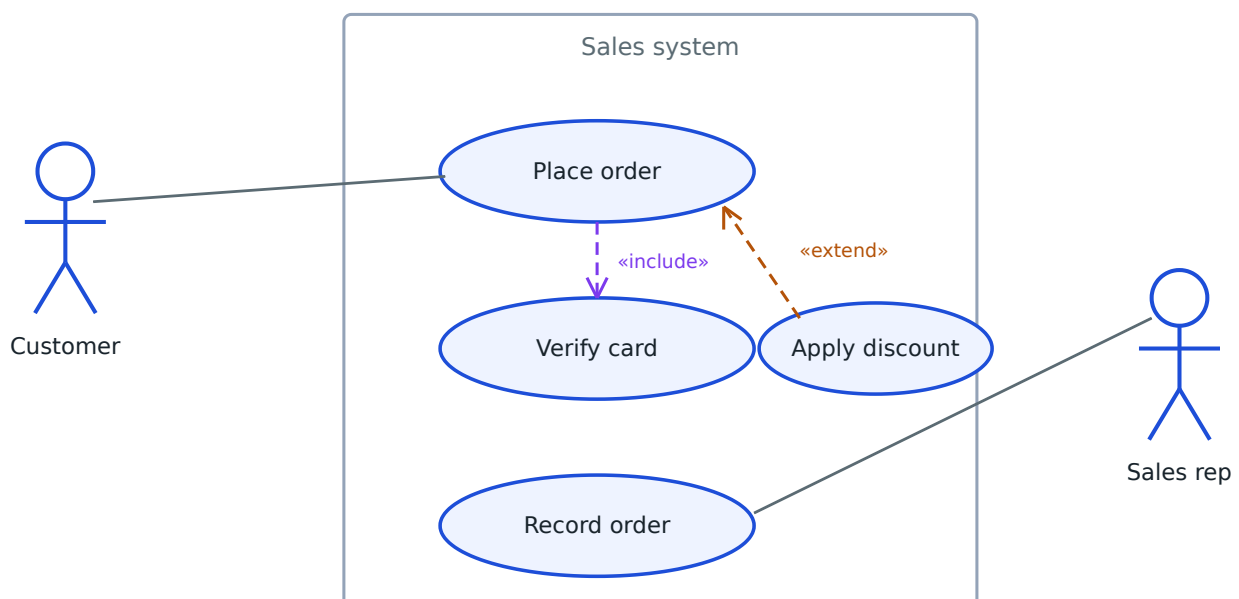
- ▶ **Association** (solid line): joins an actor to a use case it participates in.
- ▶ **Include** (dashed arrow labelled «include»): one use case *always* uses another as a sub-step. "Place order" **includes** "Verify card": the check happens every time. The arrow points to the included use case.
- ▶ **Extend** (dashed arrow labelled «extend»): one use case *optionally* adds behaviour to another under some condition. "Place order" is **extended by** "Apply discount": only sometimes. The arrow points from the extension to the base use case.
- ▶ **Generalization** (solid line with a hollow triangle): one actor or use case is a special kind of another. A "New patient" and an "Old patient" are both a "Patient"; the triangle points to the general one.

Memory hook: Include is always, Extend is maybe. The include arrow points to the step that always runs; the extend arrow points back to the base it might add to.

⚠ TRAP

WHICH WAY DOES THE ARROW POINT?

The commonest use case mistake is the direction of the include and extend arrows. Fix them with one sentence each: **the include arrow points at the thing always done** (base → included), and **the extend arrow points at the thing being added to** (extension → base). Draw both dashed, both labelled with the stereotype in guillemets, or the examiner cannot tell them from an association.



include (Verify card always runs); the amber dashed arrow is an *extend* (Apply discount runs only sometimes).

WORKED

BUILDING THE DIAGRAM FROM WORDS

A recurring scenario: a sales management system lets a sales representative record orders placed by customers and check out orders, and lets a store keeper keep inventory. Recording an order includes updating the order; checkout extends edit-order; keeping inventory includes updating product details. Draw the use case diagram.

1 Find the actors.

The roles outside the system that use it: Sales representative and Store keeper. Draw them as stick figures outside the boundary.

2 Find the use cases (the verbs).

Record order, Check out order, Update order, Edit order, Keep inventory, Update product details. Draw each as an oval inside the boundary.

3 Join actors to their use cases with associations.

Sales rep to Record order and Check out order; Store keeper to Keep inventory.

4 Add the include relationships (always).

Record order «include» Update order; Keep inventory «include» Update product details. Dashed arrows to the included case.

5 Add the extend relationship (maybe).

Check out order «extend» Edit order: the dashed arrow points from Check out to the base it extends.

Now you try: redraw it for the video store, where a member can Rent video (which includes Check for overdue) and Return video, and a manager can Print overdue report. Which relationship is the include?

TEACH

THE USE CASE NARRATIVE

A diagram shows *which* use cases exist; a **use case narrative** (or use case description) tells the story of *one* of them step by step. A simple narrative has: the use case name, the actor, a precondition, the main flow (the normal sequence of steps), and alternative flows (what happens when something is different, for example the card fails). You write a narrative when a question says "describe" or "give the flow of" a use case rather than "draw".

RECALL

COVER THE ANSWERS

1. Name the four use case relationships.

2. "Withdraw cash" always runs "Authenticate". Which relationship, and which way does the arrow point?

3. What does a use case narrative add that the diagram does not?

ANSWERS

1. Association, include, extend, generalization.

2. Include; the dashed arrow points from "Withdraw cash" to "Authenticate" (the step always done).

3. The step-by-step flow of one use case (precondition, main flow, alternatives).

◆ KEY POINT

A use case describes a complete interaction that delivers something of value to an actor, and an actor is a ROLE rather than a person. One human may be two actors, and one actor may be another system entirely. Getting that wrong turns a use case diagram into an organisation chart.

BLUNT VERSION: an actor is a role, not a person, and a use case is a whole job, not a button press.

MODULE THREE

Process and Data Flow Modelling

The data flow diagram is the workhorse of Section B. It shows how data moves through a system: who sends it in, which processes transform it, where it is stored, and what comes out. Learn its four symbols, its three levels, and the handful of mistakes the examiner hunts for, and this becomes reliable, drawable marks.



Data Flow Diagrams

By the end of this unit you can draw the four elements of a data flow diagram, build a context diagram, a Level 0 and a Level 1 from a scenario, and name and avoid the mechanical errors and illegal flows the examiner tests.

ANCHOR

Follow one form through an office. Somebody outside hands it in, that is an external entity. Somebody checks and stamps it, that is a process. It sits in a filing cabinet until it is needed, that is a data store. And it travels between all of these in somebody's hand, which is a data flow. Four things, and every data flow diagram you will ever draw is made of exactly those four.

TEACH

WHAT PROCESS MODELLING IS

Process modelling shows how a system works by following the data. The tool for it is the **data flow diagram (DFD)**: a picture of how data moves from the outside world, through the processes that transform it, into and out of the places it is stored, and back out to the world. Where a use case diagram shows *what* the system does for its users, a DFD shows *how* data flows to make that happen.

The examiner also asks why a DFD is worth drawing before you build. Good reasons: it is a clear picture that non-technical users can check; it separates *what* the system does from *how* it is built; it shows the boundary of the system and every thing it touches; and it breaks a large system into manageable processes (functional decomposition again).

MUST MEMORISE

THE FOUR ELEMENTS

A DFD has exactly **four** elements. Draw each one exactly right; a wrong symbol loses the mark however good the logic.

- ▶ **External entity** (a **rectangle**): a person, department or other system *outside* the system that sends data in or receives data out. Named with a noun (Customer, Bank).
- ▶ **Process** (a **circle** or rounded rectangle, numbered): work that transforms data. Named with a verb phrase and given a number (1.0 Verify order).
- ▶ **Data store** (an **open-ended rectangle**, two parallel lines, labelled D1, D2): data at rest, a file or table. Named with a noun (D1 Customers).
- ▶ **Data flow** (an **arrow**): data in motion between the other elements. Named with the data it carries (order details), not with a verb.



Figure 3.1 · The four DFD symbols. Learn to draw all four; the data store is the one candidates get wrong (it is open at the right, not a closed box).

A DFD is drawn at levels, each an explosion of the one above. The terminology is examinable and slightly confusing, so pin it down:

- ▶ The **context diagram** shows the *whole system as a single process*, numbered 0, with the external entities around it and the net data flows in and out. It has **no data stores**. It defines the boundary: everything inside the one circle is the system, everything drawn as a rectangle is outside it.
- ▶ The **Level 0 DFD** (also called **Diagram 0**, and in some papers the **Process 0 diagram**) explodes that single process into the *major processes* of the system, numbered 1.0, 2.0, 3.0, and adds the data stores and the flows between them.
- ▶ A **Level 1 DFD** explodes *one* of those processes into its sub-processes. A Level 1 of process 3.0 has processes numbered 3.1, 3.2, 3.3.

So when a paper says "draw a context diagram, a Level 0 and a Level 1 for the rent payment process", it wants three diagrams: the system as one bubble, then its major processes, then a close-up of the rent-payment process alone.

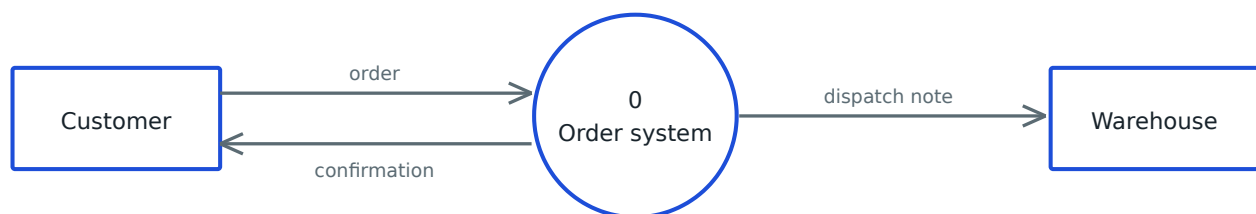


Figure 3.2 · A context diagram: the entire system is one process (numbered 0), external entities are rectangles, and only the net flows cross the boundary. No data stores appear at this level.

● MUST MEMORISE

ILLEGAL FLOWS

A data flow shows data moving from one element to another, but **a process must be at one end of every flow**. Data cannot move on its own; something has to do the moving. So these four flows are **illegal**, and drawing one is a classic lost mark:

- ▶ entity → entity (two outsiders talking bypass the system entirely)
- ▶ entity → data store (nothing processed the data before storing it)
- ▶ data store → data store (data copying itself with no process)
- ▶ data store → entity (data leaving store straight to the outside with no process)

The rule in one line: **every data flow must touch a process at one end**. An entity talks to a process, a process talks to a store, a process talks to an entity; never store-to-store, store-to-entity, or entity-to-entity, or entity-to-store.

⚠ TRAP

DATA DOES NOT MOVE ITSELF

The sentence the paper gives you, "*a data flow shows data movement from one system entity to another*," is true but incomplete, and the follow-up "*data cannot move between entities in a data flow diagram*" catches people out. What it means: data never flows straight between two **external entities**, or straight into or out of a **store** without a process. If your diagram has an arrow from a rectangle to a rectangle, or from a data store straight to a rectangle, it is wrong. Put a process bubble in the middle.

● MUST MEMORISE

MECHANICAL ERRORS

"What mechanical errors can result while constructing a DFD?" is a standard 3-mark question. There are three, all about a process whose inputs and outputs do not match:

- ▶ **Black hole:** a process with input but **no output**. Data goes in and vanishes. (A process that stores data but produces nothing.)
- ▶ **Miracle** (spontaneous generation): a process with output but **no input**. Data appears from nowhere.
- ▶ **Grey hole:** a process whose **inputs are insufficient** to produce its output. Something needed is missing (a receipt that shows a price, from an input that never carried a price).

Memory hook: in goes nothing, out comes something = miracle; in goes something, out comes nothing = black hole; in goes too little = grey hole.

TEACH

DATA BALANCING

When you explode a process into a lower level, the data flowing into and out of the exploded process on the parent diagram must be exactly the data flowing into and out of its child diagram. This rule is **data balancing** (or balancing the DFD). If process 3.0 receives "payment" and sends out "receipt" on the Level 0, then its Level 1 must also receive "payment" and, in total, send out "receipt", no more and no less. An unbalanced DFD, where a flow appears or disappears between levels, is a mechanical error, and checking the balance is one way to **validate the diagram's integrity** after you draw it, along with checking that every process has both inputs and outputs and that no flow is illegal.

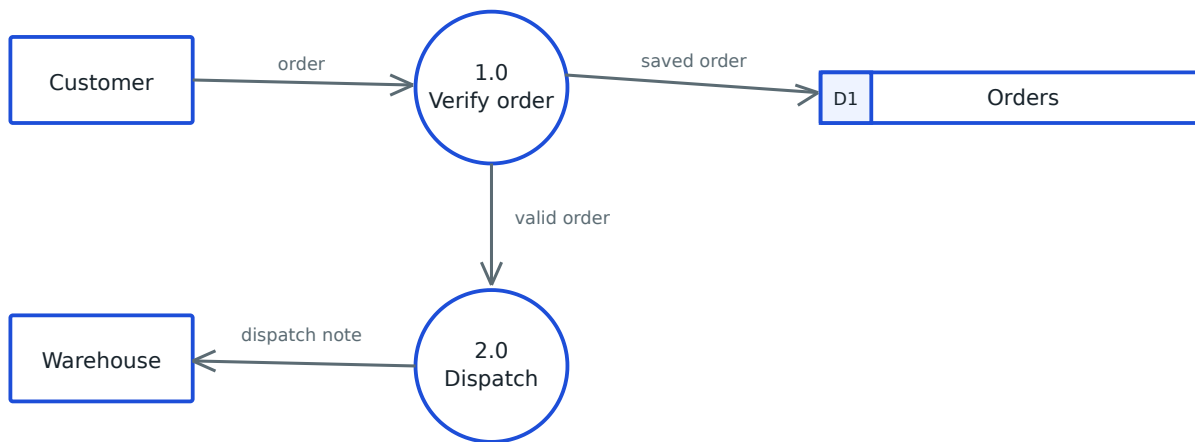


Figure 3.3 · A Level 0 DFD: the single process 0 has exploded into processes 1.0 and 2.0, a data store D1 has appeared, and every flow touches a process at one end. Balance check: the "order" in and "dispatch note" out match the context diagram.

WORKED

FROM A SCENARIO TO THREE DIAGRAMS

The routine for "draw a context diagram, a Level 0 and a Level 1" is always the same. Take the warehouse scenario: trucks arrive, drivers submit trucks, the manager tells a labourer to unload, the storekeeper takes inventory and generates a stock report for the manager.

- 1 Find the external entities (who is outside sending or receiving).
Truck driver, Warehouse manager. They become rectangles.
- 2 Context diagram: one process for the whole system.
One bubble "0 Warehouse system"; arrows in (truck, activity report) and out (stock report). No data stores yet.
- 3 Level 0: explode into the major processes.
1.0 Unload truck, 2.0 Take inventory, 3.0 Generate report; add data store D1 Stock; wire the flows so each process has input and output.
- 4 Level 1: explode one process, say 2.0 Take inventory.
2.1 Count stock, 2.2 Classify condition, 2.3 Update stock record; number them 2.x and balance the flows to match process 2.0.
- 5 Validate.
Every process has an input and an output (no black holes or miracles); no illegal flows; the levels balance.

Now you try: draw the context diagram for food ordering at a Nigerian airport, then give four reasons a DFD is helpful for modelling those processes before you build.

1. Name the four elements of a DFD and the shape of each.
2. A process has an input but produces no output. Which mechanical error?
3. Why can you not draw an arrow straight from a data store to an external entity?

ANSWERS

1. External entity (rectangle), process (numbered circle), data store (open-ended rectangle), data flow (arrow).
2. A black hole.
3. Every data flow must touch a process at one end; a store cannot send data to the outside without a process to move it.

◆ KEY POINT

A data flow diagram shows how data MOVES and is transformed, never when things happen or in what order. It has no decisions, no loops and no timing. Two rules then follow that the examiner checks first: every process must have at least one input and at least one output, and a child diagram must balance with the parent process it explodes.

BLUNT VERSION: a process must earn its outputs from its inputs. No inputs is a miracle, no outputs is a black hole, and not enough inputs is a grey hole.

MODULE FOUR

Data Modelling with Entity-Relationship Diagrams

Where a data flow diagram shows data moving, an entity-relationship diagram shows data at rest: the things a business keeps facts about and how they connect. This is the most heavily drawn topic on the paper. The text teaches it in Chen notation; the exam asks for the crow's foot, so learn both symbols cold.



The Entity-Relationship Model

By the end of this unit you can draw an entity-relationship diagram from a narrative, read cardinality off a diagram, use the crow's-foot and Chen notations, choose an identifier, and interpret a relationship by forward and backward chaining.

TEACH

TWO NOTATIONS, ONE FOR THE EXAM

The entity-relationship diagram is examined heavily and is taught in the course text, but in the **Chen** style (relationships as diamonds, attributes as ovals). The exam, by contrast, asks you to draw it in the **Information Engineering** style (the crow's foot). This unit teaches both and says clearly which the paper wants, so you are never caught out. Read it slowly and draw every figure yourself.

The **entity-relationship (E-R) model** describes the data a business needs to store as a set of **entities** and the **relationships** between them. Drawn out, it is an **entity-relationship diagram (ERD)**. It answers the question "what things does this business keep information about, and how are they connected?" It says nothing about process; that is the data flow diagram's job. This is the single difference the examiner asks for between a data model and a process model.

MUST MEMORISE

THE ELEMENTS OF AN ERD

An ERD is built from four things:

- ▶ **Entity:** a thing the business stores data about (Customer, Order, Patient). Drawn as a **rectangle** with the entity name, singular, in capitals.
- ▶ **Attribute:** a fact about an entity (a customer's name, an order's date). In crow's-foot notation the attributes are listed inside the entity's rectangle.
- ▶ **Identifier (or key):** the attribute that uniquely identifies each instance of an entity. The chosen one is the **primary key (PK)**. When an entity holds the key of another to record a relationship, that borrowed key is a **foreign key (FK)**.
- ▶ **Relationship:** an association between entities, drawn as a **line** with a verb naming it (a customer *places* an order).

TEACH

CHOOSING AN IDENTIFIER

The paper asks you to give "two options for an identifier for a student entity and justify your choice." An identifier must be **unique** and **unchanging**. For a student, the two options are typically the **matriculation number** and the **email address**. Justify: the matriculation number is best because it is assigned once, never reused, and never changes; an email can be changed or reassigned, so it is a weaker key. The lesson generalises: prefer a stable, system-assigned identifier over one the user can change.

● MUST MEMORISE

CARDINALITY: THE CARDINAL RELATIONSHIPS

Cardinality is how many instances of one entity relate to how many of another. There are three cardinal relationships, and naming and drawing them is worth marks every year:

- ▶ **One-to-one (1:1):** each instance of A relates to at most one of B, and vice versa. One employee manages one department, and one department has one manager.
- ▶ **One-to-many (1:M):** one instance of A relates to many of B, but each B to one A. One customer places many orders, but each order belongs to one customer.
- ▶ **Many-to-many (M:N):** each A relates to many B and each B to many A. A student takes many courses, and each course has many students.

Memory hook: read the sentence both ways. "One customer places many orders" and "one order is placed by one customer" gives you 1:M. Always test a relationship in both directions; that is what tells you the cardinality.

TEACH

CARDINALITY VERSUS ORDINALITY

The paper asks you to "differentiate between cardinality and ordinality." Keep them apart:

- ▶ **Cardinality** is the **maximum** number on each side: one or many. It is the 1, the M and the N above.
- ▶ **Ordinality** (also called optionality or the minimum) is whether the relationship is **mandatory or optional**: must there be at least one, or is zero allowed? "An order must have a customer" is mandatory (minimum one); "a customer may have no orders yet" is optional (minimum zero).

Together they give the full rule: a customer relates to **zero or many** orders (optional, many), an order relates to **exactly one** customer (mandatory, one).

● MUST MEMORISE

CROW'S-FOOT SYMBOLS

The **Information Engineering** style, the **crow's-foot** notation, puts the cardinality and ordinality at the ends of the relationship line with two marks: an inner mark for the minimum and an outer mark for the maximum. Learn the four combinations.

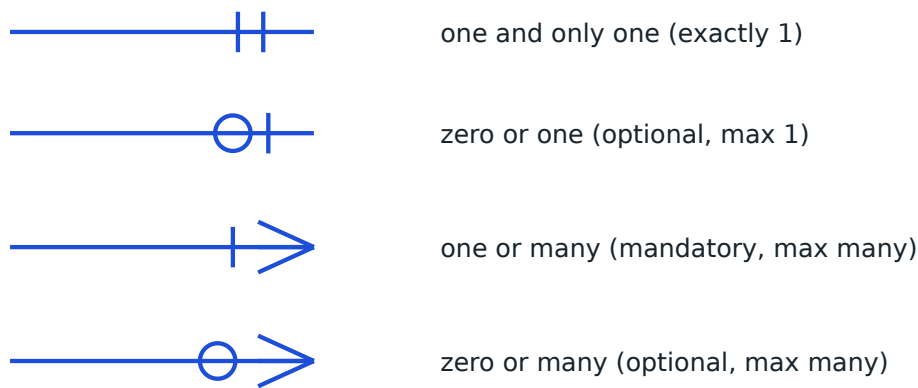


Figure 4.1 · Crow's-foot cardinality. The bar means "one", the circle means "zero" (optional), and the crow's foot means "many". The inner mark is the minimum, the outer mark the maximum.

TEACH

THE NOTATION STYLES

Three notations appear in the papers. They say the same things with different symbols, and you should be able to use whichever a question asks for:

- ▶ **Information Engineering (crow's foot):** entities are rectangles with attributes listed inside; cardinality is the bar-circle-crowsfoot marks above. This is the style the current paper asks for ("using Information Engineering style") and the one to default to.
- ▶ **Martin's notation:** a close cousin of crow's foot, also using the crow's-foot marks for "many"; the paper sometimes says "the Martin's ERD notation style."
- ▶ **Chen notation:** the original style. Entities are rectangles, but each **relationship is a diamond** on the line, and **attributes are ovals** hung off the entity. Cardinality is written as 1, M or N beside the line rather than as symbols.

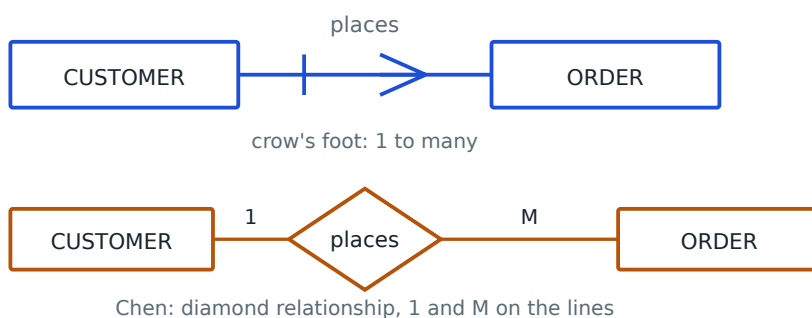


Figure 4.2 · The same 1:M relationship in crow's-foot (top) and Chen (bottom) notation. Chen puts the relationship in a diamond and writes the numbers on the lines; crow's foot puts marks at the ends.

Take the patient billing scenario: *a patient must be assigned to only one doctor, but a doctor can have many patients; a patient can have only one insurance ID.* Draw the ERD.

- 1 Find the entities (the nouns kept as records).
PATIENT, DOCTOR, INSURANCE. Three rectangles.
- 2 Find the relationships (the verbs).
A doctor treats patients; a patient has insurance. Draw a line for each with the verb on it.
- 3 Work out the cardinality of each, both ways.
"One doctor, many patients" and "each patient, one doctor" gives DOCTOR 1:M PATIENT. "A patient has only one insurance" gives PATIENT 1:1 INSURANCE (or M:1 if insurers are shared, but the fraud rule fixes it at one each).
- 4 Put the crow's-foot marks on.
On the DOCTOR-PATIENT line: one bar at DOCTOR (one), a crow's foot at PATIENT (many). On PATIENT-INSURANCE: a bar at each end (one to one).
- 5 Choose the keys.
patientId (PK) in PATIENT, doctorId (PK) in DOCTOR; PATIENT holds doctorId and insuranceId as foreign keys (FK) to record its relationships.

Now you try: KHMS schools have one or more annexes; each annex has one or more teachers, but a teacher is assigned to one annex; each teacher coordinates one or more classes, but a class is coordinated by one teacher. Draw the ERD, marking every cardinality.

TEACH

READING AN ERD: FORWARD AND BACKWARD CHAINING

The 25/26 paper prints an ERD and asks you to "read and interpret it using forward and backward chaining", and to read the cardinal relationships between the entities. Reading an ERD means turning its symbols back into English sentences, one relationship at a time, in both directions:

- ▶ **Forward chaining** reads the relationship in the direction of the verb, from the first entity to the second: "a CUSTOMER *places* zero or many ORDERs."
- ▶ **Backward chaining** reads it the other way: "an ORDER *is placed by* one and only one CUSTOMER."

Do this for every line and you have interpreted the whole diagram. The cardinality you report for each relationship is exactly what the crow's-foot marks say: look at the mark nearest each entity, translate it (bar = one, foot = many, circle = optional), and write the sentence.

⚠ TRAP

M:N NEEDS A LINK, AND THE VERB BELONGS ON THE LINE

Two ERD mistakes cost marks. First, a **many-to-many** relationship that carries its own data (a STUDENT takes a COURSE *with a grade*) cannot store that data on either entity: the grade belongs to the pairing. In logical design this becomes a **linking (associative) entity** between them, turning one M:N into two 1:M. Second, the **relationship name is a verb and sits on the line**, not inside a box; leaving a line unlabelled, or writing a noun on it, loses the reading marks.

1. Give the three cardinal relationships and an example of each.
2. What is the difference between cardinality and ordinality?
3. In crow's foot, what do a bar, a circle and a crow's foot each mean?

ANSWERS

1. One-to-one (employee manages department), one-to-many (customer places orders), many-to-many (student takes courses).
2. Cardinality is the maximum (one or many); ordinality is the minimum, whether the relationship is mandatory (at least one) or optional (zero allowed).
3. Bar = one, circle = zero (optional), crow's foot = many.

◆ KEY POINT

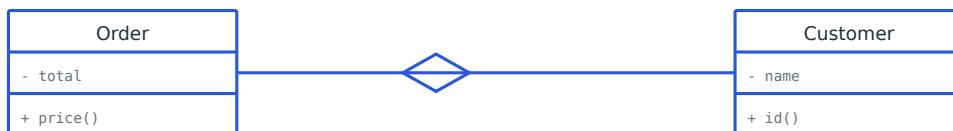
The marks at the end of a relationship line describe the entity at the OTHER end, and each end carries two of them: the inner mark gives the maximum and the outer mark gives the minimum. A crow's foot is many, a bar is one, a circle is zero. Read every line out loud in both directions before you commit to it.

BLUNT VERSION: read the far end, and say the sentence both ways round. Most lost ERD marks are a correct diagram read backwards.

MODULE FIVE

Object-Oriented Modelling with UML

The last family of diagrams, and the richest source of Section B marks outside the ERD. This module teaches the Unified Modeling Language: the class diagram and its relationships, the object-oriented concepts the examiner names every year, and the two behavioural diagrams, activity and sequence. All of it is in the course text; this module makes it drawable under exam time.



UML and Class Diagrams

By the end of this unit you can say what UML is and who created it, sort UML diagrams into the two categories, draw a class with its attributes and methods, show relationships and multiplicities, and tell an object from a class.

TEACH

WHAT UML IS

The **Unified Modeling Language (UML)** is a standard set of diagram notations for modelling object-oriented systems. It was created in the 1990s by **Grady Booch, James Rumbaugh and Ivar Jacobson**, often called the "three amigos", who merged their separate methods into one, and it is maintained by the Object Management Group. Its uses: to visualise, specify, construct and document the parts of a software system, so that analysts, designers and programmers share one language.

The paper asks you to "write short notes on UML, stating its inventors and its uses." That is the paragraph above: what it is, the three inventors, and the four uses (visualise, specify, construct, document).

MUST MEMORISE

THE TWO UML CATEGORIES

The paper asks you to "categorise UML diagrams into two broad categories, mention two diagrams in each, and describe their function." The two categories are:

- ▶ **Structural diagrams** show the static structure, the things that make up the system and how they are arranged. Two examples: the **class diagram** (classes and their relationships) and the **object diagram** (objects at a moment in time). Their function is to model what the system *is*.
- ▶ **Behavioural diagrams** show the dynamic behaviour, how the system acts over time. Two examples: the **use case diagram** (what users can do) and the **activity diagram** (the flow of a process); the **sequence diagram** is a third. Their function is to model what the system *does*.

Memory hook: structure is a noun, behaviour is a verb. Class and object diagrams are nouns (things); use case, activity and sequence diagrams are verbs (happenings).

TEACH

OBJECT VERSUS CLASS

"Differentiate between an object and a class" is a standard 2-mark question. A **class** is a blueprint or template that defines the attributes and methods a kind of thing has; an **object** is a specific instance created from that class, with real values. *Car* is a class; *your particular car*, with its registration and colour, is an object. One class, many objects. This idea comes from object-oriented programming (OOP), where a program is built from objects that hold data and behaviour together.

A **class** is drawn as a rectangle in **three compartments**: the class **name** at the top, its **attributes** (data) in the middle, and its **methods** or operations (behaviour) at the bottom. A **visibility** mark sits before each member: **minus** for private (hidden inside the class), **plus** for public (visible outside). The paper asks you to "draw a class with at least two attributes and two methods", so practise the box until it is automatic.

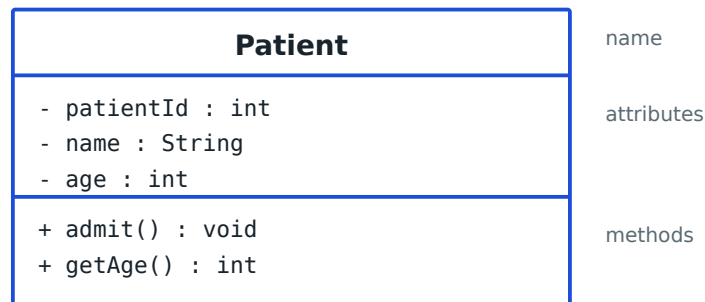


Figure 5.1 · A class box. Name, then attributes, then methods. Minus is private, plus is public. This is the unit of a class diagram.

Classes are joined by relationships, each with a different line-end, and each line carries a **multiplicity** saying how many objects take part. Multiplicity is the class-diagram version of cardinality: **1** (exactly one), **0..1** (optional), ***** or **0..*** (many), **1..*** (one or more). The relationships:

- ▶ **Association** (plain line): the classes are connected. A ward *has* patients.
- ▶ **Aggregation** (**hollow diamond** at the whole): a "has-a" where the part can exist without the whole. A department aggregates lecturers; the lecturers survive if the department closes.
- ▶ **Composition** (**filled diamond** at the whole): a stronger "has-a" where the part cannot exist without the whole. A ward is composed of a convenience room; no ward, no room.
- ▶ **Generalization** / inheritance (**hollow triangle** at the parent): an "is-a". A new patient and an old patient are both a patient.

Memory hook: hollow diamond is a loose has-a, filled diamond is a life-or-death has-a, hollow triangle is an is-a.

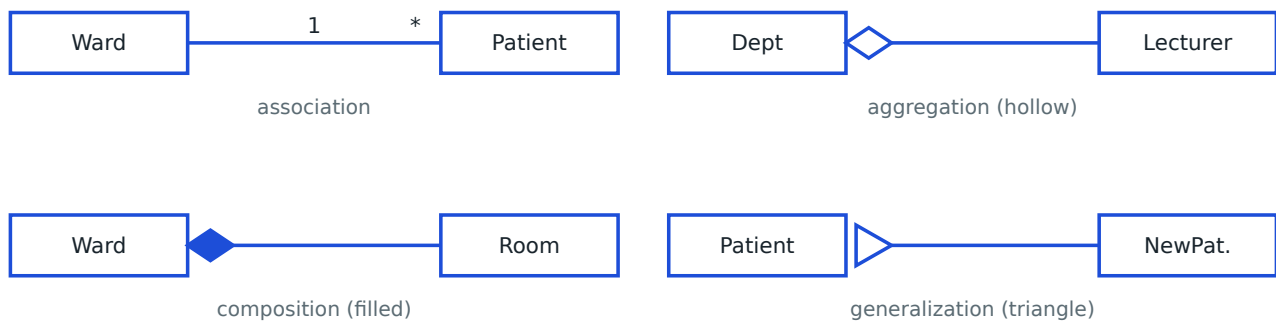


Figure 5.2 · The four class relationships. Hollow diamond = aggregation, filled diamond = composition, hollow triangle = generalization, plain line = association. The 1 and * are multiplicities.

WORKED

A CLASS DIAGRAM FROM A SCENARIO

Take the hospital scenario: *a patient is made up of one patient detail (Pid, name, age, gender); a patient is either a new patient or an old patient; a ward holds at least 8 patients; a ward is made up of one ward detail (WardNo, No_of_Bed, WType); a convenience room is part of a ward; a ward has at least two nurses.* Draw the class diagram with relationships and multiplicities.

- 1 Make a class for each noun.
Patient, PatientDetail, NewPatient, OldPatient, Ward, WardDetail, ConvenienceRoom, Nurse. Give each its attributes.
- 2 "Is made up of one detail" is composition.
Patient has a filled diamond to PatientDetail (1); Ward has a filled diamond to WardDetail (1). The detail cannot exist without its owner.
- 3 "Either a new or an old patient" is generalization.
NewPatient and OldPatient both point with a hollow triangle to Patient.
- 4 "A ward holds at least 8 patients", "at least two nurses" are associations with multiplicity.
Ward to Patient marked 1 on the ward end and 8.. on the patient end; Ward to Nurse marked 2..*.*
- 5 "A convenience room is part of a ward" is composition or aggregation.
"Part of" a ward, and it makes no sense without the ward, so a filled diamond from Ward to ConvenienceRoom.

Now you try: draw the class diagram for an order system where an order is made up of three order items, an order is either a special order or a normal order, and a customer places at most 20 orders. Which relationship is each?

1. Who invented UML, and name the two diagram categories.
2. What are the three compartments of a class box?
3. Which line-end means composition, and what does composition imply?

ANSWERS

1. Booch, Rumbaugh and Jacobson; structural and behavioural diagrams.
2. Name, attributes, methods.
3. A filled diamond at the whole; the part cannot exist without the whole.

◆ KEY POINT

A class box has three compartments in a fixed order: the name, then the attributes, then the operations. The lines between boxes carry the meaning, and their multiplicities sit at the end NEAR the class they describe, so a class diagram is read the same way an entity-relationship diagram is: across the line, in both directions.

BLUNT VERSION: the class box is name, has, does. Never in another order.

MODULE FIVE · UNIT 2

Object-Oriented Design Concepts

By the end of this unit you can explain encapsulation, inheritance, polymorphism, composition, aggregation and association with an example each, and state the drawbacks of multiple inheritance.

ANCHOR

A car and its wheels: take the wheels off and both the car and the wheels still exist, so that is a hollow diamond. A house and its rooms: demolish the house and the rooms are gone with it, so that is a filled diamond. The two symbols differ by whether the middle is coloured in, and they mean opposite things about what survives what.

"Briefly describe the following object-oriented design concepts with relevant examples" is on almost every paper. Learn one clean sentence and one example for each:

- ▶ **Encapsulation:** bundling data and the methods that act on it inside one class, and hiding the data behind those methods (private attributes, public methods). Example: a BankAccount hides its balance and exposes deposit() and withdraw().
- ▶ **Inheritance:** a class (the child or subclass) takes on the attributes and methods of another (the parent or superclass) and can add its own. Example: a SavingsAccount inherits from Account and adds interest.
- ▶ **Polymorphism:** the same method name behaves differently on different classes. Example: draw() on a Circle and draw() on a Square both draw, each in its own way.
- ▶ **Composition:** an object is built from other objects that cannot exist without it (a strong "has-a"). Example: a House is composed of Rooms.
- ▶ **Aggregation:** an object is built from other objects that can exist independently (a weak "has-a"). Example: a Team aggregates Players, who exist without the team.
- ▶ **Association:** two classes are related and can use each other, but neither owns the other. Example: a Doctor treats a Patient.

● MUST MEMORISE

COMPOSITION VS AGGREGATION

The examiner loves to test composition against aggregation because they look almost the same. The one distinction: **composition is a life-or-death has-a** (the part dies with the whole, filled diamond); **aggregation is a loose has-a** (the part lives on, hollow diamond). House and Rooms is composition; Team and Players is aggregation.

Memory hook: break a House and the Rooms go with it (composition); disband a Team and the Players walk away (aggregation).

Multiple inheritance is when one class inherits from **more than one** parent at once. It is powerful but the paper asks for its **drawbacks** (worth 2.5 to 3.5 marks), so learn three:

- ▶ **The diamond problem (ambiguity):** if two parents both provide a method with the same name, the child does not know which to inherit.
- ▶ **Increased complexity:** the class hierarchy is harder to understand, design and maintain.
- ▶ **Name clashes:** attributes or methods with the same name in different parents collide, and resolving them is error-prone.

This is why several modern languages (such as Java) forbid multiple inheritance of classes and offer interfaces instead.

1. Define encapsulation and give an example.
2. What is polymorphism?
3. State two drawbacks of multiple inheritance.

ANSWERS

1. Bundling data with the methods that use it and hiding the data behind them; a BankAccount hides its balance and exposes deposit() and withdraw().
2. The same method name behaving differently on different classes (draw() on a Circle versus a Square).
3. Any two of: the diamond problem (ambiguity), increased complexity, name clashes.

◆ KEY POINT

Aggregation and composition are both whole to part, and they differ on LIFETIME alone. If the part can outlive the whole it is aggregation, drawn with a hollow diamond; if the part dies with the whole it is composition, drawn with a filled one. Ownership, importance and containment are not the test.

BLUNT VERSION: the test is what survives, not what owns. Hollow means the part lives on.

MODULE FIVE · UNIT 3

Activity and Sequence Diagrams

By the end of this unit you can draw an activity diagram of a process flow and a sequence diagram of an interaction over time, the two behavioural diagrams the paper asks for by name.

TEACH

THE ACTIVITY DIAGRAM

An **activity diagram** is a flowchart of a process: it shows the order in which actions happen, including choices and parallel work. Its symbols:

- ▶ A **filled circle** for the start, and a filled circle in a ring for the end.
- ▶ A **rounded rectangle** for each action ("Check patient record").
- ▶ A **diamond** for a decision, with the condition on each outgoing arrow ("current patient?" splitting into yes and no).
- ▶ A **solid bar** (fork and join) where flows split to run in parallel and merge again.
- ▶ Arrows for the flow of control from one action to the next.

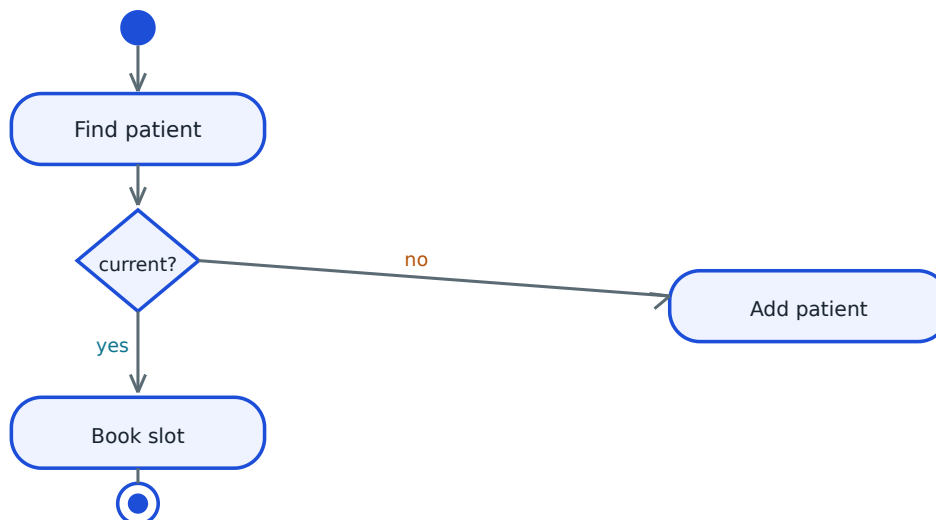


Figure 5.3 · An activity diagram. Filled circle starts it, rounded rectangles are actions, the diamond is a decision with a labelled branch on each arrow, and the ringed circle ends it.

TEACH

THE SEQUENCE DIAGRAM

A **sequence diagram** shows how objects interact *over time*, message by message. Time runs down the page. Its parts:

- ▶ A box at the top for each participating object or actor, with a dashed **lifeline** hanging down from it.
- ▶ A thin **activation bar** on a lifeline while that object is doing something.
- ▶ A solid arrow for a **message** (a call) from one lifeline to another, labelled with the operation; a dashed arrow for the **return**.

You draw one when a question says "draw a sequence diagram for the interaction", for example a sales representative entering an order of up to twelve products: the messages go from the rep to the system and back, one per product.

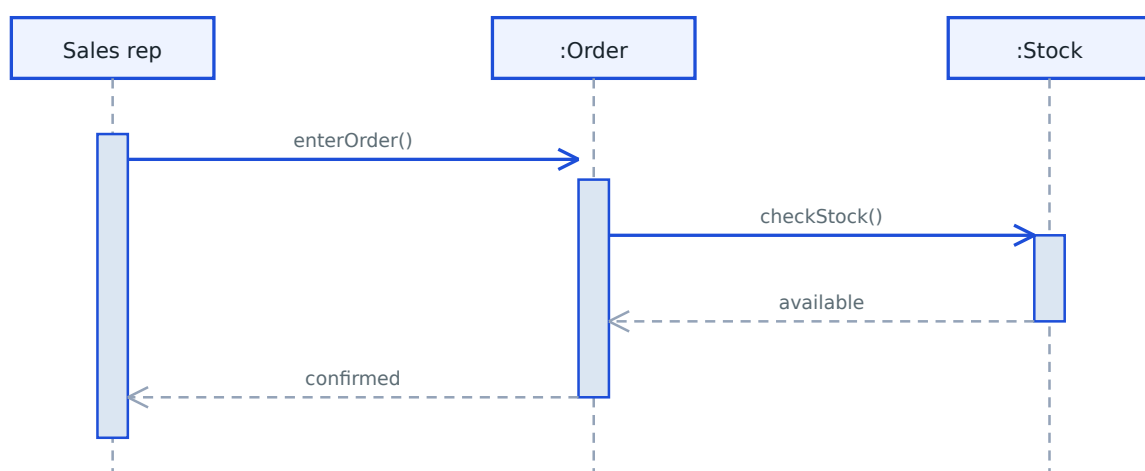


Figure 5.4 · A sequence diagram. Lifelines hang from each object, time runs downward, solid arrows are calls, dashed arrows are returns, and the thin bars show when each object is active.

1. What does a diamond mean in an activity diagram?
 2. In a sequence diagram, which way does time run, and what is a lifeline?
 3. Is a sequence diagram structural or behavioural?
-

ANSWERS

1. A decision, with a condition labelling each outgoing branch.
2. Time runs down the page; a lifeline is the dashed vertical line showing an object's existence through time.
3. Behavioural (it shows behaviour over time).

◆ KEY POINT

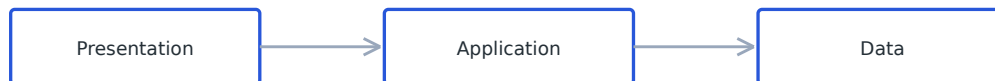
The two behavioural diagrams answer different questions. An activity diagram shows the **FLOW** of a process: what happens next, where it branches, what runs in parallel, and who does it if there are swimlanes. A sequence diagram shows an **INTERACTION** over time: which participant sends which message to which other, in what order, with time running down the page.

BLUNT VERSION: activity answers what happens next. Sequence answers who says what to whom, and when.

MODULE SIX

Design, Architecture and Delivery

The Design and Implementation phases in the detail the paper asks for: how a system is structured across tiers, how you decide whether to build or buy it, how it is installed and supported, and how the project around it is managed. These are the marks in Section A and the last parts of the Section B scenario questions.



System Design and Architecture

By the end of this unit you can describe the client-server tiers, name the functions software architecture must provide and the non-functional requirements that shape hardware, state interface design principles, choose a system acquisition strategy, and design and populate a small relational database.

TEACH

CLIENT-SERVER TIERS

NOT IN THE COURSE TEXT A system's work can be split into three logical parts: **presentation** (the screens the user sees), **application** logic (the processing and rules), and **data** (storage and access). How these are spread across machines gives the **client-server tiers**, and the paper asks you to "explain the various client-server tiers to convince your boss":

- ▶ **Single-tier:** all three parts on one machine (a mainframe or a standalone app). Simple, but does not scale.
- ▶ **Two-tier:** a client holds the presentation (and often the application logic) and talks to a server that holds the data. The classic client and database server.
- ▶ **Three-tier:** presentation on the client, application logic on a separate middle (application) server, and data on a database server. Each tier can be scaled and maintained on its own; this is the common web architecture.
- ▶ **N-tier (multi-tier):** the middle tier is split further into more specialised servers. Most scalable, most complex.

Sell the three-tier to the boss on scalability and maintainability: because the tiers are separate, you can grow or replace one without disturbing the others.

TEACH

SOFTWARE ARCHITECTURAL FUNCTIONS

"Mention the software architectural functions the system will need to perform properly." Software architecture must provide four logical functions, which map onto the tiers above:

- ▶ **Data storage:** holding the data (the database).
- ▶ **Data access logic:** the processing needed to read and write that data (the queries).
- ▶ **Application logic:** the business rules and processing.
- ▶ **Presentation logic:** displaying information to the user and accepting their input.

● MUST MEMORISE

HARDWARE NON-FUNCTIONAL REQUIREMENTS

"The choice of hardware architecture depends on four non-functional requirements; the first is operational, name two others." The four categories of non-functional requirement that shape the hardware are:

- ▶ **Operational requirements:** the environment the system must run in (operating system, network, hardware it must coexist with).
- ▶ **Performance requirements:** speed, capacity, throughput and availability.
- ▶ **Security requirements:** access control, encryption, protection of data.
- ▶ **Cultural and political requirements:** local laws, languages, customs and standards the system must respect.

Memory hook: O P S C, "Operational, Performance, Security, Cultural". These are the non-functional "-ilities" of Module Two, grouped for choosing hardware.

TEACH

INTERFACE DESIGN PRINCIPLES

Interface design decides how the system interacts with the outside world, chiefly the graphical user interface (GUI) the user works through. "Outline three principles of interface design":

- ▶ **Consistency:** the same action looks and works the same way everywhere in the system, so the user can transfer what they learn.
- ▶ **Simplicity (and clarity):** show only what is needed, in plain layout and language, so the interface is easy to learn and use.
- ▶ **User feedback:** the system tells the user what is happening, confirms actions, and reports errors clearly.

Other acceptable principles: minimise user effort, prevent and handle errors gracefully, and respect the user's control.

● MUST MEMORISE

SYSTEM ACQUISITION STRATEGY

When a system is needed but the team lacks the time or skill to build it, you choose a **system acquisition strategy**. The three options, and the one to pick under time pressure:

- ▶ **Build (custom development):** write it in-house. Best fit, most control, but slowest and needs the skills.
- ▶ **Buy (a packaged or off-the-shelf solution):** purchase ready-made software. Fast and cheaper, but may not fit exactly.
- ▶ **Outsource:** pay another company to build or run it. Good when you lack the skill or capacity in-house.

When a scenario says "the time frame is short and the project skills are not strategic", the answer is to **buy a packaged solution** or **outsource**, not to build. Supporting factors to cite: cost, time available, in-house skills, how well packages fit the need, and how strategic (core to the business) the system is.

The paper asks you to "design and populate a relational database of Customer, Order and Payment, with primary keys, at least three columns and five rows." A relational database stores data in **tables**; each table has a **primary key (PK)** column that uniquely identifies a row, and a **foreign key (FK)** to link to another table.

CUSTOMER

CUSTID (PK)	NAME	PHONE
C01	Ada Obi	0803...
C02	Bola Ade	0806...
C03	Chidi Eze	0701...
C04	Dele Ojo	0809...
C05	Efe Uche	0812...

ORDER

ORDERID (PK)	CUSTID (FK)	TOTAL
O01	C01	15000
O02	C03	8000
O03	C01	22000
O04	C05	5000
O05	C02	13000

The Payment table follows the same shape: payId (PK), orderId (FK), amount, with five rows. Notice custId is the primary key of Customer and a foreign key in Order: that FK is what records which customer placed each order.

Now you try: add the Payment table with five rows, and say which column links it back to the Order table.

1. In a three-tier architecture, what sits in each tier?
2. Name the four non-functional requirements that shape hardware choice.
3. Time is short and the skills are not strategic. Which acquisition strategy?

ANSWERS

1. Presentation on the client, application logic on a middle server, data on a database server.
2. Operational, performance, security, cultural/political.
3. Buy a packaged solution, or outsource; do not build in-house.

Project Management, Installation and Support

By the end of this unit you can list the project planning activities, decide whether to accept new features, give the reasons projects succeed or fail, name the stakeholders and their views, and describe installation and support.

● MUST MEMORISE

PROJECT PLANNING ACTIVITIES

"Identify the five activities in project planning" (the Planning phase of Module One, as a delivery task):

1. Define the problem (the business problem and scope).
2. Produce the project schedule.
3. Confirm project feasibility (technical, economic, operational, schedule, legal).
4. Staff the project (allocate funds, assign members, obtain resources).
5. Launch the project.

TEACH

SCOPE CREEP AND FEATURE CREEP

Near the end of a project a client often asks for new features. **Scope creep** is uncontrolled growth of what the project must deliver; **feature creep** is the same at the level of individual features. The paper asks: as project manager, do you agree or refuse to add new features late? The exam-safe answer is to **refuse (or defer)**, and give reasons:

- ▶ It threatens the schedule and budget already agreed.
- ▶ It risks the quality and stability of what is nearly finished.
- ▶ New features belong in a change request or a later release, assessed properly, not bolted on under time pressure.

This is the "do not be afraid to cancel or revise scope" principle from Module One in action.

TEACH

WHY PROJECTS SUCCEED OR FAIL

"Enumerate reasons a project will succeed and reasons it can fail." The 25/26 paper gives you the first of each: a system succeeds when it is **acceptable to the customer** and fails if it **cannot establish upper-management commitment**. Others:

- ▶ **Succeed:** clear requirements, strong user involvement, realistic schedule and budget, good project management, management support.
- ▶ **Fail:** unclear or changing requirements, lack of user involvement, poor planning and estimation, weak management support, inadequate testing.

A **stakeholder** is anyone with an interest in the system. "Name three system stakeholders and describe how each views the system" wants you to show that different people want different things:

- ▶ **Users** want the system easy to use and helpful to their daily work.
- ▶ **Managers** want it to deliver business value on time and within budget.
- ▶ **Developers** (the technical team) want clear requirements and a system that is feasible to build and maintain.

Others: owners, customers, and regulators. The point the examiner wants is that these views can conflict, and the analyst must balance them.

Three loose ends the paper picks up:

- ▶ **Other job roles** an analyst can grow into: project manager, software architect, software tester, product designer, business analyst.
- ▶ **Installation (conversion)** uses one of the four strategies from Module One: direct cutover, parallel, phased, pilot. Choose by weighing risk against cost.
- ▶ **Support activities** (the Support phase): **maintain** the system (fix bugs, minor adjustments), **enhance** it (add capability as the business changes), and **support the users**, which includes running a help desk. "What three activities will the support group participate in?" is exactly these three.

Finally, the paper asks how the analyst explains to stakeholders what a **model** is, given that "a model is a representation of an important aspect of the real world." Explain it plainly: a model is a simplified picture of the system that leaves out detail so everyone can see and agree the important parts before it is built, the way an architect's drawing lets a client approve a building before a brick is laid. The diagrams throughout this manual, the use case, the DFD, the ERD, the class diagram, are all models in exactly this sense.

- 1. List the five project planning activities.**
 - 2. A client asks for new features near the deadline. What do you do, and why?**
 - 3. Name the three support-phase activities.**
-

ANSWERS

1. Define the problem, produce the schedule, confirm feasibility, staff the project, launch it.
2. Refuse or defer them, because they threaten the agreed schedule, budget and quality; handle them as a change request or later release.
3. Maintain the system, enhance the system, support the users (help desk).

REFERENCE

The Notation Legend and Must-Memorise List

Everything the examiner expects you to know cold, on a few pages, for the last hour before the hall. If you can reproduce the symbols on the next page and recite the lists after it from memory, you can answer any question on the paper.



The Notation Legend

Every symbol in the course on one page. Cover the right column and name each symbol; cover the left and draw each one.

● MUST MEMORISE

EVERY SYMBOL, ONE PAGE

DIAGRAM	SYMBOL	MEANS
Use case	stick figure	actor (a role outside the system)
Use case	oval	a use case (a verb phrase)
Use case	solid line	association (actor takes part)
Use case	dashed arrow, include	always uses (points to the included case)
Use case	dashed arrow, extend	optionally adds (points to the base case)
DFD	rectangle	external entity
DFD	numbered circle	process (a verb)
DFD	open-ended rectangle	data store (a noun, D1)
DFD	arrow	data flow (the data)
ERD	rectangle	entity
ERD	bar on line	one
ERD	circle on line	zero (optional)
ERD	crow's foot	many
ERD (Chen)	diamond	relationship
Class	three-part box	class: name / attributes / methods
Class	minus / plus	private / public visibility
Class	hollow diamond	aggregation (loose has-a)
Class	filled diamond	composition (life-or-death has-a)
Class	hollow triangle	generalization (is-a)
Activity	filled circle / ringed circle	start / end
Activity	rounded rectangle	action
Activity	diamond	decision (condition on each branch)
Sequence	dashed vertical line	lifeline (time runs down)
Sequence	solid / dashed arrow	message (call) / return

Four pairs are almost identical and each is a lost mark if confused. Fix them now:

- ▶ **Hollow vs filled diamond** (class): hollow is aggregation (part survives), filled is composition (part dies with the whole).
- ▶ **Include vs extend arrow** (use case): include points to the step always run; extend points to the base it might add to.
- ▶ **Bar vs crow's foot** (ERD): bar is one, crow's foot is many; the circle beside either makes it optional.
- ▶ **Closed box vs open-ended rectangle**: a closed rectangle is an entity or an external entity; the data store is open at the right.

REFERENCE • TWO

The Must-Memorise List

The lists the examiner asks you to reproduce. Recite each one from memory, then check. If you know these, Section A is done and Section B has no surprises.

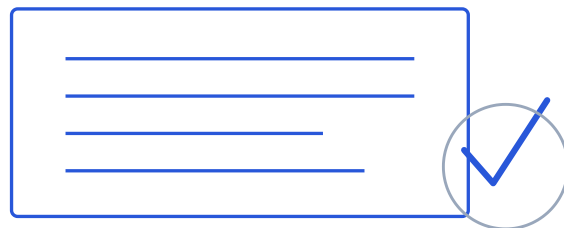
- ▶ **SDLC phases (PADIS):** Planning, Analysis, Design, Implementation, Support.
- ▶ **Analyst's skills:** technical, business, people (people is the most important).
- ▶ **Problem-solving sequence:** understand, verify benefit > cost, define requirements, alternatives, recommend, detail, implement, monitor.
- ▶ **Feasibility (TEOSL):** Technical, Economic, Operational, Schedule, Legal.
- ▶ **Elicitation techniques:** interview, questionnaire, observation, document review, JAD, prototyping.
- ▶ **Requirement types:** functional (what it does) vs non-functional (a quality); and business / user / system levels.
- ▶ **Use case relationships:** association, include, extend, generalization.
- ▶ **DFD elements:** external entity, process, data store, data flow.
- ▶ **DFD mechanical errors:** black hole (no output), miracle (no input), grey hole (too little input).
- ▶ **Cardinal relationships:** one-to-one, one-to-many, many-to-many.
- ▶ **Object-oriented concepts:** encapsulation, inheritance, polymorphism, composition, aggregation, association.
- ▶ **Multiple inheritance drawbacks:** the diamond problem, complexity, name clashes.
- ▶ **UML categories:** structural (class, object) and behavioural (use case, activity, sequence). Inventors: Booch, Rumbaugh, Jacobson.
- ▶ **Client-server tiers:** single, two, three, n-tier.
- ▶ **Hardware non-functional requirements (OPSC):** operational, performance, security, cultural/political.
- ▶ **Acquisition strategies:** build, buy, outsource.
- ▶ **Conversion strategies:** direct cutover, parallel, phased, pilot.
- ▶ **Support activities:** maintain, enhance, support the users (help desk).

- ▶ **Analyst:** a business problem solver, the bridge between users and the technical team.
- ▶ **Model vs tool vs technique vs methodology:** a model represents the system; a tool is software you use; a technique is a way of doing one activity; a methodology is the whole package.
- ▶ **Predictive vs adaptive:** predictive plans everything upfront; adaptive plans as it goes in short iterations.
- ▶ **Cardinality vs ordinality:** cardinality is the maximum (one or many); ordinality is the minimum (mandatory or optional).
- ▶ **Object vs class:** a class is the blueprint; an object is an instance of it.
- ▶ **Composition vs aggregation:** composition the part dies with the whole; aggregation the part lives on.
- ▶ **Data model vs process model:** a data model (ERD) shows data at rest; a process model (DFD) shows data in motion.
- ▶ **Forward vs backward chaining:** read a relationship in the direction of the verb, then the reverse.

WORKED PAPER 2025/2026

The 2025/2026 Paper, Fully Worked

The most recent paper, the one your sitting will most resemble, quoted exactly as it was printed and then answered in full. Each question appears first in an As-printed box in the examiner's own words, then a Break-it-down box reads it in your shoes, then the answer. Cover the answer and attempt each question before you read on.



Section A: Questions One and Two

You answer one of these two in the hall. Both are worth working: they drill the ERD, the DFD, the class, use case and activity diagrams in one sitting.

AS PRINTED

PQ 25/26 Q1

A landlord owns one of many houses and asks the only estate agent he knows to help find a suitable tenant. The agent (who manages many houses) contacts his supervisor to inform him about the availability of the house. The supervisor indicates interest and agrees to rent the house.

After the agreement, the supervisor moves into the house with the help of the agent. Before occupation, they both clean the house. Once settled, the supervisor occupies the house and pays rent through the agent, who then forwards the payment to the landlord.

- a. Draw an ERD to model the relationships between the identified entities (4.5 mks)
- b. Data Flow Diagram (DFD):
 - i. Draw a Context diagram to model the entire system (1.5 mks)
 - ii. Draw a Level 0 DFD to show various processes involved (4.5 mks)
 - iii. Draw a Level 1 DFD to model the process of rent payment (3 mks)
- c. The resulting system is successful when it is acceptable to the customer and it will fail if it cannot establish upper-management commitment to the project. Enumerate (3) other reasons a project will succeed and (3) other reasons it can fail (3 mks)
- d. Read and interpret the following (using forward and backward chaining) (1 mk)

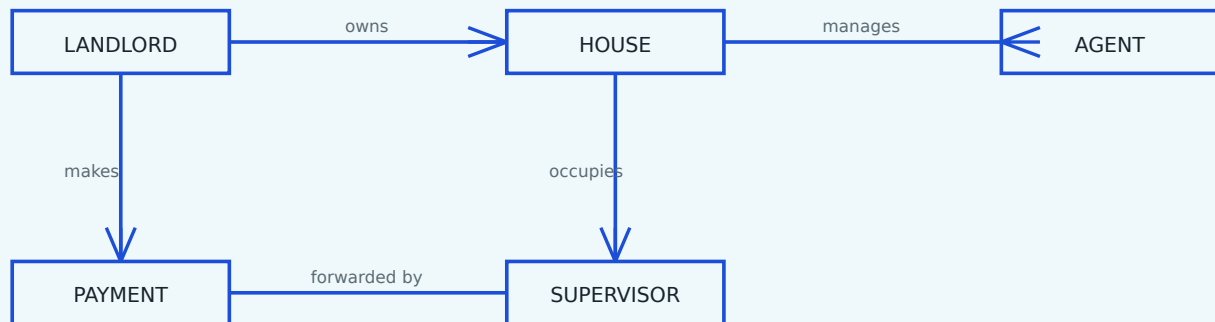
Under part d the paper prints a small crow's-foot fragment: a **Customer** box joined to an **Order** box, with a single bar at the Customer end and a circle-and-crow's-foot at the Order end. It is reproduced as our own figure in the answer to d.

BREAK IT DOWN

- GIVEN** A landlord, an estate agent, a supervisor (the tenant), houses, and rent payments that flow supervisor to agent to landlord.
- ASKED** Four separate things: an ERD (a), three DFDs (b), reasons a project succeeds and fails (c), and reading a printed ERD fragment (d). Parts a, b and d test Modules Three and Four; part c tests Module Six.
- MARKS** 17.5 in all. Part b is worth the most (9), so the three DFDs deserve the most time and care.
- WATCH** The examiner asks the ERD in no particular notation, so use the crow's foot. In the DFDs, keep every flow touching a process, and make the three levels balance.

A. ENTITY-RELATIONSHIP DIAGRAM (4.5 MARKS)

The nouns kept as records are LANDLORD, HOUSE, AGENT, SUPERVISOR (the tenant) and PAYMENT. The relationships from the story: a landlord *owns* many houses; an agent *manages* many houses; an agent *serves* a supervisor; a supervisor *occupies* a house; a supervisor *makes* payments; an agent *forwards* payments to the landlord.



One LANDLORD owns many HOUSEs; one AGENT manages many HOUSEs; one HOUSE is occupied by one SUPERVISOR; one SUPERVISOR makes many PAYMENTs, each forwarded by the AGENT to the LANDLORD.

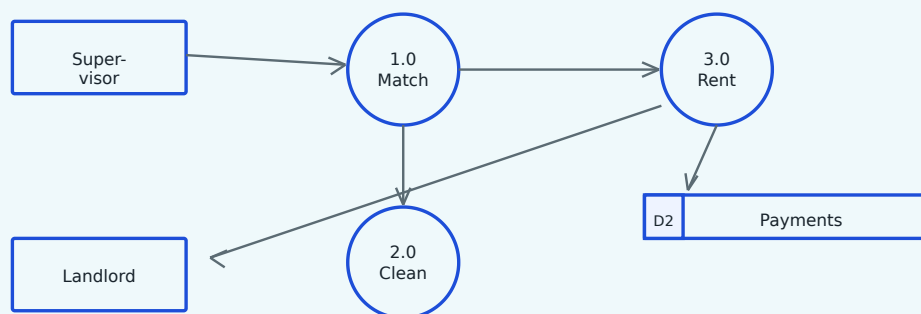
ANSWER, CONTINUED

B. DATA FLOW DIAGRAMS (9 MARKS)

Context diagram: one process "0 Rental system" in the middle, with SUPERVISOR and LANDLORD as external entities. Flows: supervisor sends "house request" and "rent payment" in; the system sends "house availability" back to the supervisor and "forwarded payment" to the landlord. No data stores at this level (draw it exactly as Figure 3.2 in Module Three, with these labels).

Level 0: explode process 0 into the major processes: 1.0 Match tenant to house, 2.0 Agree and clean, 3.0 Collect and forward rent. Add data stores D1 Houses and D2 Payments. Wire the flows so each process has an input and an output, and so the "house request" in and "forwarded payment" out still match the context diagram (balance).

Level 1 of process 3.0 (rent payment): explode 3.0 into 3.1 Receive payment from supervisor, 3.2 Record payment (to D2 Payments), 3.3 Forward payment to landlord. The total flows in and out (payment in, forwarded payment out) balance process 3.0.



The rental Level 0 DFD. Process 0 has exploded into 1.0, 2.0 and 3.0; the rent flows through 3.0 to the landlord and into D2 Payments; every flow touches a process, and the diagram balances the context.

ANSWER, CONTINUED

C. THREE MORE REASONS TO SUCCEED AND THREE TO FAIL (3 MARKS)

Succeed (besides customer acceptance): clear and agreed requirements; strong user involvement throughout; a realistic schedule and budget with good project management.

Fail (besides lack of upper-management commitment): unclear or constantly changing requirements; little or no user involvement; poor planning, estimation or testing.

D. READING THE FRAGMENT (1 MARK)

The bar at the Customer end means "one" and the circle-with-crow's-foot at the Order end means "zero or many". So, **forward chaining:** one CUSTOMER places zero or many ORDERS. **Backward chaining:** each ORDER is placed by one and only one CUSTOMER.

A patient visits a hospital to use the patient billing system in order to make enquiries about the types of drugs he can buy. He must have insurance or he won't be attended to by the Nurse. The nurse checks his vitals and refers him to a doctor who in turn refers him to the Insurance Carrier to confirm the patient has his insurance covered. On confirmation, the doctor advises the patient on which drugs to use for his condition.

Note that a patient must be assigned to only one doctor but a doctor can have many patients.

A patient can have only one insurance ID to prevent insurance fraud.

- a. Draw a class diagram for this scenario. [Hint: Show multiplicity where necessary] (5 mks)
- b. Draw a use case diagram for the system. (5 mks)
- c. Draw an activity diagram to show the system's process flow. (5 mks)
- d. State three (3) drawbacks of multiple inheritance. (2.5 mks)

BREAK IT DOWN

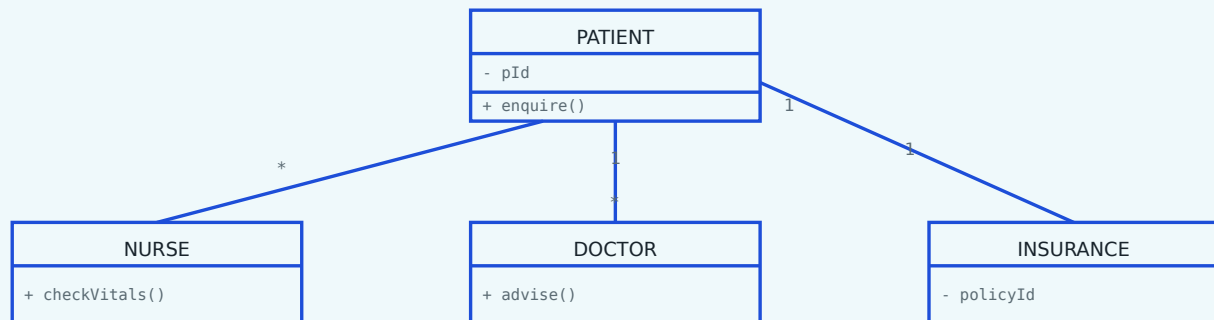
GIVEN A patient billing system with a Patient, a Nurse, a Doctor and an Insurance Carrier, and two firm rules: one doctor per patient (a doctor has many), one insurance ID per patient.

ASKED Three diagrams (class, use case, activity) and one written part (drawbacks of multiple inheritance). All four come straight from Module Five.

WATCH The class diagram must show the multiplicities the rules give you (Patient 1 to Doctor's many; Patient 1 to Insurance 1). Do not confuse the three diagrams: the class diagram is structural, the other two behavioural.

A. CLASS DIAGRAM (5 MARKS)

Four classes: PATIENT, NURSE, DOCTOR, INSURANCE. Give each attributes and a method. The rules give the multiplicities: a DOCTOR treats many PATIENTs but each PATIENT has one DOCTOR (1 to *); a PATIENT has one INSURANCE and an insurance covers one patient (1 to 1); a NURSE attends many PATIENTs (1 to *).



PATIENT to DOCTOR is many-to-one (a doctor has many patients, a patient one doctor); PATIENT to INSURANCE is one-to-one; NURSE attends many patients.

B. USE CASE DIAGRAM (5 MARKS)

Actors: PATIENT, NURSE, DOCTOR, INSURANCE CARRIER. Use cases inside the boundary: Make enquiry, Check vitals, Refer to doctor, Confirm insurance, Advise on drugs. "Attend patient" **includes** "Check insurance" (it must always happen, since he will not be attended without insurance). Draw it as Figure 2.1 in Module Two with these labels.

C. ACTIVITY DIAGRAM (5 MARKS)

Start, then action "Patient makes enquiry", then a decision "has insurance?": the "no" branch ends (not attended); the "yes" branch flows to "Nurse checks vitals", "Refer to doctor", "Doctor confirms insurance with carrier", "Advise on drugs", then end. Draw it as Figure 5.3 with this flow.

D. DRAWBACKS OF MULTIPLE INHERITANCE (2.5 MARKS)

The diamond problem (ambiguity when two parents give the same method); increased complexity of the class hierarchy; and name clashes between members inherited from different parents.

WORKED PAPER 2025/2026**Section B: Questions Three to Six**

You answer three of these four. Q3 and Q5 are written questions; Q4 mixes architecture with a database design; Q6 is requirements and UML. Practise all four.

Tikki Tikka Buttry and Pastries need to automate their food selling business in order to meet up with the demands of operating in Nigeria. They hired Billy Butcher as their analyst to guide them on the steps to take in order to make their automation a success, they also hired a graphic designer and a JavaScript programmer to join the team; they call themselves the Boys.

- a. Briefly explain the THREE questions that the Boys will need to answer during the Feasibility Analysis phase of their system planning (3 mks)
- b. Billy's first approach to problem solving is Research and Understand the Problem. Outline any other six (6) steps he can take (3 mks)
- c. A model is a representation of an important aspect of the real world. How can the Boys properly explain what a model is to the stakeholders? (4 mks)
- d. The most important skill Billy can have is People knowledge and skill; explain in detail any other skill he can possess to do a great job (3 mks)
- e. Billy needs to ensure there is adequate support for the users of the new system. What three activities will the support group need to participate in? (3 mks)
- f. Mention any three other job roles that Billy can fit into (1.5 mks)

BREAK IT DOWN

GIVEN A food business automating, with an analyst (Billy) and a small team.

ASKED Six short written parts, all straight recall from Modules One and Six: feasibility, the problem-solving steps, what a model is, the analyst's skills, support activities and job roles.

WATCH Each part names the exact list it wants. Answer the number asked (three questions, six steps, three activities, three roles) and no more.

A. THREE FEASIBILITY QUESTIONS (3 MARKS)

Feasibility analysis asks whether the project is worth pursuing, from three angles: **Is it technically feasible?** (do we have the technology and skill to build it?); **Is it economically feasible?** (do the benefits outweigh the costs?); and **Is it operationally feasible?** (will the users accept and use it, and does it fit the business?). Schedule and legal feasibility complete the five, but these three are the core questions.

B. SIX MORE PROBLEM-SOLVING STEPS (3 MARKS)

After researching and understanding the problem: verify the benefits outweigh the costs; define the requirements; develop alternative solutions; recommend the best solution; define the details of the chosen solution; implement it (and then monitor the results).

C. EXPLAINING WHAT A MODEL IS (4 MARKS)

Tell the stakeholders a model is a simplified picture of the system that leaves out detail so everyone can see and agree the important parts before it is built, the way an architect's drawing lets a client approve a building before a brick is laid. The use case diagram, DFD and class diagram they will be shown are all such models.

D. ANOTHER SKILL (3 MARKS)

Besides people skills, Billy needs **technical knowledge and skills**: an understanding of computers, networks, databases and the tools and techniques of system building, so he can understand and communicate technical requirements even though the programmer writes the code. (Business knowledge is an equally acceptable answer.)

E. THREE SUPPORT ACTIVITIES (3 MARKS)

Maintain the system (fix bugs and make minor adjustments); enhance the system (add capability as the business grows); and support the users, which includes running a help desk.

F. THREE OTHER JOB ROLES (1.5 MARKS)

Project manager, software architect, and product designer (also acceptable: software tester, business analyst).

Vought Industries have decided to acquire a Pharmaceutical Information System; the time frame is short and the project skills are not strategic

a. What System Acquisition Strategy fits these requirements? Enumerate (3) other factors that support your choice. (3 mks)

b. Mention any two (2) software architectural functions that Vought Industries will need in order to make the system perform properly. (3 mks)

c. The choice of hardware architecture depends on four (4) nonfunctional requirements, the first is operational requirements, enumerate any other two types of hardware requirements. (3 mks)

d. Interface design is the process of defining how the system will interact with external entities. Outline any three (3) principles of Interface design. (3 mks)

e. Design and populate a Relational Database consisting of Customer, Order and Payment Type for the Pharmaceutical System (use dummy names and values and ensure to include primary keys for all unique IDs. You should have a minimum of 3 columns and 5 rows) (5.5 mks)

BREAK IT DOWN

- GIVEN** A firm buying a system, under time pressure, with no strategic in-house skills. Those two clues decide part a.
- ASKED** The acquisition strategy and four short parts from Module Six Unit 1, then a small database to design and populate.
- WATCH** "Short time frame" and "skills not strategic" both point away from building. Part e wants real dummy rows, not a description: draw the tables.

A. ACQUISITION STRATEGY (3 MARKS)

Because the time frame is short and the skills are not strategic, **buy a packaged (off-the-shelf) solution, or outsource**, rather than build in-house. Supporting factors: the time available (buying is fastest); the in-house skills (they are lacking); and how strategic the system is (it is not core, so a package is fine); also cost and how well available packages fit the need.

B. TWO SOFTWARE ARCHITECTURAL FUNCTIONS (3 MARKS)

Any two of: data storage, data access logic, application (business) logic, and presentation logic.

C. TWO MORE HARDWARE NON-FUNCTIONAL REQUIREMENTS (3 MARKS)

Besides operational requirements: performance requirements and security requirements (the fourth is cultural and political requirements).

D. THREE INTERFACE DESIGN PRINCIPLES (3 MARKS)

Consistency (the same action works the same way everywhere); simplicity and clarity (show only what is needed); and user feedback (the system confirms actions and reports errors clearly).

E. DESIGN AND POPULATE THE DATABASE (5.5 MARKS)

CUSTID (PK)	NAME	PHONE
C01	Ada Obi	0803...
C02	Bola Ade	0806...
C03	Chidi Eze	0701...
C04	Dele Ojo	0809...
C05	Efe Uche	0812...

ORDERID (PK)	CUSTID (FK)	DRUG
O01	C01	Paracetamol
O02	C03	Amoxicillin
O03	C01	Vitamin C
O04	C05	Ibuprofen
O05	C02	Metformin

PAYID (PK)	ORDERID (FK)	TYPE
P01	O01	Cash
P02	O02	Card
P03	O03	Insurance
P04	O04	Card
P05	O05	Transfer

Each table has a primary key; custId links Order back to Customer and orderId links Payment Type back to Order, both as foreign keys.

AS PRINTED

PQ 25/26 Q5

A federal teaching hospital intends to replace its fragmented patient appointment, clinical record, laboratory request, and medication tracking systems with a single integrated Clinical Workflow and Patient Safety Management System. The current system is partly paper-based and partly supported by disconnected departmental applications. Doctors complain that patient records are incomplete, nurses report delays in medication administration, laboratory scientists complain about duplicated test requests, and hospital administrators are concerned about regulatory compliance, audit trails, and data privacy.

The hospital's board wants the system delivered within twelve months. However, stakeholders disagree on system priorities. Doctors want speed and flexibility, nurses want ease of use, laboratory staff want accurate workflow integration, and administrators insist on strict documentation, traceability, and reliability. Failure in the new system could affect patient safety.

a. Briefly compare seven (7) methodologies that can be used in this project. (7 mks)

[Hint: your answer should be short and concisely state why the methodologies may be suitable or not]

b. Identify five (5) activities in project planning. (5 mks)

c. Using the case, formulate at least:

- i. two (2) business requirements,
- ii. two (2) user requirements,
- iii. three (3) functional requirements,
- iv. two (2) non-functional requirements, and
- v. two (2) system requirements, (5.5 mks)

BREAK IT DOWN

GIVEN A hospital replacing fragmented systems, safety critical, twelve-month deadline, stakeholders who disagree.

ASKED Compare seven methodologies, list five planning activities, and formulate requirements at every level. Modules One, Two and Six.

WATCH Part a wants a *comparison* with a reason each is suitable or not for this safety-critical project, not just a list. Part c wants the exact counts across five levels.

A. COMPARE SEVEN METHODOLOGIES (7 MARKS)

- ▶ **Waterfall:** strong documentation and traceability suit a safety-critical system, but its rigidity is risky if requirements shift.
- ▶ **Prototyping:** good for the unclear priorities here, but the prototype can be mistaken for the finished safety system.
- ▶ **RAD:** fast, which suits the twelve-month deadline, but weak for a large, complex, safety-critical hospital system.
- ▶ **Spiral:** its risk analysis each cycle fits a safety-critical project well, but it is heavy and slow.
- ▶ **Incremental:** delivers working pieces early, useful given the deadline, but integration across departments must be managed.
- ▶ **Agile:** handles the disagreeing stakeholders through constant feedback, but its light documentation clashes with the audit-trail requirement.
- ▶ **V-shape:** pairs each stage with its test, excellent for the reliability and traceability a hospital demands, but as rigid as waterfall.

ANSWER, CONTINUED**B. FIVE PROJECT PLANNING ACTIVITIES (5 MARKS)**

Define the problem; produce the project schedule; confirm feasibility; staff the project; and launch the project.

C. FORMULATE REQUIREMENTS (5.5 MARKS)

- ▶ **Business:** cut medication-administration delays; achieve regulatory compliance with full audit trails.
- ▶ **User:** a nurse can administer and record medication at the bedside; a doctor can see a complete patient record in one place.
- ▶ **Functional:** the system shall record each patient appointment; the system shall flag duplicated laboratory test requests; the system shall log every change to a record for audit.
- ▶ **Non-functional:** the system shall be available 99.9% of the time; the system shall encrypt all patient data (privacy).
- ▶ **System:** the system shall store records as time-stamped entries linked to a patient ID; the system shall integrate with the existing departmental applications.

A private university wants to build an AI-supported programming learning platform for undergraduate students. The system should help students learn Python, Java, and C++ by providing coding exercises, automated feedback, debugging support, lecturer dashboards, plagiarism alerts, and adaptive learning recommendations. The university expects the system to support 8,000 students across three campuses.

At the start of the project, requirements are unclear. Students want instant answers to coding problems, lecturers want the system to promote independent thinking rather than answer copying, administrators want analytics on student performance, and the ICT unit is concerned about security, scalability, integration with the learning management system, and maintenance cost. Halfway through the project, university management requests additional features: AI-generated lecture notes, automatic grading of essays, and integration with the university payment portal.

- Identify the weaknesses of the as-is system, the expected improvements in the to-be system, and the assumptions that must be validated before development begins. (3.5 mks).
- Design a requirements elicitation plan for the system using at least four techniques. For each technique, identify the stakeholders involved, the type of information to be collected, and why the technique is suitable for this project. (4 mks).
- Write short notes on: Unified Modeling Language (UML) (also state the inventors and its uses). (2 mks)
- Categorise UML diagrams into two broad categories. Mention two diagrams in each of the categories and describe its function (4 mks)
- With the aid of diagrams, explain the following class relationships in system analysis: (i) Inheritance, (ii) Composition, (iii) Aggregation, and (iv) Association. (4 mks)

The paper writes "Categories UML diagrams" for "Categorise", and "describe its function" for "its function"; both are the paper's own slips, kept here as printed. Do not copy them into your answer.

BREAK IT DOWN

- GIVEN** An AI learning platform, unclear requirements, and mid-project feature requests (the scope-creep trap in disguise).
- ASKED** As-is/to-be analysis, an elicitation plan, and three straight-recall UML parts. Modules Two and Five.
- WATCH** Part a wants three lists (weaknesses, improvements, assumptions). Part b wants four techniques each with stakeholder, information and justification, not just a list of technique names.

A. AS-IS, TO-BE AND ASSUMPTIONS (3.5 MARKS)

As-is weaknesses: no single platform, so learning is unsupported; no automated feedback or plagiarism checking; no analytics for administrators. **To-be improvements:** one platform teaching three languages with automated feedback, debugging support, plagiarism alerts and analytics. **Assumptions to validate:** that 8,000 students across three campuses have reliable network access; that the platform can integrate with the existing learning management system; that the AI feedback is accurate enough to be trusted.

B. ELICITATION PLAN (4 MARKS)

- ▶ **Interview** lecturers: information on the learning outcomes and marking rules; suitable because they hold the academic requirements and can be probed in depth.
- ▶ **Questionnaire** to students: information on what help they want; suitable because it cheaply reaches 8,000 scattered students.
- ▶ **Observation** of a lab session: information on how students really code and get stuck; suitable because it shows real behaviour.
- ▶ **Document review** of the learning management system: information on existing data and integration points; suitable because it grounds the design in what exists.

C. SHORT NOTES ON UML (2 MARKS)

UML is a standard set of diagram notations for modelling object-oriented systems, created by Booch, Rumbaugh and Jacobson. Its uses: to visualise, specify, construct and document a software system in one shared language.

D. TWO CATEGORIES OF UML DIAGRAM (4 MARKS)

Structural diagrams model what the system is: the class diagram (classes and relationships) and the object diagram (objects at a moment). **Behavioural** diagrams model what it does: the use case diagram (what users can do) and the activity diagram (a process flow); the sequence diagram is another.

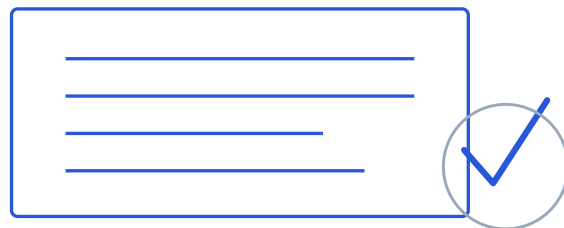
E. FOUR CLASS RELATIONSHIPS (4 MARKS)

Inheritance (hollow triangle, is-a): a Postgraduate is a Student. **Composition** (filled diamond, life-or-death has-a): an Order is made of Order Lines. **Aggregation** (hollow diamond, loose has-a): a Department has Lecturers who survive it. **Association** (plain line): a Doctor treats a Patient. Draw the four side by side as Figure 5.2 in Module Five.

WORKED PAPER 2024/2025

The 2024/2025 Paper, Fully Worked

The paper from the session before the target, in the same three-hour, one-from-A and three-from-B shape. Working it after the 2025/2026 paper shows you how stable the examiner's habits are: the same scenarios (patient appointments, sales management, the hospital ward) and the same diagram set come round again and again.



Section A: Questions One and Two

The two written Section A questions: requirements and methodology, then the life cycle and its stakeholders. Answer one in the hall; work both here.

AS PRINTED

PQ 24/25 Q1

As a systems analyst working on a new library management system, you are tasked with gathering system requirements from various stakeholders.

a. Identify and briefly explain two appropriate requirement elicitation techniques you would use. For each technique, justify why it is suitable for this project and specify one advantage and one potential limitation. (4 Marks)

b. Based on your selected elicitation techniques, outline:

i. Three functional requirements. (3 Marks)

ii. Three non-functional requirements. (3 Marks)

iii. List one reason why requirements elicitation should involve users from different organizational levels. (0.5 Mark)

c. You are the chosen project manager for a new line of clothing about to be opened. You very much need the project to succeed and are willing to give it your all.

i. The requirements of this work MAY NOT be clearly stated at the beginning. Choose an appropriate System Development Lifecycle (SDLC) methodology and map out the phases of the project. (4 Marks)

ii. You are nearing the end of the project and the client is insisting on adding new features but you want to prevent scope and feature creep. Do you refuse or agree to add the new features? Give three cogent reasons for your decision as Project Manager. (3 Marks)

BREAK IT DOWN

ASKED Elicitation techniques with justification (a); requirements at two levels plus a reason (b); a methodology choice for unclear requirements (c i); and the scope-creep decision (c ii). Modules Two, One and Six.

WATCH Part a wants four things per technique: name, why it suits, one advantage, one limitation. Part c i's clue is "may not be clearly stated", which points straight at an adaptive method.

A. TWO ELICITATION TECHNIQUES (4 MARKS)

Interview the head librarian: suitable because they hold the borrowing rules and can be probed in depth; advantage, rich detailed information; limitation, slow and only a few people. **Questionnaire** to student borrowers: suitable because it cheaply reaches the large, scattered student body; advantage, wide reach at low cost; limitation, no follow-up and often a low response rate.

B. REQUIREMENTS AND A REASON (6.5 MARKS)

Functional: the system shall let a member borrow a book; shall let a member reserve a book; shall calculate and record fines. **Non-functional:** the system shall respond within two seconds; shall be available 99.9% of the time; shall keep member data secure. **Reason for multiple levels:** different levels (a student, a librarian, a head of library) use the system differently, so gathering from all of them captures needs that a single level would miss.

C. METHODOLOGY AND SCOPE CREEP (7 MARKS)

i. Unclear requirements call for an adaptive method: choose **prototyping** or **RAD**, which discover requirements as they go. Map the phases: plan, then build a prototype, review it with the client, refine, and repeat until the requirements settle, then implement and support. **ii. Refuse (or defer)** the new features, because they threaten the agreed schedule and budget, they risk the quality of the nearly finished system, and they should be handled as a change request or a later release.

- a. You are to train a team of young professional on System Development Lifecycle (SDLC) and have chosen a four phased cycle namely; analysis, design, implementation and support. State the objectives of these phases and the activities involved in each. (8 marks)
- b. Name three system stakeholders and describe how each one views an information system. (3 marks)
- c. i) Differentiate between tools and techniques with respect to system development. (2 marks) ii) Give two examples of tools and techniques. (2 marks)
- d. XYZ Conglomerate has contracted you to develop a new information system for its administrative processes. Outline the key information you would need about the business in order to design a system that effectively meets its needs. (2.5 marks)

BREAK IT DOWN

WATCH Part a names four phases (not five): the four after planning. For each give an objective and its activities. Parts b, c, d are short recall from Modules One and Six.

A. FOUR PHASES: OBJECTIVES AND ACTIVITIES (8 MARKS)

Analysis: objective, understand and document what the system must do; activities, gather information, define and prioritise requirements, build prototypes, evaluate alternatives. **Design:** objective, specify how it will be built; activities, design the architecture, network, interfaces, database and controls. **Implementation:** objective, a reliable working system with trained users; activities, construct, test, convert data, train and document, install. **Support:** objective, keep it running for years; activities, maintain, enhance, support the users (help desk).

B. THREE STAKEHOLDERS (3 MARKS)

Users want it easy and helpful for daily work; managers want business value on time and on budget; developers want clear requirements and a feasible, maintainable system.

C. TOOLS VERSUS TECHNIQUES (4 MARKS)

A tool is a software product used to do the work; a technique is a way of completing one activity. Tools: a modelling package, an IDE. Techniques: interviewing, cost-benefit analysis.

D. KEY BUSINESS INFORMATION FOR XYZ (2.5 MARKS)

The business goals and processes, the people and their roles, the data the business keeps, the existing systems and their weaknesses, and the rules and constraints the new system must respect.

WORKED PAPER 2024/2025

Section B: Questions Three to Six

The four diagram questions. Q3 and Q4 are ERD and DFD heavy; Q5 is class diagrams and object-oriented concepts; Q6 is the analyst plus a use case and activity.

- a. What are the elements of a Data Flow diagram? Enumerate any four (4) you know. (2 marks)
- b. Define what is meant by a data store in a process model. How should a data store be named? (1.5 marks)
- c. Read and interpret the Entity Relationship Diagram (ERD) presented in Figure 1 by providing the cardinal relationships between the entities (3 marks)
- d. A warehouse has recently enforced a new management system to streamline the receiving process of stock. When truck drivers arrive, they are required to submit their trucks to the warehouse. On submission, the warehouse manager instructs the manual laborer to unload the trucks. It is important to note that a truck may or may not be carrying any stock. Once unloading is complete, the storekeeper is responsible for taking inventory of all available stock (spoilt, missing and in-good condition). The storekeeper then generates a comprehensive stock report, which is disseminated to the warehouse manager, the laborer involved in the unloading and the truck driver. Separately, both the truck drivers (regarding their delivery completion) and the labourers (regarding their unloading tasks) are required to submit individual activity reports directly to the manager.
 - i. Draw an ERD to model the relationships between all identified entities. (5 marks)
 - ii. Draw a Level 0 DFD to model the various processes involved. (4.5 marks)
- e. After drawing the Data flow diagrams for the above scenario, outline three (3) ways you can validate the integrity and correctness of the diagrams (1.5 marks)

Part c refers to a printed Figure 1: a large crow's-foot ERD for a music service (entities CUSTOMER, SALE, TARGETED PROMOTION, CUSTOMER FAVORITE, CUSTOMER INTEREST and AVAILABLE TUNE, with relationships such as makes, is targeted by, adds, creates, includes and promotes). The answer reads its cardinalities below.

BREAK IT DOWN

ASKED Two recall parts (a, b), a diagram-reading part (c), a full ERD plus Level 0 DFD from the warehouse story (d), and diagram validation (e). Modules Three and Four.

WATCH In d, the entities are TRUCK DRIVER, TRUCK, WAREHOUSE MANAGER, LABORER, STOREKEEPER, STOCK, STOCK REPORT, ACTIVITY REPORT. "A truck may or may not be carrying stock" is an optional relationship (zero or many).

A. DFD ELEMENTS (2 MARKS)

External entity, process, data store, data flow.

B. DATA STORE (1.5 MARKS)

A data store is data at rest in the system, a file or table where data is held between processes. It is named with a **noun** (for example D1 Stock), never a verb, and given a store number.

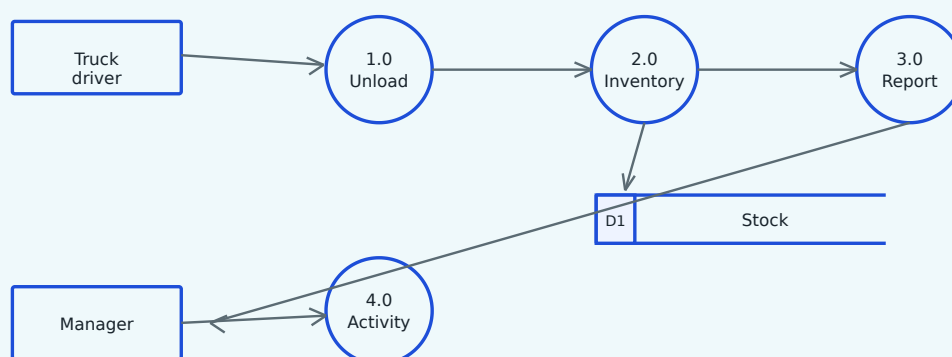
C. READING FIGURE 1 (3 MARKS)

Read each relationship both ways from its crow's-foot marks. For example: one CUSTOMER makes many SALES, and each SALE is made by one CUSTOMER (1:M); one CUSTOMER is targeted by many TARGETED PROMOTIONS, and each promotion targets many customers (M:N); a CUSTOMER adds many CUSTOMER FAVORITES (1:M); an AVAILABLE TUNE is included in many CUSTOMER INTERESTS (M:N). State the cardinality (one or many, optional or mandatory) for each line the figure draws.

ANSWER, CONTINUED**D. WAREHOUSE ERD AND LEVEL 0 DFD (9.5 MARKS)**

ERD: entities TRUCK DRIVER, TRUCK, MANAGER, LABORER, STOREKEEPER, STOCK, STOCK REPORT, ACTIVITY REPORT. A TRUCK DRIVER submits many TRUCKS; a TRUCK may carry zero or many STOCK (optional); a LABORER unloads many TRUCKS; a STOREKEEPER takes inventory of STOCK and generates STOCK REPORTS; drivers and labourers submit ACTIVITY REPORTS to the MANAGER. Crow's foot throughout, with the optional (circle) mark on the truck-to-stock line.

Level 0 DFD: external entities TRUCK DRIVER and MANAGER; processes 1.0 Receive and unload truck, 2.0 Take inventory, 3.0 Generate stock report, 4.0 Submit activity report; data stores D1 Stock and D2 Reports. Wire the flows so each process has input and output, and balance them against the context diagram.



The warehouse Level 0 DFD. Each process has an input and an output, no flow skips a process, and the stock report reaches the manager, driver and labourer.

E. VALIDATE THE DIAGRAMS (1.5 MARKS)

Check every process has both an input and an output (no black holes or miracles); check no illegal flow (every flow touches a process); and check the levels balance (the flows in and out of a process match its explosion).

- a. As a system analyst, you have been asked to draw a Data Flow diagram to explain the processes involved in food ordering at a Nigerian airport. Outline any four (4) reasons it is helpful to first Model the processes. (2 marks)
- b. i) Using appropriate diagrammatic illustrations, explain the various cardinal relationship in ERD (use Information Engineering style) (3 marks)
- ii) What makes a relationship optional? With a diagram, give a practical illustration. (1.5 marks)
- iii) What does the term 'Data Balancing' mean? (1 mark)
- c. In a tabular form, outline three major differences between Process Modelling and Data Modelling (1.5 marks)
- d. You are to develop a sales management system that allows sales representative to record orders placed by customers, checkout orders, and also allows the store keeper to keep inventory of product in the warehouse. A customer can place more than one order however each order must be placed by one customer. Each sales representative serves zero to many customers per time, however each customer is served by only one sales representative. Each order contains one or more products and zero or more products are stores in only one warehouse. Draw an entity relationship diagram for this scenario. (3 marks)
- e. 'Data cannot move between entities in a Data flow diagram.' Explain this sentence with an appropriate diagrammatic example. (1 mark)
- f. We wish to collect information about researchers, their institutions and research interests and then design a database that will track their activities. Draw an ERD to model the relationships between identified entities. [Hint: Each researcher is associated with only one institution and each researcher has several research interests]. (3 marks)
- g. You are designing a hospital management system for a client who has given you a short time frame of just one month. Which system acquisition strategy do you choose? Enumerate any three factors that supports your choice. (1.5 marks)

BREAK IT DOWN

- ASKED** Seven short parts, a to g, spanning DFD (a, e), ERD notation and theory (b, c, d, f) and acquisition (g). Modules Three, Four and Six.
- WATCH** Part b i explicitly says Information Engineering style, so use the crow's foot. Part d's "zero or many" and "one or more" phrases give you the exact cardinalities to mark.

A. FOUR REASONS TO MODEL (2 MARKS)

A model is a clear picture non-technical users can check; it separates what the system does from how it is built; it shows the system boundary and everything it touches; and it breaks a large system into manageable processes.

B. CARDINALITY, OPTIONALITY, DATA BALANCING (5.5 MARKS)

i. In crow's foot: one-to-one (a bar at each end), one-to-many (a bar at one end, a crow's foot at the other), many-to-many (a crow's foot at each end); draw each on a line between two entity rectangles. **ii.** A relationship is optional when the minimum is zero, drawn with a **circle** on the line (a customer may have zero orders). **iii.** Data balancing means the data flowing into and out of a process on a parent diagram must match the data flowing into and out of its child diagram when it is exploded.

C. PROCESS MODELLING VERSUS DATA MODELLING (1.5 MARKS)

PROCESS MODELLING (DFD)	DATA MODELLING (ERD)
Shows data in motion	Shows data at rest
Built from processes and flows	Built from entities and relationships
Answers how data moves	Answers what data is stored

ANSWER, CONTINUED**D. SALES MANAGEMENT ERD (3 MARKS)**

Entities SALES REP, CUSTOMER, ORDER, PRODUCT, WAREHOUSE. SALES REP serves zero-or-many CUSTOMERs, each CUSTOMER served by one rep; CUSTOMER places one-or-many ORDERs, each ORDER by one customer; ORDER contains one-or-many PRODUCTs; WAREHOUSE stores zero-or-many PRODUCTs. Mark the optional (circle) and many (crow's foot) ends as the words say.

E. DATA CANNOT MOVE BETWEEN ENTITIES (1 MARK)

A data flow must touch a process at one end, so you cannot draw an arrow straight from one external entity rectangle to another: put a process bubble between them. For example, Customer to Bank is wrong; Customer to "1.0 Process payment" to Bank is right.

F. RESEARCHER ERD (3 MARKS)

Entities RESEARCHER, INSTITUTION, RESEARCH INTEREST. A RESEARCHER belongs to one INSTITUTION (many researchers per institution, so INSTITUTION 1:M RESEARCHER); a RESEARCHER has many RESEARCH INTERESTs (1:M, or M:N if interests are shared). Crow's foot with the one bar at the institution end.

G. ACQUISITION STRATEGY (1.5 MARKS)

With only one month, **buy a packaged solution or outsource**, not build. Factors: the very short time available, the lack of in-house build capacity in the time, and that a hospital admin system is not so unique that a package cannot fit.

- a. As the head of an interview panel to recruit system analyst for your company, explain in detail three (3) skills you will be looking for in the successful candidate. (7 marks)
- b. Draw the Class diagram showing the Relationships and Multiplicities for the following analysis information of hospital management system
- A Patient is made-up of one Patient Details (Patient details is PId, Name, Age, gender)
 - A Patient can either be a New Patient or an Old Patient.
 - A Ward can be accommodated by at least 8 Patients per time.
 - A Ward is made-up of one Ward Details (Ward details is WardNo., No_of_Bed, WType).
 - A Convenience Room is part of a Ward.
 - A Ward has at least two Nurses attached to it per time. (5 marks)
- c. Briefly describe the following Object-Oriented Design concepts with relevant examples: i) Polymorphism, ii) Inheritance, iii) Composition and iv) Aggregation (4.5 marks)

BREAK IT DOWN

ASKED The analyst's skills (a), a class diagram from a bulleted spec (b), and four object-oriented concepts (c). Modules One and Five.

WATCH Each bullet in b maps to a relationship: "made-up of one" is composition, "either or" is generalization, "at least 8" and "at least two" are multiplicities, "part of" is composition.

ANSWER

A. THREE SKILLS (7 MARKS)

Technical: understanding of computers, networks, databases and the tools of building, so the analyst can understand and communicate technical requirements. **Business:** understanding of how organisations work and what this business does, so the system fits real goals. **People (interpersonal):** the most important; communicating in interviews and reviews, managing expectations and resolving conflict, because the analyst depends on users, managers and programmers.

B. HOSPITAL CLASS DIAGRAM (5 MARKS)

Classes PATIENT, PATIENTDETAILS, NEWPATIENT, OLDPATIENT, WARD, WARDDETAILS, CONVENIENCEROOM, NURSE. PATIENT is composed of one PATIENTDETAILS (filled diamond, 1); NEWPATIENT and OLDPATIENT generalise to PATIENT (hollow triangle); WARD holds 8..* PATIENTs; WARD is composed of one WARDDETAILS; CONVENIENCEROOM is part of WARD (filled diamond); WARD has 2..* NURSEs. Draw it exactly as the worked example in Module Five.

C. FOUR OOD CONCEPTS (4.5 MARKS)

Polymorphism: the same method behaves differently on different classes (draw() on a Circle versus a Square). **Inheritance:** a subclass takes on a superclass's members (SavingsAccount from Account). **Composition:** a life-or-death has-a (a House of Rooms). **Aggregation:** a loose has-a (a Team of Players).

- a. A System analyst is a problem solver. Explain your answer with tangible reasons. (5.5 marks)
- b. Patients call or come to the hospital to make new appointments as well as change or cancel appointments. Check to make sure the patient is a current patient and has no unpaid bills. If this is a new patient, perform "Add patient" before continuing. If the patient has unpaid bills, forward the call or patient to the account's office. If it is a change or cancellation, find the current appointment file and cancel it. Match the patient's desired appointment times with available dates and times and schedule new appointment.
 - i) Draw a Use Case Diagram for this scenario (6 marks)
 - ii) Draw an activity diagram for the scenario. (6 marks)

BREAK IT DOWN

ASKED Why the analyst is a problem solver (a), then a use case diagram and an activity diagram for the appointment process (b). Modules One, Two and Five.
This appointment scenario is the same one the 18/19 and 22/23 papers used.

ANSWER

A. THE ANALYST AS PROBLEM SOLVER (5.5 MARKS)

The analyst studies a business to find where it loses time, money or accuracy, works out whether technology can help, and specifies the system that solves the problem. The solution is usually an information system, but not always. The analyst researches and understands the problem, verifies the benefit outweighs the cost, defines requirements, develops alternatives, recommends the best, details it, implements and monitors, which is exactly a problem-solving sequence. That is why the analyst is a business problem solver and the bridge between users and the technical team.

B. USE CASE AND ACTIVITY DIAGRAMS (12 MARKS)

Use case (i): actor PATIENT; use cases Make appointment, Change appointment, Cancel appointment, Add patient. "Make appointment" **includes** "Check patient and bills". Draw it as Figure 2.1 in Module Two with these labels.

Activity (ii): start, "Receive call", decision "current patient?" (no leads to "Add patient" then continue), decision "unpaid bills?" (yes leads to "Forward to accounts" then end), then decision "change or cancel?": if so "Find and cancel appointment"; otherwise "Match desired times to available dates" and "Schedule appointment"; end. Draw it as Figure 5.3.

OBJECTIVE SECTION

Both Class Tests, Every Option Explained

All sixty questions from the two class tests, grouped by the part of this book that teaches them, and answered in full. Not only why the answer is the answer: why each of the others is not. The which-two form is the one the examination uses, so it is the one to practise.

Which TWO statements describe this practice and its purpose?

~~this practice is called gray holing~~

~~its purpose is to remove the context diagram~~

- ☒ this practice is called balancing
- ☐ its purpose is to keep parent and child consistent

How to Attack an Objective Question

The paper carries an objective section as well as its three written questions, and it is answered under more time pressure than anything else on it. This part is where you practise that half.

TEACH

WHAT IS HERE, AND WHERE IT CAME FROM

Both class tests are reproduced below, thirty questions each, sixty in all, grouped by the part of this book that teaches them rather than by which test they came from, so that a run of questions drills one topic at a time. Every option is explained, the right one and the wrong ones alike.

The stems and options were read off the test screenshots by eye. That is a weaker guarantee than the two written papers in this book carry, since those are held character for character against a transcript by a gate, and it is said here rather than left for you to assume.

⚠ TRAP

THE FORM THAT COSTS MARKS ALL BY ITSELF

The lecturer named test two's format as the one the examination will use, and test two is **multi-select**: a question that asks which **TWO** of the options are correct. Test one was single-answer. That difference is worth real marks on its own, and it is the reason this part exists in the form it does.

On a which-two question, **both** choices must be right for the mark. A candidate who knows the topic perfectly and stops at the first correct option scores nothing. So the habit to build is: read all four options every time, decide about each one separately, and only then count how many you have marked. If you have marked one, or three, you have not finished the question.

The other habit is that the two correct options usually do different jobs. A typical pair is one option that NAMES the thing and one that states its PURPOSE, as in the balancing question below: one says the practice is called balancing, the other says why it is done. Looking for that split makes the pair easier to find than hunting for two statements that both feel true.

◆ KEY POINT

In a four-option question, the options that are wrong are wrong for a reason you can state. Finding those reasons is faster and safer than recognising the right answer, because recognition is exactly what a well-built distractor attacks.

BLUNT VERSION: do not hunt for the right one. Kill the wrong ones, and on a which-two, count what is left.

- 1 **Read the stem twice and underline the instruction.** NOT, BEST, TWO, PRIMARILY and EXCEPT each flip or narrow the question. A stem asking which two is a different question from one asking which one, and the paper mixes both.
- 2 **Answer it before you read the options where you can.** If you know a black hole is a process with inputs and no outputs, decide that first, then read the options as a lookup rather than as a suggestion.
- 3 **Strike the impossible.** On these tests two of the four usually die at once: an option that names a different concept, or one that is true of the subject but does not answer the question asked.
- 4 **On the last two, split on the exact wording.** It is nearly always one word: only, all, every, eliminates, requires. An option saying a context diagram "eliminates the need for" something is almost always the distractor.

HOW TO USE THIS PART

Sit one group at a time, timed at 45 seconds a question, with the answers covered. Then read the whole answer block for that group, including the questions you got right: on an objective paper a right answer for the wrong reason is a mark you will not get twice.

OBJECTIVE SECTION

Module One, the Analyst, the Life Cycle and Methodologies

30 questions. Module One. Compulsory Question One. Cover the answers before you start.

1. **TEST 1 Q1** Systems Development Methodology is MOST accurately defined as: (a) A regulatory compliance framework mandating the documentation standards for government information systems (b) The set of hardware and software tools used by developers to construct and test information systems (c) The standard process followed by an organisation to conduct all the steps necessary to analyse, design, implement, and maintain information systems (d) A specific programming language or development framework chosen by an organisation for building software
2. **TEST 1 Q2** The 'get the user involved' principle in successful systems development emphasises that user involvement throughout the process: (a) Is only necessary for consumer-facing systems and not required for internal back-office applications (b) Should be limited to the initial requirements phase to prevent users from disrupting the technical development work (c) Helps ensure that requirements are accurate, reduces resistance to change, and increases user acceptance of the final system (d) Increases development costs significantly but is mandatory for regulatory compliance in most industries
3. **TEST 1 Q3** The Waterfall Model is characterised by which of the following fundamental features? (a) A traditional, linear approach in which each phase must be completed before the next phase begins, with key deliverables presented for approval at each phase end (b) A flexible framework that allows requirements to evolve and develop-

- ment activities to be revisited throughout the project (c) An iterative development cycle where system components are built, tested, and refined through multiple rapid iterations (d) A collaborative approach that integrates users into every phase of development through continuous prototyping and feedback
4. **TEST 1 Q4** Building a system based on incomplete or misunderstood requirements is MOST likely to result in which of the following outcomes? (a) Over budgeting, late delivery, or delivery of a system that does not fully solve the problems it was intended to address (b) Improved system performance but with reduced functionality compared to what users originally requested (c) Increased user satisfaction because a simpler system is easier to learn and operate (d) Faster delivery due to reduced scope, though with some features deferred to a future release
 5. **TEST 1 Q5** What is the key purpose of the 'Joint Application Development (JAD)' technique as it relates to prototyping? (a) JAD replaces prototyping entirely by using structured group meetings to eliminate the need for visual demonstrations (b) JAD produces the final system without any prototyping by having all stakeholders simultaneously define and approve requirements (c) JAD is used exclusively during the implementation phase to coordinate the joint efforts of developers and testers (d) When JRP/JAD incorporates prototyping as a means for confirming requirements and obtaining design approvals, the benefits of prototyping are realised within a structured group collaboration framework
 6. **TEST 1 Q6** How does the adaptive approach to SDLC fundamentally differ from the predictive approach in handling requirements? (a) The adaptive approach requires more extensive documentation than the predictive approach to manage changing requirements (b) The adaptive approach relies entirely on automated tools to manage requirements, while the predictive approach depends on manual documentation (c) In the adaptive approach, requirements evolve over time through continuous user feedback and incremental delivery, whereas the predictive approach defines requirements upfront before development begins (d) The adaptive approach follows a fixed sequential process while the predictive approach allows for iterative development cycles
 7. **TEST 1 Q7** Which of the following BEST explains the 'justify systems as capital investments' principle in systems development? (a) All information systems should be classified as capital assets on the organisation's balance sheet for tax purposes (b) Information systems should be replaced on a fixed capital depreciation schedule regardless of their operational effectiveness (c) Information systems require significant financial resources and should be evaluated like other capital investments using cost-benefit analysis to determine whether expected benefits justify the costs and risks involved (d) Systems analysts must obtain capital funding approval from shareholders before any development activities can commence
 8. **TEST 1 Q8** Which of the following is a recognised ADVANTAGE of the RAD methodology? (a) RAD is best suited for safety-critical systems because it emphasises rigorous testing at every development phase (b) RAD produces the most comprehensive technical documentation of any methodology, supporting long-term maintenance (c) User feedback makes the model very effective, a time-box approach mitigates cost and schedule risk, and development costs are lower because the system matches user expectations (d) RAD eliminates all project risks by freezing requirements at the beginning and adhering strictly to the original scope
 9. **TEST 1 Q9** When a systems analyst is said to be performing 'system design', what is the analyst primarily doing? (a) Translating requirements identified during system analysis into a detailed blueprint that specifies how the system will be structured and how

its components will interact (b) Gathering requirements from users through interviews, questionnaires, and observation (c) Studying the existing business processes to identify inefficiencies and problem areas (d) Testing the completed system to verify it meets functional and non-functional specifications

10. **TEST 1 Q10** Systems design specifications are best described as documents that: (a) Outline the business problems and high-level requirements gathered during the analysis phase (b) Describe the construction details of all system components including databases, networks, program modules, operating procedures, user interfaces, and conversion plans (c) Record the testing outcomes and user acceptance results following system implementation (d) Document the feasibility study results including economic, technical, and operational assessments
11. **TEST 1 Q11** Which of the following BEST describes why developing information systems without a clear methodology is problematic? (a) It is only problematic for large organisations; small organisations can successfully develop systems without a formal methodology (b) It always results in technically superior systems because developers have greater creative freedom without constraints (c) It reduces the time to market but typically produces systems with excessive documentation and redundant features (d) It often leads to cost overruns, missed deadlines, and systems that fail to meet user requirements
12. **TEST 1 Q12** According to the methodology selection criteria, which approach is BEST suited for projects with very short time schedules? (a) The V-Shape Model, because its emphasis on testing from the start prevents costly defect correction near the deadline (b) The Parallel Model, because dividing the system into independent subprojects allows concurrent development tracks (c) The Waterfall model, because its structured sequential approach prevents time-wasting rework between phases (d) RAD methodologies, because they are designed to accelerate development, with iterative development and system prototyping allowing the team to adjust system functionality to meet a specific delivery date
13. **TEST 1 Q13** The principle 'establish standards for development and documentation' in successful systems development serves which primary purpose? (a) To ensure consistency, quality, and clarity across system development activities, making systems easier to understand, maintain, and enhance over time (b) To satisfy external regulatory bodies that mandate specific documentation formats for audit and compliance purposes (c) To ensure all development teams use identical programming languages and database management systems (d) To create bureaucratic checkpoints that prevent developers from making unauthorised changes to system requirements
14. **TEST 1 Q14** The Incremental Model is BEST described as a methodology that: (a) Builds the entire system in one continuous development cycle before delivering it to users for acceptance testing (b) Develops the system in small, manageable portions called increments, where each increment adds new functionality and builds on previous releases, combining linear and iterative approaches (c) Develops the system by having multiple teams work on competing implementations, with the best solution selected at project completion (d) Requires all requirements to be undefined at project start, with features discovered entirely through user interaction with prototypes
15. **TEST 1 Q15** Which of the following statements about the Waterfall model's documentation is accurate? (a) The Waterfall model's documentation is informal and maintained only by developers without requiring management approval (b) The Waterfall model produces extensive written documentation and requires formal reviews and user and IT management sign-off at the end of each phase to ensure strict monitoring (c) The

Waterfall model intentionally minimises documentation to keep teams focused on delivering working software quickly (d) The Waterfall model produces documentation only at the implementation phase when the system is ready for user acceptance

16. **TEST 1 Q16** According to the analyst's problem-solving approach, what must the analyst do FIRST before recommending any solution? (a) Develop a set of possible alternative solutions and present them to management for selection (b) Confirm with management that the benefits of solving the problem outweigh the associated costs (c) Research and thoroughly understand the problem, including who is involved, what business processes are affected, and what other systems would be impacted (d) Define the detailed design specifications and create a blueprint for the new system
17. **TEST 1 Q17** In the adaptive SDLC approach, what is a 'sprint' and what does it produce? (a) A specialised testing cycle conducted after each major release to identify and resolve performance deficiencies (b) A planning meeting held at the start of the project to define the complete set of system requirements (c) An accelerated version of the waterfall model where all phases are compressed into a single two-week period (d) A short, time-boxed development cycle that includes analysis, design, coding, testing, and review, producing a working increment of the system at its conclusion
18. **TEST 1 Q18** Which of the following is a DISADVANTAGE of evolutionary prototyping? (a) It completely prevents users from seeing system requirements as they are being gathered, reducing accuracy (b) It has a tendency to abandon structured program development for 'code-and-fix' development, overall maintainability may be overlooked, and the process may continue forever due to scope creep (c) It produces excessive formal documentation that slows down the development process and increases overhead costs (d) It requires all requirements to be fully specified before any prototype can be developed and evaluated
19. **TEST 1 Q19** The Parallel Model differs from the traditional Waterfall model primarily in which way? (a) The Parallel Model requires two separate development teams to work on different versions of the system for competitive evaluation (b) The Parallel Model introduces a dedicated testing phase that runs concurrently with every development phase from the beginning (c) The Parallel Model allows requirements to evolve during development through continuous user feedback and iterative enhancements (d) The overall system is divided into subprojects after the general analysis and high-level design phases, with each subproject moving through remaining phases simultaneously before integration
20. **TEST 1 Q20** Which of the following BEST describes the primary objective of the design phase in the SDLC? (a) To monitor the installed system and make enhancements in response to changing business requirements (b) To construct software components, verify and test the system, and prepare it for deployment to users (c) To gather information from stakeholders and define the functional requirements of the new system (d) To design the solution system based on requirements defined during analysis, developing an architectural structure for software components, databases, user interfaces, and the operating environment
21. **TEST 1 Q21** In the table comparing methodology usefulness, which methodology is rated 'Excellent' for projects with UNCLEAR user requirements? (a) The V-Model is rated Excellent because its rigorous testing phases help uncover hidden or unclear requirements (b) Waterfall and Parallel models are rated Excellent because their structured approach forces requirements clarification (c) The Iterative model alone is rated Excellent while all other methodologies are rated Poor for unclear requirements

(d) System Prototyping, Throwaway Prototyping, and Agile Development are all rated Excellent for unclear user requirements

22. **TEST 1 Q22** Which of the following is NOT listed as a popular Agile methodology in the course material? (a) Critical Path Method (CPM) (b) Scrum (c) Extreme Programming (XP) (d) Kanban
23. **TEST 1 Q23** The principle of 'divide and conquer' in systems development MOST closely corresponds to which analytical concept? (a) Separating business requirements from technical specifications to prevent scope creep during development (b) Breaking a large system into smaller, manageable components or subsystems to simplify development and reduce complexity (c) Dividing the project budget equally among planning, analysis, design, and implementation phases (d) Distributing system development work among multiple competing vendor teams to accelerate delivery
24. **TEST 1 Q24** The 'support phase' of the SDLC is MOST accurately described as: (a) Activities to upgrade and maintain the system after deployment, including fixing bugs, making enhancements, and supporting users through a help desk (b) The process of conducting feasibility studies and producing the initial project proposal for management approval (c) The phase where analysts gather and document user requirements before system design begins (d) The activities carried out during system construction to ensure technical components meet design specifications
25. **TEST 1 Q25** Why is testing considered one of the most critical steps in the implementation phase of the SDLC? (a) Because testing generates the official documentation required for regulatory compliance and audit purposes (b) Because the cost of bugs can be immense, and most organisations spend more time and attention on testing than on writing the programs in the first place (c) Because testing determines the final project budget and influences decisions about future system enhancements (d) Because testing is the only phase where users are directly involved in the system development process
26. **TEST 1 Q26** The concept of 'functional decomposition' in systems analysis refers to: (a) The approach of dividing a system into smaller, manageable components or subsystems to focus analysis on individual areas of a business (b) The method of assigning different system functions to separate teams of developers during implementation (c) The process of documenting the functional requirements of a system in a structured specification document (d) The technique of breaking down a user's functional requirements into technical design specifications
27. **TEST 1 Q27** The Waterfall model is MOST suitable for which type of project scenario? (a) A large, expensive, and complicated project with clear objectives, stable and comprehensive requirements, and no pressure for immediate implementation (b) A startup environment where rapid prototyping and quick delivery of a minimum viable product are the primary goals (c) A project with frequently changing customer requirements that demand continuous user feedback and iterative delivery (d) A project where the development team is highly experienced with agile techniques and self-organising workflows
28. **TEST 1 Q28** The principle 'design systems for growth and change' in successful systems development is MOST concerned with: (a) Ensuring systems are flexible and scalable to accommodate future business expansion, technological advancements, and changing user requirements, thereby extending the system's useful life (b) Building systems exclusively on the latest technology platforms regardless of organisational budget constraints (c) Designing systems with the maximum number of features possible so fu-

ture enhancements are not needed (d) Establishing rigid technical standards that prevent unauthorised modifications to the system after deployment

29. **TEST 1 Q29** Throwaway (rapid or exploratory) prototyping is MOST appropriate when: (a) The project has well-understood and stable requirements that are unlikely to change during the development lifecycle (b) The organisation requires the highest possible system reliability and has zero tolerance for defects in production (c) The development team is highly experienced and needs no exploratory phase before committing to the final design (d) There is a lack of familiarity with the technology, requirements are unclear, or there are complex technical risks that must be explored through design prototypes before the actual system is built
30. **TEST 1 Q30** When system reliability is a HIGH priority, for example in medical equipment or missile control systems, which methodology is MOST appropriate according to the selection criteria? (a) RAD, because its time-box approach ensures rapid delivery of reliable system increments to users (b) Agile development, because its iterative nature allows defects to be detected and corrected after each sprint (c) The Incremental Model, because delivering high-risk functions first ensures the most critical components are reliable (d) The V-model, which is useful when reliability is important because it emphasises testing at each development phase through verification and validation

1. The answer is **The standard process followed by an organisation to conduct all the steps necessary to analyse, design, implement, and maintain information systems**. A methodology is a PROCESS, not a product and not a toolset. It is the organisation's standard way of getting from a business need to a working, maintained system, and it names the steps, their order, and what each must produce. Waterfall, RAD, Agile and the spiral are all methodologies because each answers the same question differently: in what order do we analyse, design, implement and maintain.

- ✗ **A regulatory compliance framework mandating the documentation standards for government information systems.** Wrong. Compliance frameworks constrain how you document; they do not tell you how to develop.
- ✗ **The set of hardware and software tools used by developers to construct and test information systems.** Wrong, and it is the commonest confusion: those are CASE tools. Tools support a methodology, they are not one.
- ✓ **The standard process followed by an organisation to conduct all the steps necessary to analyse, design, implement, and maintain information systems.** Correct. Standard process, covering analyse, design, implement and maintain.
- ✗ **A specific programming language or development framework chosen by an organisation for building software.** Wrong. The language is an implementation choice made INSIDE a methodology.

2. The answer is **Helps ensure that requirements are accurate, reduces resistance to change, and increases user acceptance of the final system**. Three benefits, and the question wants all three: accuracy, less resistance, more acceptance. Users are the only people who know what the work actually involves, so keeping them in the room is how requirements stay true. The second and third benefits are political rather than technical: a system people helped shape is a system people will use.

- ✗ **Is only necessary for consumer-facing systems and not required for internal back-office applications.** Wrong. Internal systems fail for exactly the same reason, and their users are easier to reach.
- ✗ **Should be limited to the initial requirements phase to prevent users from disrupting the technical development work.** Wrong, and it is the mistake the principle exists to prevent. Requirements change, and users are the ones who notice.
- ✓ **Helps ensure that requirements are accurate, reduces resistance to change, and increases user acceptance of the final system.** Correct. Accuracy, reduced resistance, and acceptance.
- ✗ **Increases development costs significantly but is mandatory for regulatory compliance in most industries.** Wrong on both halves: involvement REDUCES the cost of rework, and it is good practice rather than a regulation.

3. The answer is **A traditional, linear approach in which each phase must be completed before the next phase begins, with key deliverables presented for approval at each phase end**. Linear, sequential, and gated. Water falls downhill and does not flow back up, which is the whole image: finish a phase, get it approved, and only then start the next. Its strength is control and documentation; its weakness is that a requirement discovered late is expensive, because going back means going back a long way.

- ✓ **A traditional, linear approach in which each phase must be completed before the next phase begins, with key deliverables presented for approval at each phase end.** Correct. Linear, each phase complete before the next, deliverables approved at each gate.

- ✗ **A flexible framework that allows requirements to evolve and development activities to be revisited throughout the project.** Wrong. That flexibility describes the ADAPTIVE approaches, which exist because Waterfall lacks it.
- ✗ **An iterative development cycle where system components are built, tested, and refined through multiple rapid iterations.** Wrong. That is iterative or incremental development.
- ✗ **A collaborative approach that integrates users into every phase of development through continuous prototyping and feedback.** Wrong. Continuous prototyping and user integration describe RAD and JAD.

4. The answer is **Over budgeting, late delivery, or delivery of a system that does not fully solve the problems it was intended to address.** Bad requirements are the most expensive defect in the whole life cycle, because everything downstream is built on them. The error is not discovered until testing or delivery, by which time the design, the code and the data structures have all committed to it. That is why fact-finding gets its own phase rather than being folded into design.

- ✓ **Over budgeting, late delivery, or delivery of a system that does not fully solve the problems it was intended to address.** Correct. Over budget, late, and still not solving the problem.
- ✗ **Improved system performance but with reduced functionality compared to what users originally requested.** Wrong. Misunderstanding requirements does not improve performance; it wastes effort on the wrong thing.
- ✗ **Increased user satisfaction because a simpler system is easier to learn and operate.** Wrong, and it confuses a system that is SMALLER by accident with one that is SIMPLER by design.
- ✗ **Faster delivery due to reduced scope, though with some features deferred to a future release.** Wrong. Deferring scope on purpose is a plan. Misunderstanding it is not, and it costs time rather than saving it.

5. The answer is **When JRP/JAD incorporates prototyping as a means for confirming requirements and obtaining design approvals, the benefits of prototyping are realised within a structured group collaboration framework.** JAD and prototyping are partners, not rivals. JAD puts users, analysts and managers in one room to agree requirements quickly; a prototype gives that room something concrete to react to, because people criticise a screen far better than they criticise a paragraph. Put them together and you confirm requirements AND obtain design approval in the same session.

- ✗ **JAD replaces prototyping entirely by using structured group meetings to eliminate the need for visual demonstrations.** Wrong. It uses prototyping rather than eliminating it.
- ✗ **JAD produces the final system without any prototyping by having all stakeholders simultaneously define and approve requirements.** Wrong. JAD gathers and agrees requirements; it does not produce the finished system.
- ✗ **JAD is used exclusively during the implementation phase to coordinate the joint efforts of developers and testers.** Wrong phase. JAD belongs to requirements and design, not implementation.
- ✓ **When JRP/JAD incorporates prototyping as a means for confirming requirements and obtaining design approvals, the benefits of prototyping are realised within a structured group collaboration framework.** Correct. Prototyping inside a structured group framework, confirming requirements and gaining approvals.

6. The answer is **In the adaptive approach, requirements evolve over time through continuous user feedback and incremental delivery, whereas the predictive approach defines requirements upfront before development begins.** The names say it. PREDICTIVE predicts: it decides everything at the start and then follows the plan, which suits projects where the requirements are genuinely known and stable.

ADAPTIVE adapts: it expects requirements to change, so it delivers in increments and lets feedback reshape what comes next.

- ✗ **The adaptive approach requires more extensive documentation than the predictive approach to manage changing requirements.** Wrong. Adaptive approaches deliberately favour working software over heavy documentation.
- ✗ **The adaptive approach relies entirely on automated tools to manage requirements, while the predictive approach depends on manual documentation.** Wrong. Neither is defined by its tooling.
- ✓ **In the adaptive approach, requirements evolve over time through continuous user feedback and incremental delivery, whereas the predictive approach defines requirements upfront before development begins.** Correct. Evolving through feedback and increments, against fixed upfront definition.
- ✗ **The adaptive approach follows a fixed sequential process while the predictive approach allows for iterative development cycles.** Wrong, and it is exactly backwards. The fixed sequential process is the PREDICTIVE one.

7. The answer is **Information systems require significant financial resources and should be evaluated like other capital investments using cost-benefit analysis to determine whether expected benefits justify the costs and risks involved.** A system competes for money with every other thing the organisation could buy, so it has to earn its place the same way: cost-benefit analysis, weighed against risk. This is why economic feasibility exists as a study in its own right, and why an analyst who cannot express benefits in money has not finished the argument.

- ✗ **All information systems should be classified as capital assets on the organisation's balance sheet for tax purposes.** Wrong. Accounting classification is not the principle; the principle is about JUSTIFYING the spend.
- ✗ **Information systems should be replaced on a fixed capital depreciation schedule regardless of their operational effectiveness.** Wrong. Replacement on a schedule ignores whether the system still works, which is the opposite of evaluating it.
- ✓ **Information systems require significant financial resources and should be evaluated like other capital investments using cost-benefit analysis to determine whether expected benefits justify the costs and risks involved.** Correct. Evaluate like any capital investment, using cost-benefit analysis against costs and risks.
- ✗ **Systems analysts must obtain capital funding approval from shareholders before any development activities can commence.** Wrong. Shareholder approval is not what this means, and few projects would ever start.

8. The answer is **User feedback makes the model very effective, a time-box approach mitigates cost and schedule risk, and development costs are lower because the system matches user expectations.** Rapid Application Development buys speed with user contact and a fixed clock. The TIME BOX is the key idea: the deadline is fixed and the scope flexes to fit, which is the reverse of Waterfall and is what keeps cost and schedule risk contained. Costs fall because rework falls, since users have been correcting the thing all along.

- ✗ **RAD is best suited for safety-critical systems because it emphasises rigorous testing at every development phase.** Wrong. Safety-critical work goes to the V-model, which pairs a test with every phase.
- ✗ **RAD produces the most comprehensive technical documentation of any methodology, supporting long-term maintenance.** Wrong, and it is RAD's best-known DISADVANTAGE: speed comes at the cost of documentation.
- ✓ **User feedback makes the model very effective, a time-box approach mitigates cost and schedule risk, and development costs are lower because the system matches user expectations.** Correct. User feedback, the time box, and lower cost through fewer surprises.

- ✗ **RAD eliminates all project risks by freezing requirements at the beginning and adhering strictly to the original scope.** Wrong. Freezing requirements is Waterfall's approach, and no method eliminates all risk.

9. The answer is **Translating requirements identified during system analysis into a detailed blueprint that specifies how the system will be structured and how its components will interact.** Analysis asks WHAT the system must do; design answers HOW it will do it. The handover between them is the moment requirements become a blueprint: structure, components, and the way those components talk to each other. If you can keep what against how straight, four exam questions answer themselves.

- ✓ **Translating requirements identified during system analysis into a detailed blueprint that specifies how the system will be structured and how its components will interact.** Correct. Requirements turned into a blueprint of structure and interaction.
- ✗ **Gathering requirements from users through interviews, questionnaires, and observation.** Wrong. That is fact-finding, which belongs to ANALYSIS.
- ✗ **Studying the existing business processes to identify inefficiencies and problem areas.** Wrong. Studying the current system is analysis too, earlier still.
- ✗ **Testing the completed system to verify it meets functional and non-functional specifications.** Wrong. That is testing, which comes after implementation.

10. The answer is **Describe the construction details of all system components including databases, networks, program modules, operating procedures, user interfaces, and conversion plans.** A design specification is a builder's document: detailed enough that somebody else could construct the system from it. Learn the list, because the examiner uses it: databases, networks, program modules, operating procedures, user interfaces, and conversion plans. Notice that it covers the changeover too, not only the software.

- ✗ **Outline the business problems and high-level requirements gathered during the analysis phase.** Wrong. High-level problems and requirements belong to the analysis phase that came before.
- ✓ **Describe the construction details of all system components including databases, networks, program modules, operating procedures, user interfaces, and conversion plans.** Correct. Construction detail across all six areas.
- ✗ **Record the testing outcomes and user acceptance results following system implementation.** Wrong. Test results come after the build.
- ✗ **Document the feasibility study results including economic, technical, and operational assessments.** Wrong. Feasibility is a study done early to decide whether to proceed at all.

11. The answer is **It often leads to cost overruns, missed deadlines, and systems that fail to meet user requirements.** Three symptoms, and they are the same three the whole course keeps returning to: over cost, over time, and under-delivering. A methodology is not bureaucracy for its own sake; it is the thing that makes each of those failures visible early enough to fix, by naming what must be produced and approved before the next step begins.

- ✗ **It is only problematic for large organisations; small organisations can successfully develop systems without a formal methodology.** Wrong. Size changes the weight of the process, not the need for one.
- ✗ **It always results in technically superior systems because developers have greater creative freedom without constraints.** Wrong. Freedom without structure is what produces the failures listed in the correct option.
- ✗ **It reduces the time to market but typically produces systems with excessive documentation and redundant features.** Wrong on both counts, and it is self-contradictory: skipping a methodology does not generate excessive documentation.

- ✓ **It often leads to cost overruns, missed deadlines, and systems that fail to meet user requirements.** Correct. Cost overruns, missed deadlines, and systems that miss the requirement.

12. The answer is **RAD methodologies, because they are designed to accelerate development, with iterative development and system prototyping allowing the team to adjust system functionality to meet a specific delivery date.** The clue is in the name: RAD is RAPID Application Development, and short schedules are the problem it was built for. The mechanism is the time box, which fixes the date and flexes the functionality, so the team adjusts what ships rather than when it ships. Learn the selection criteria as a table of problem against answer, because this examiner asks it repeatedly.

- ✗ **The V-Shape Model, because its emphasis on testing from the start prevents costly defect correction near the deadline.** Wrong. The V-model suits RELIABILITY, not speed; all that testing takes time.
- ✗ **The Parallel Model, because dividing the system into independent subprojects allows concurrent development tracks.** Wrong. The Parallel model shortens the schedule somewhat, but RAD is the one the criteria name for very short schedules.
- ✗ **The Waterfall model, because its structured sequential approach prevents time-wasting rework between phases.** Wrong, and backwards: Waterfall is the slowest to first delivery because nothing ships until the end.
- ✓ **RAD methodologies, because they are designed to accelerate development, with iterative development and system prototyping allowing the team to adjust system functionality to meet a specific delivery date.** Correct. Built for speed, using iteration, prototyping, and a fixed delivery date.

13. The answer is **To ensure consistency, quality, and clarity across system development activities, making systems easier to understand, maintain, and enhance over time.** Standards exist for the person who comes after you. Most of a system's life is MAINTENANCE, carried out by people who were not in the room when it was built, and consistent practice plus clear documentation is what makes that possible. Consistency, quality, clarity, then maintainability: that chain is the answer.

- ✓ **To ensure consistency, quality, and clarity across system development activities, making systems easier to understand, maintain, and enhance over time.** Correct. Consistency and clarity now, maintainability and enhancement later.
- ✗ **To satisfy external regulatory bodies that mandate specific documentation formats for audit and compliance purposes.** Wrong. Compliance may be a side benefit, but it is not the purpose.
- ✗ **To ensure all development teams use identical programming languages and database management systems.** Wrong. Standards govern HOW work is done and documented, not which tools are used.
- ✗ **To create bureaucratic checkpoints that prevent developers from making unauthorised changes to system requirements.** Wrong, and it describes change control as an obstacle rather than the principle's aim.

14. The answer is **Develops the system in small, manageable portions called increments, where each increment adds new functionality and builds on previous releases, combining linear and iterative approaches.** Increments are slices of working system, each one adding function to what is already delivered. The phrase worth memorising is that it COMBINES linear and iterative: each increment runs a small waterfall inside it, while the sequence of increments is iterative. That is why it delivers value early without abandoning structure.

- ✗ **Builds the entire system in one continuous development cycle before delivering it to users for acceptance testing.** Wrong. One big delivery at the end is Waterfall.

- ✓ **Develops the system in small, manageable portions called increments, where each increment adds new functionality and builds on previous releases, combining linear and iterative approaches.** Correct. Small increments, each building on the last, linear and iterative together.
- ✗ **Develops the system by having multiple teams work on competing implementations, with the best solution selected at project completion.** Wrong. Competing teams describes nothing in this course.
- ✗ **Requires all requirements to be undefined at project start, with features discovered entirely through user interaction with prototypes.** Wrong, and too extreme: increments still need requirements, just not all of them at once.

15. The answer is The Waterfall model produces extensive written documentation and requires formal reviews and user and IT management sign-off at the end of each phase to ensure strict monitoring. Documentation is Waterfall's defining strength and its defining cost. Each phase ends with a deliverable, that deliverable is formally reviewed, and BOTH user and IT management sign it off before the next phase starts. Those sign-off gates are what make the model auditable, and also what make change expensive once a gate has closed.

- ✗ **The Waterfall model's documentation is informal and maintained only by developers without requiring management approval.** Wrong. Formal approval at each gate is the model's central control.
- ✓ **The Waterfall model produces extensive written documentation and requires formal reviews and user and IT management sign-off at the end of each phase to ensure strict monitoring.** Correct. Extensive documentation, formal review, and sign-off by user and IT management at each phase end.
- ✗ **The Waterfall model intentionally minimises documentation to keep teams focused on delivering working software quickly.** Wrong. Minimising documentation is the AGILE preference, and it is Waterfall's opposite.
- ✗ **The Waterfall model produces documentation only at the implementation phase when the system is ready for user acceptance.** Wrong. Documentation is produced throughout, phase by phase.

16. The answer is Research and thoroughly understand the problem, including who is involved, what business processes are affected, and what other systems would be impacted. Understand before you solve. The steps run in order: research and understand the problem, then verify that the benefits of solving it outweigh the costs, then develop alternatives, then recommend. Every wrong option here is a LATER step in that same sequence, which is the trap: they are all things the analyst does, just not first.

- ✗ **Develop a set of possible alternative solutions and present them to management for selection.** Wrong step, and it comes third. You cannot generate alternatives to a problem you have not understood.
- ✗ **Confirm with management that the benefits of solving the problem outweigh the associated costs.** Wrong step, and it comes second. Cost-benefit follows understanding.
- ✓ **Research and thoroughly understand the problem, including who is involved, what business processes are affected, and what other systems would be impacted.** Correct. Understand the problem, who is involved, which processes and which other systems are affected.
- ✗ **Define the detailed design specifications and create a blueprint for the new system.** Wrong phase entirely. Design comes long after the problem is understood and a solution chosen.

17. The answer is A short, time-boxed development cycle that includes analysis, design, coding, testing, and review, producing a working increment of the system at its conclusion. A sprint is a whole miniature project: it contains every activity, from analysis through review, and it ends with something that WORKS. Two features

carry the marks. It is TIME-BOXED, so the clock is fixed and the scope moves, and it produces a working increment rather than a document or a phase.

- ✗ **A specialised testing cycle conducted after each major release to identify and resolve performance deficiencies.** Wrong. Testing happens inside a sprint, not after releases as a separate cycle.
- ✗ **A planning meeting held at the start of the project to define the complete set of system requirements.** Wrong. Defining all requirements upfront is the predictive approach a sprint exists to avoid.
- ✗ **An accelerated version of the waterfall model where all phases are compressed into a single two-week period.** Wrong, and it is the tempting one: a sprint is not a compressed waterfall, because it repeats and adapts rather than running once.
- ✓ **A short, time-boxed development cycle that includes analysis, design, coding, testing, and review, producing a working increment of the system at its conclusion.** Correct. Short, time-boxed, all activities included, ending in a working increment.

18. The answer is **It has a tendency to abandon structured program development for 'code-and-fix' development, overall maintainability may be overlooked, and the process may continue forever due to scope creep.** Three named dangers, and the question wants all three: the slide into code-and-fix, maintainability being forgotten, and a process that never ends because every demonstration produces new requests. That last one has a name worth knowing: SCOPE CREEP. The prototype becomes the product without ever having been designed to be one.

- ✗ **It completely prevents users from seeing system requirements as they are being gathered, reducing accuracy.** Wrong, and inverted: letting users SEE the system early is prototyping's main advantage.
- ✓ **It has a tendency to abandon structured program development for 'code-and-fix' development, overall maintainability may be overlooked, and the process may continue forever due to scope creep.** Correct. Code-and-fix, neglected maintainability, and endless scope creep.
- ✗ **It produces excessive formal documentation that slows down the development process and increases overhead costs.** Wrong. Prototyping produces too LITTLE documentation, not too much.
- ✗ **It requires all requirements to be fully specified before any prototype can be developed and evaluated.** Wrong. Prototyping exists precisely because requirements cannot all be specified upfront.

19. The answer is **The overall system is divided into subprojects after the general analysis and high-level design phases, with each subproject moving through remaining phases simultaneously before integration.** The split happens AFTER general analysis and high-level design, which is the detail the examiner is testing. You analyse and design the whole once, then break it into subprojects that run their remaining phases side by side, then integrate. That shortens the schedule without the subprojects contradicting one another, because they share one design.

- ✗ **The Parallel Model requires two separate development teams to work on different versions of the system for competitive evaluation.** Wrong. Competing versions is not what parallel means here.
- ✗ **The Parallel Model introduces a dedicated testing phase that runs concurrently with every development phase from the beginning.** Wrong. Testing alongside every phase is the V-MODEL.
- ✗ **The Parallel Model allows requirements to evolve during development through continuous user feedback and iterative enhancements.** Wrong. Evolving requirements describes the adaptive approaches.

- ✓ **The overall system is divided into subprojects after the general analysis and high-level design phases, with each subproject moving through remaining phases simultaneously before integration.** Correct. Divided into subprojects after general analysis and high-level design, then integrated.

20. The answer is **To design the solution system based on requirements defined during analysis, developing an architectural structure for software components, databases, user interfaces, and the operating environment.** Design turns WHAT into HOW, and the four things it structures are worth memorising as a list: software components, databases, user interfaces, and the operating environment. Every wrong option here is a different SDLC phase, so this question is really asking whether you can name the phases apart.

- ✗ **To monitor the installed system and make enhancements in response to changing business requirements.** Wrong. Monitoring and enhancing is SUPPORT, the phase after implementation.
- ✗ **To construct software components, verify and test the system, and prepare it for deployment to users.** Wrong. Building and testing is IMPLEMENTATION.
- ✗ **To gather information from stakeholders and define the functional requirements of the new system.** Wrong. Gathering and defining requirements is ANALYSIS.
- ✓ **To design the solution system based on requirements defined during analysis, developing an architectural structure for software components, databases, user interfaces, and the operating environment.** Correct. Architectural structure for components, databases, interfaces and the operating environment.

21. The answer is **System Prototyping, Throwaway Prototyping, and Agile Development are all rated Excellent for unclear user requirements.** All three work the same way: they put something in front of the user EARLY, so the requirement gets discovered by reaction rather than by description. The manual says it plainly on page 41: when requirements are unclear, system prototyping and throwaway prototyping are more appropriate, because they provide prototypes for users to interact with early in the SDLC. Agile joins them because each sprint delivers something a user can react to.

- ✗ **The V-Model is rated Excellent because its rigorous testing phases help uncover hidden or unclear requirements.** Wrong. The V-model tests against requirements you already have; it cannot discover ones nobody has stated.
- ✗ **Waterfall and Parallel models are rated Excellent because their structured approach forces requirements clarification.** Wrong, and backwards. Structure DEMANDS clear requirements rather than producing them.
- ✗ **The Iterative model alone is rated Excellent while all other methodologies are rated Poor for unclear requirements.** Wrong. It is not one methodology alone, and the others are not all rated Poor.
- ✓ **System Prototyping, Throwaway Prototyping, and Agile Development are all rated Excellent for unclear user requirements.** Correct. All three let users interact with something early, which is how unclear requirements become clear.

22. The answer is **Critical Path Method (CPM).** The manual lists five popular agile methods on page 40: Crystal Clear, Scrum, Extreme Programming, Lean Software Development, and Kanban. The Critical Path Method is a PROJECT SCHEDULING technique for working out which chain of tasks determines the finish date. It is a planning tool, not a development methodology, and it belongs to the predictive world rather than the agile one.

- ✓ **Critical Path Method (CPM).** Correct, this is the odd one out. CPM is a scheduling technique, not an agile method.
- ✗ **Scrum.** Wrong, it is on the list, and it is the best known of the five.

- ✗ **Extreme Programming (XP).** Wrong, it is on the list.
- ✗ **Kanban.** Wrong, it is on the list.

23. The answer is **Breaking a large system into smaller, manageable components or subsystems to simplify development and reduce complexity.** Divide and conquer is functional decomposition wearing its everyday name. Break the system into subsystems, let each team hold one part in their head, and make sure the parts still work as a whole. The same idea drives levelled data flow diagrams, where one process on a Level 0 diagram becomes a whole Level 1 diagram of its own.

- ✗ **Separating business requirements from technical specifications to prevent scope creep during development.** Wrong. That separation is useful but it is not what dividing and conquering means.
- ✓ **Breaking a large system into smaller, manageable components or subsystems to simplify development and reduce complexity.** Correct. Smaller manageable components, simpler development, less complexity.
- ✗ **Dividing the project budget equally among planning, analysis, design, and implementation phases.** Wrong. Nothing in the principle concerns budget allocation.
- ✗ **Distributing system development work among multiple competing vendor teams to accelerate delivery.** Wrong. Competing vendors is not decomposition; it is duplication.

24. The answer is **Activities to upgrade and maintain the system after deployment, including fixing bugs, making enhancements, and supporting users through a help desk.** Three activities, and the question names all three: maintain, enhance, support. MAINTAIN means fixing bugs and making minor adjustments. ENHANCE means the bigger changes forced by new regulations, new competition or new infrastructure. SUPPORT means the help desk. Remember too that support is part of the overall SDLC but is not normally counted as part of the initial development project.

- ✓ **Activities to upgrade and maintain the system after deployment, including fixing bugs, making enhancements, and supporting users through a help desk.** Correct. Upgrade and maintain after deployment, with bug fixing, enhancement and the help desk.
- ✗ **The process of conducting feasibility studies and producing the initial project proposal for management approval.** Wrong. Feasibility and the project proposal belong to PLANNING.
- ✗ **The phase where analysts gather and document user requirements before system design begins.** Wrong. Gathering and documenting requirements is ANALYSIS.
- ✗ **The activities carried out during system construction to ensure technical components meet design specifications.** Wrong. Construction against design specifications is IMPLEMENTATION.

25. The answer is **Because the cost of bugs can be immense, and most organisations spend more time and attention on testing than on writing the programs in the first place.** This is page 22 word for word, and the striking half is the second one: more time goes on testing than on writing the programs. The reason is economic rather than technical. A defect caught in testing costs a fix; the same defect caught by a customer costs the fix plus the damage plus the trust, which is why organisations spend where they do.

- ✗ **Because testing generates the official documentation required for regulatory compliance and audit purposes.** Wrong. Documentation is a by-product, not the reason.
- ✓ **Because the cost of bugs can be immense, and most organisations spend more time and attention on testing than on writing the programs in the first place.** Correct. The cost of bugs, and more effort spent testing than programming.
- ✗ **Because testing determines the final project budget and influences decisions about future system enhancements.** Wrong. Budget decisions belong to planning, and are revised throughout.

- ✗ **Because testing is the only phase where users are directly involved in the system development process.** Wrong. Users are involved throughout, most heavily in analysis.

26. The answer is The approach of dividing a system into smaller, manageable components or subsystems to focus analysis on individual areas of a business.

Functional decomposition is the analyst's basic move: a system contains subsystems, and those contain subsystems of their own. It lets you focus on one area of the business, on a group of related areas, or on the interfaces between them. Note the trap in the wrong options: decomposition divides the SYSTEM, not the work, the document, or the requirements.

- ✓ **The approach of dividing a system into smaller, manageable components or subsystems to focus analysis on individual areas of a business.** Correct. Dividing the system into subsystems so analysis can focus on one area at a time.
- ✗ **The method of assigning different system functions to separate teams of developers during implementation.** Wrong. That divides the TEAM, which is a management decision.
- ✗ **The process of documenting the functional requirements of a system in a structured specification document.** Wrong. That is writing a requirements specification.
- ✗ **The technique of breaking down a user's functional requirements into technical design specifications.** Wrong. That is the analysis-to-design translation, a different step.

27. The answer is A large, expensive, and complicated project with clear objectives, stable and comprehensive requirements, and no pressure for immediate implementation. Straight from page 30's list of when to use Waterfall. Every clause matters: LARGE and complicated, because the structure pays for itself at scale; CLEAR objectives and STABLE requirements, because going back is expensive; and NO pressure for immediate implementation, because nothing ships until the end. The list adds one more worth knowing: Waterfall also suits teams that are inexperienced or may change during implementation, because the documentation carries the knowledge.

- ✓ **A large, expensive, and complicated project with clear objectives, stable and comprehensive requirements, and no pressure for immediate implementation.** Correct, and it matches the manual's list clause for clause.
- ✗ **A startup environment where rapid prototyping and quick delivery of a minimum viable product are the primary goals.** Wrong. Rapid delivery of a minimum viable product is RAD or Agile.
- ✗ **A project with frequently changing customer requirements that demand continuous user feedback and iterative delivery.** Wrong, and it is Waterfall's worst case: changing requirements are what it handles badly.
- ✗ **A project where the development team is highly experienced with agile techniques and self-organising workflows.** Wrong. An experienced agile team is a reason to use Agile.

28. The answer is Ensuring systems are flexible and scalable to accommodate future business expansion, technological advancements, and changing user requirements, thereby extending the system's useful life. Two words carry it:

FLEXIBLE and SCALABLE. Three things change over a system's life, and the manual names them all: the business expands, the technology moves, and users want different things. Designing for that extends the system's useful life, which protects the money already spent. That last clause is why this principle sits beside justifying systems as capital investments.

- ✓ **Ensuring systems are flexible and scalable to accommodate future business expansion, technological advancements, and changing user requirements, thereby extending the system's useful life.** Correct. Flexible and scalable, for expansion, new technology and changing requirements.

- ✗ **Building systems exclusively on the latest technology platforms regardless of organisational budget constraints.** Wrong. Newest is not the same as adaptable, and budget always constrains.
- ✗ **Designing systems with the maximum number of features possible so future enhancements are not needed.** Wrong, and it is the opposite: cramming in features now does not remove the need to change later.
- ✗ **Establishing rigid technical standards that prevent unauthorised modifications to the system after deployment.** Wrong. Rigidity prevents change rather than accommodating it.

29. The answer is **There is a lack of familiarity with the technology, requirements are unclear, or there are complex technical risks that must be explored through design prototypes before the actual system is built.** Throwaway prototyping is a RISK tool. You build something disposable precisely where you do not yet know enough: an unfamiliar technology, a requirement nobody can state, a technical problem that might not have a solution. You learn the answer, you throw the prototype away, and you then build the real system properly with a structured methodology.

- ✗ **The project has well-understood and stable requirements that are unlikely to change during the development lifecycle.** Wrong. Stable, well-understood requirements are the case for Waterfall.
- ✗ **The organisation requires the highest possible system reliability and has zero tolerance for defects in production.** Wrong here, though it is genuinely tempting: the manual DOES say throwaway prototyping suits high reliability. But the question asks when it is most appropriate overall, and unfamiliarity, unclear requirements and technical risk are the criteria the manual names first.
- ✗ **The development team is highly experienced and needs no exploratory phase before committing to the final design.** Wrong. If no exploration is needed, there is nothing for a throwaway prototype to do.
- ✓ **There is a lack of familiarity with the technology, requirements are unclear, or there are complex technical risks that must be explored through design prototypes before the actual system is built.** Correct. Unfamiliar technology, unclear requirements, or complex technical risks explored before the real build.

30. The answer is **The V-model, which is useful when reliability is important because it emphasises testing at each development phase through verification and validation.** The V-model pairs EVERY development phase with its own testing phase: down the left side you build, up the right side you verify and validate. Nothing is deferred to a test at the end. Medical equipment and missile control are the manual's own examples of critical reliability, against games and internet video where it is merely important. One honest note. Page 41 says throwaway prototyping is most appropriate when reliability is a high priority, and that the V-model is useful when reliability is important. Throwaway prototyping is not offered here, so the V-model is the intended answer, and it is the right one among the four given.

- ✗ **RAD, because its time-box approach ensures rapid delivery of reliable system increments to users.** Wrong. RAD trades quality for speed, which is the wrong trade for safety-critical work.
- ✗ **Agile development, because its iterative nature allows defects to be detected and corrected after each sprint.** Wrong. Detecting defects AFTER each sprint is later than a safety-critical system can afford.
- ✗ **The Incremental Model, because delivering high-risk functions first ensures the most critical components are reliable.** Wrong. Ordering the work by risk is not the same as verifying every phase.
- ✓ **The V-model, which is useful when reliability is important because it emphasises testing at each development phase through verification and validation.** Correct. Verification and validation built into every phase, which is what V stands for.

Module Two, Fact-Finding and Use Cases

16 questions. Module Two. Part of compulsory Question Two. Cover the answers before you start.

1. **TEST 2 Q3** Match each fact-finding technique with its core definition. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Observation of the work environment	1. Reviewing organisation charts, memos, forms, reports and databases to understand current business rules and processes
B. Sampling of existing documentation, forms and databases	2. Highly structured group meetings conducted to analyse problems and define requirements
C. Discovery prototyping	3. Building a small-scale, representative or working model of requirements so users can interact with it and give feedback
D. Questionnaires	4. Systems analyst participates in or watches a person performing job activities to learn about the system firsthand
E. Joint requirements planning (JRP)	5. Special-purpose document used to collect information and opinions from many respondents quickly and at low cost

2. **TEST 2 Q4** An analyst wants a department head to describe the credit-approval process in his own words rather than following a rigid script, while she keeps only one overarching topic in mind for the session. Which two features characterise the interview approach she is using? **WHICH TWO** (a) The interviewer relies on the interviewee to help direct the conversation (b) It is conducted with only a general goal or subject in mind (c) It uses a specific, fixed set of questions prepared in advance (d) It restricts every response to a predefined set of choices
3. **TEST 2 Q10** According to requirements discovery theory, a "problem" in the context of requirements discovery is best defined as which of the following? (a) A defect discovered exclusively during the system testing phase (b) Any deviation from the originally approved project schedule (c) A disagreement between two stakeholders about a user-interface colour scheme (d) An undesirable situation that prevents the organisation from fully achieving its mission, vision, goals, and objectives
4. **TEST 2 Q11** In the 'Order Glasses' use-case diagram, the Patient actor interacts with 'Get Prescription', 'Select Frame', 'Place Order', 'Make Payment', and 'Fitting and Adjustment'. Before finalising the diagram, the analyst checks her actor identification. Which two questions did she likely use to confirm the Patient qualifies as an actor? **WHICH TWO** (a) Which internal process transforms input data into output data? (b) Who or what provides input to the system? (c) Who or what receives output from the system? (d) Are there any events automatically triggered at a predetermined time within the store's internal database?
5. **TEST 2 Q13** Match each use-case narrative element with its definition. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Preconditions	1. The step-by-step sequence of actions between actor and system when everything works correctly
B. Main flow (basic flow)	2. Conditions that must be true before the use case can begin
C. Alternative flow (extension)	3. Describes what must be true after the use case has completed
D. Postconditions	4. Describes variations, exceptions, or error situations that branch off the main flow

6. **TEST 2 Q14** While reviewing personnel files during fact-finding for a new HR system, an analyst discovers detailed salary and medical history information about several employees. Which two actions are consistent with the stated fact-finding ethics for systems analysts? **WHICH TWO** (a) Refrain from using the sensitive information for any purpose beyond the project (b) Use the medical history information to make personal judgements about employee reliability (c) Take steps to protect the sensitive information from people who might misuse it (d) Share interesting salary details informally with curious colleagues

7. **TEST 2 Q16** Match each actor in the bank generalisation example with its correct role.

MATCH

ITEM	DEFINITIONS, SHUFFLED
A. Customer	1. A specialised customer type that inherits Customer use cases such as Make Deposit and Withdraw Money
B. Savings Account Holder	2. A specialised customer type that inherits Customer use cases such as Make Deposit and Withdraw Money
C. Current Account Holder	3. A specialised customer type that inherits Customer use cases such as Make Deposit and Withdraw Money

8. **TEST 2 Q17** Which of the following best describes the outcome for an analyst who violates fact-finding ethics by misusing sensitive information gathered during a project? (a) The project's requirements definition document becomes legally void (b) The organisation is required to restart the entire systems development life cycle from the planning phase (c) The analyst risks losing the respect, credibility, and confidence of users and management, and both the analyst and organisation may face legal liability (d) The analyst automatically receives a formal promotion for demonstrating initiative

9. **TEST 2 Q19** An analyst is observing a warehouse clerk to understand how order picking really happens. Which two of the following are recommended guidelines for conducting this observation effectively? **WHICH TWO** (a) Focus primarily on trivial, one-off activities to capture edge cases (b) Keep a low profile so the worker's normal behaviour is not disrupted (c) Obtain permission from the appropriate supervisor before observing (d) Interrupt the worker regularly to ask clarifying questions in real time

10. **TEST 2 Q23** Match each association notation with its correct meaning. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Association with an arrowhead touching the use case	1. A line connecting an actor directly to a use case, representing participation in that use case
B. Basic association line (general)	2. A line connecting an actor directly to a use case, representing participation in that use case
C. Association without an arrowhead	3. A line connecting an actor directly to a use case, representing participation in that use case

11. TEST 2 Q24 Match each type of actor with its definition. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Primary business actor	1. Directly interfaces with the system to initiate or trigger a business event
B. Primary system actor	2. Benefits from the execution of the use case by receiving something of measurable or observable value
C. External server actor	3. Receives something of measurable or observable value (output) from the use case but is not the primary actor
D. External receiver actor	4. Responds to a request from the use case

12. TEST 2 Q25 Match each observation guideline with its underlying rationale. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Do not interrupt individuals while they are working	1. Preserves accurate detail before memory of the activity fades
B. Take notes during or immediately following the observation	2. Avoids disrupting the natural workflow being studied
C. Obtain permission from the appropriate supervisor before observing	3. Reduces the likelihood that the worker alters normal behaviour because they feel watched
D. Keep a low profile during the observation	4. Ensures the observation is authorised and does not improperly disrupt operations

13. TEST 2 Q26 Which two of the following are legitimate reasons for conducting an interview during fact-finding? **WHICH TWO** (a) To solicit ideas and opinions from stakeholders (b) To verify facts already gathered through other techniques (c) To automatically resolve all conflicting stakeholder requirements without further analysis (d) To eliminate the need for any written requirements documentation
14. TEST 2 Q29 Which two of the following are documented consequences of incorrect system requirements? **WHICH TWO** (a) The system will automatically pass all user acceptance testing (b) The system may be delivered later than promised (c) The system may cost more than originally projected (d) The system's data flows will automatically balance across all decomposition levels
15. TEST 2 Q30 Which two outputs are produced when performing use-case modelling? **WHICH TWO** (a) The use-case narrative (b) The entity-relationship diagram (c) The use-case diagram (d) The Gantt chart
16. TEST 2 Q6 Match each banking use case with its correct relationship description. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Make deposit	1. Depends on Make deposit being performed first
B. Make withdrawal	2. Depends on Open account being performed first
C. Withdraw Money	3. Depicted in the example as an additional use case specific to the Current Account Holder
D. Request Overdraft	4. An inherited use case available to both Savings Account Holder and Current Account Holder via generalisation

1. The pairing, in full. Five techniques, and the quickest way to hold them apart is to ask WHO is in the room and WHAT is being examined. Observation watches one person working. Sampling reads documents rather than talking to anyone. Discovery prototyping shows a model and collects reactions. Questionnaires reach many people cheaply but shallowly. JRP puts everyone in one highly structured meeting.

- **Observation of the work environment** goes with Systems analyst participates in or watches a person performing job activities to learn about the system firsthand
- **Sampling of existing documentation, forms and databases** goes with Reviewing organisation charts, memos, forms, reports and databases to understand current business rules and processes
- **Discovery prototyping** goes with Building a small-scale, representative or working model of requirements so users can interact with it and give feedback
- **Questionnaires** goes with Special-purpose document used to collect information and opinions from many respondents quickly and at low cost
- **Joint requirements planning (JRP)** goes with Highly structured group meetings conducted to analyse problems and define requirements

2. The two answers are **The interviewer relies on the interviewee to help direct the conversation** and **It is conducted with only a general goal or subject in mind**. This is the UNSTRUCTURED interview. It is conducted with only a general goal in mind and lets the interviewee steer, which is exactly what you want when you do not yet know enough to write good questions. Its opposite, the structured interview, uses a fixed set of prepared questions and is better once you know what to ask.

- ✓ **The interviewer relies on the interviewee to help direct the conversation.** Correct. Letting the interviewee direct the conversation is the defining feature.
- ✓ **It is conducted with only a general goal or subject in mind.** Correct. A general goal or subject rather than a script.
- ✗ **It uses a specific, fixed set of questions prepared in advance.** Wrong. A fixed prepared set of questions describes the STRUCTURED interview.
- ✗ **It restricts every response to a predefined set of choices.** Wrong. Restricting responses to set choices describes a closed-ended question or a questionnaire.

3. The answer is **An undesirable situation that prevents the organisation from fully achieving its mission, vision, goals, and objectives**. Notice how wide this definition is. A problem is anything standing between the organisation and its mission, vision, goals and objectives. It is not restricted to bugs, or to lateness, or to anything technical at all. That breadth is the point: it is why the analyst starts with the business rather than the software.

- ✗ **A defect discovered exclusively during the system testing phase.** Wrong, and far too narrow. Testing defects are one small kind of problem.
- ✗ **Any deviation from the originally approved project schedule.** Wrong. Schedule slippage is a project management issue, not the definition.
- ✗ **A disagreement between two stakeholders about a user-interface colour scheme.** Wrong. A preference disagreement is not an obstacle to the mission.
- ✓ **An undesirable situation that prevents the organisation from fully achieving its mission, vision, goals, and objectives.** Correct. An undesirable situation preventing the organisation achieving its mission, vision, goals and objectives.

4. The two answers are **Who or what provides input to the system?** and **Who or what receives output from the system?**. An actor is defined by sitting OUTSIDE the system and exchanging something with it, so the two identifying questions are about in-

put and output across the boundary. A patient supplies a prescription and a payment, and receives glasses and a fitting, so on both counts the Patient is an actor.

- ✗ **Which internal process transforms input data into output data?.** Wrong. An internal transformation is a PROCESS, and processes are inside the boundary, which is what actors are not.
- ✓ **Who or what provides input to the system?.** Correct. Who or what provides input is one of the two identifying questions.
- ✓ **Who or what receives output from the system?.** Correct. Who or what receives output is the other.
- ✗ **Are there any events automatically triggered at a predetermined time within the store's internal database?.** Wrong. That describes a TIME or temporal event, which can be an actor in some notations but is not how a patient was identified.

5. The pairing, in full. Read them as a story with a before and an after.

PRECONDITIONS are what must already be true to start. The MAIN FLOW is the happy path, where nothing goes wrong. ALTERNATIVE FLOWS are the branches for exceptions and errors. POSTCONDITIONS are what is true once it is over. Pre and post are the bookends; main and alternative are the story between them.

- **Preconditions** goes with Conditions that must be true before the use case can begin
- **Main flow (basic flow)** goes with The step-by-step sequence of actions between actor and system when everything works correctly
- **Alternative flow (extension)** goes with Describes variations, exceptions, or error situations that branch off the main flow
- **Postconditions** goes with Describes what must be true after the use case has completed

6. The two answers are **Refrain from using the sensitive information for any purpose beyond the project** and **Take steps to protect the sensitive information from people who might misuse it**. Two duties, and they are the two halves of confidentiality. The first is about YOU: do not use what you learn for anything outside the project. The second is about EVERYONE ELSE: actively protect it from people who might misuse it. Access granted for a purpose is limited to that purpose, and protecting it is a duty rather than a courtesy.

- ✓ **Refrain from using the sensitive information for any purpose beyond the project.** Correct. Use it only for the project it was given for.
- ✗ **Use the medical history information to make personal judgements about employee reliability.** Wrong, and it is precisely the misuse the ethics forbid.
- ✓ **Take steps to protect the sensitive information from people who might misuse it.** Correct. Actively protect it from misuse by others.
- ✗ **Share interesting salary details informally with curious colleagues.** Wrong. Informal sharing is a breach however casual it feels.

7. The pairing, in full. Generalisation between actors works exactly like inheritance.

CUSTOMER is the abstract parent holding everything common. Both account holders inherit those use cases, and the current account holder adds one of its own, Request Overdraft, which the savings holder does not get. Draw the shared behaviour once on the parent and only the differences on the children.

- **Customer** goes with The abstract actor representing any person who uses the banking system and can make deposits or withdrawals
- **Savings Account Holder** goes with A specialised customer type that inherits Customer use cases such as Make Deposit and Withdraw Money
- **Current Account Holder** goes with A specialised customer type that inherits Customer use cases but may also access Request Overdraft

8. The answer is **The analyst risks losing the respect, credibility, and confidence of users and management, and both the analyst and organisation may face le-**

gal liability. Two kinds of damage, and the professional one is listed first for a reason. An analyst works by being trusted with things people would not otherwise say, so losing respect, credibility and confidence ends the ability to do the job at all. The legal liability, falling on both the analyst and the organisation, comes on top of that.

- ✗ **The project's requirements definition document becomes legally void.** Wrong. A breach of ethics does not void a document.
- ✗ **The organisation is required to restart the entire systems development life cycle from the planning phase.** Wrong. Nothing forces a restart of the life cycle.
- ✓ **The analyst risks losing the respect, credibility, and confidence of users and management, and both the analyst and organisation may face legal liability.** Correct. Lost trust and credibility, plus legal liability for analyst and organisation.
- ✗ **The analyst automatically receives a formal promotion for demonstrating initiative.** Wrong, and it is the joke option.

9. The two answers are **Keep a low profile so the worker's normal behaviour is not disrupted** and **Obtain permission from the appropriate supervisor before observing.** Observation exists to see what people ACTUALLY do rather than what they say they do, and both correct answers protect that. Keeping a low profile limits the effect of being watched, and getting the supervisor's permission first keeps the exercise authorised and stops it disrupting operations.

- ✗ **Focus primarily on trivial, one-off activities to capture edge cases.** Wrong. Trivial one-off activities are the least representative thing you could watch.
- ✓ **Keep a low profile so the worker's normal behaviour is not disrupted.** Correct. A low profile keeps behaviour normal.
- ✓ **Obtain permission from the appropriate supervisor before observing.** Correct. Permission first, from the appropriate supervisor.
- ✗ **Interrupt the worker regularly to ask clarifying questions in real time.** Wrong, and it destroys the point: interrupting changes the very workflow you came to see.

10. The pairing, in full. The arrowhead carries the meaning, and it points at what was STARTED. An arrow touching the use case says the actor at the far end initiated it, which is what a primary actor does. No arrowhead means the actor was involved without starting anything, which is what an external server or receiver actor does. The plain line is simply participation.

- **Association with an arrowhead touching the use case** goes with Indicates the use case was initiated by the actor at the other end of the line
- **Basic association line (general)** goes with A line connecting an actor directly to a use case, representing participation in that use case
- **Association without an arrowhead** goes with Indicates an interaction between the use case and an external server or receiver actor

11. The pairing, in full. Split them on two questions. Is the actor PRIMARY or EXTERNAL, that is does it drive the use case or serve it? And does it GIVE or RECEIVE? The primary business actor benefits, the primary system actor touches the system and triggers the event, the external server responds when asked, and the external receiver gets an output without being the reason the use case ran. Watch the pair that catches people: primary BUSINESS actor benefits from the outcome, primary SYSTEM actor is the one physically interacting with the system. They are often different people, such as a customer who benefits and a clerk who types.

- **Primary business actor** goes with Benefits from the execution of the use case by receiving something of measurable or observable value
- **Primary system actor** goes with Directly interfaces with the system to initiate or trigger a business event

- **External server actor** goes with Responds to a request from the use case
- **External receiver actor** goes with Receives something of measurable or observable value (output) from the use case but is not the primary actor

12. The pairing, in full. Three of the four defend the same thing: seeing the work as it really is. Not interrupting protects the workflow, keeping a low profile protects the behaviour, and permission protects the operation. Only the note-taking rule is about YOU rather than them, and it exists because detail fades fast once you leave the room.

- **Do not interrupt individuals while they are working** goes with Avoids disrupting the natural workflow being studied
- **Take notes during or immediately following the observation** goes with Preserves accurate detail before memory of the activity fades
- **Obtain permission from the appropriate supervisor before observing** goes with Ensures the observation is authorised and does not improperly disrupt operations
- **Keep a low profile during the observation** goes with Reduces the likelihood that the worker alters normal behaviour because they feel watched

13. The two answers are **To solicit ideas and opinions from stakeholders** and **To verify facts already gathered through other techniques**. Interviews do two jobs that no document can. They gather ideas and OPINIONS, which only exist in people's heads, and they VERIFY what you learned elsewhere, because a document tells you what the process is supposed to be while a person tells you what it actually is. Both wrong options overclaim: interviewing neither settles conflicts by itself nor removes the need to write anything down.

- ✓ **To solicit ideas and opinions from stakeholders.** Correct. Ideas and opinions come from people.
- ✓ **To verify facts already gathered through other techniques.** Correct. Verifying facts gathered by other means is a standard purpose.
- ✗ **To automatically resolve all conflicting stakeholder requirements without further analysis.** Wrong. Conflicts surface in interviews; resolving them takes analysis and negotiation.
- ✗ **To eliminate the need for any written requirements documentation.** Wrong. What is said still has to be documented, or it is lost.

14. The two answers are **The system may be delivered later than promised** and **The system may cost more than originally projected**. The same two consequences the manual gives on page 7 and repeats in Module One: LATE and OVER BUDGET, alongside a system that does not solve the problem. The reason is that a requirements error is discovered last, after the design, the code and the data structures have all been built on it, so fixing it means unpicking everything above it.

- ✗ **The system will automatically pass all user acceptance testing.** Wrong, and it is the opposite: wrong requirements are what acceptance testing exists to catch.
- ✓ **The system may be delivered later than promised.** Correct. Late delivery is one of the named consequences.
- ✓ **The system may cost more than originally projected.** Correct. Cost overrun is the other.
- ✗ **The system's data flows will automatically balance across all decomposition levels.** Wrong. Balancing is a property of how you DRAW the diagrams, unrelated to requirement quality.

15. The two answers are **The use-case narrative** and **The use-case diagram**. Use-case modelling always produces a PAIR, and marks are lost by naming only one. The DIAGRAM is the picture: actors, use cases, and the lines between them, showing scope at a glance. The NARRATIVE is the text behind each use case: preconditions, main flow, alternative flows, postconditions. The diagram shows what exists; the narrative says what happens.

- ✓ **The use-case narrative.** Correct. The narrative is the written half.

- ✗ **The entity-relationship diagram.** Wrong. An ERD is the output of DATA modelling, not use-case modelling.
- ✓ **The use-case diagram.** Correct. The diagram is the visual half.
- ✗ **The Gantt chart.** Wrong. A Gantt chart is a project scheduling tool.

16. The pairing, in full. Two different relationships are being tested at once, which is what makes this hard. The first two are **DEPENDS ON**, and the manual gives the chain on page 62: withdrawing cash depends on first depositing cash, and depositing cash depends on opening a bank account. Read it as an order in time, because a depends-on relationship says one use case cannot happen until another has. The last two are **GENERALISATION**. Customer is the abstract parent, so Withdraw Money and Check Balance are inherited by BOTH account holders. Request Overdraft belongs only to the Current Account Holder, because it is the extra use case that specialisation adds. So: depends-on is about **ORDER**, generalisation is about **INHERITANCE**. If a description mentions one being performed first, it is depends-on; if it mentions who has access, it is generalisation.

- **Make deposit** goes with Depends on Open account being performed first
- **Make withdrawal** goes with Depends on Make deposit being performed first
- **Withdraw Money** goes with An inherited use case available to both Savings Account Holder and Current Account Holder via generalisation
- **Request Overdraft** goes with Depicted in the example as an additional use case specific to the Current Account Holder

OBJECTIVE SECTION

Module Three, Data Flow Diagrams

13 questions. Module Three. Part of compulsory Question Two. Cover the answers before you start.

1. **TEST 2 Q1** When decomposing the Level 0 DFD of an ATM system into a Level 1 DFD for 'Withdraw Cash', the analyst ensures every input and output shown for 'Withdraw Cash' at Level 0 is still accounted for among the Level 1 sub-processes. Which two statements describe this practice and its purpose? **WHICH TWO** (a) This practice is called gray-holing (b) Its purpose is to eliminate the need for a context diagram (c) This practice is called balancing (d) Its purpose is to ensure consistency between the parent process and its decomposed sub-processes
2. **TEST 2 Q2** In a Data Flow Diagram, which symbol represents "data at rest," as opposed to "data in motion"? (a) Data flow (b) Process (c) Data store (d) External agent
3. **TEST 2 Q7** Which of the following most accurately describes why DFDs are described as "purely logical, not physical models"? (a) Because they can only be drawn using computer-aided software engineering tools (b) Because they represent the physical wiring and network topology of the system's servers (c) Because they capture what the organisation does and how it processes data without specifying implementation details such as programming languages or hardware (d) Because they must always be paired with a corresponding entity-relationship diagram before they can be considered valid
4. **TEST 2 Q9** A DFD process labelled "Create a new member account" shows inputs of "Membership application" and "Existing account" but has no outgoing data flow at all. This situation is best classified as which type of error? (a) A balancing violation (b) A gray hole (c) A miracle (d) A black hole

5. **TEST 2 Q12** An analyst drafts a DFD in which data flows directly from the 'Employees' data store to the 'Accounts Receivable Department' external entity, without passing through any process. Which two statements correctly evaluate this design? **WHICH TWO**
- (a) No correction is needed because data stores are considered a type of process in Gane-Sarson notation (b) A process is needed to retrieve and present the data from the store to the external agent (c) This is an illegal data flow because an external agent cannot access a data store directly (d) This is a legal data flow because data stores may output directly to any external agent that needs the information
6. **TEST 2 Q15** According to the rules for constructing a Context Diagram (Process 0), which of the following is generally true? (a) The Context Diagram must show every sub-process contained within the system (b) The Context Diagram is drawn only after all Level 1 DFDs have been completed (c) Data stores are usually excluded unless they are owned by systems or processes other than the one being documented (d) Data stores must always be shown explicitly, regardless of ownership
7. **TEST 2 Q18** Match each illegal data flow with the correction required to fix it. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Data flowing directly from one data store to another	1. A process must be inserted to exchange the data flow between the two boundaries
B. Data flowing directly from one external agent to another	2. A process must be inserted to exchange the data flow between the two boundaries
C. Data flowing directly from a data store to an external agent	3. A process must be inserted to exchange the data flow between the two boundaries

8. **TEST 2 Q20** Match each DFD error type with its correct description. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Miracle	1. A process's inputs are insufficient to logically produce its stated outputs
B. Gray hole	2. A process's inputs are insufficient to logically produce its stated outputs
C. Black hole	3. A process's inputs are insufficient to logically produce its stated outputs

9. **TEST 2 Q21** Which of the following data flows would be considered illegal according to standard DFD construction rules? (a) A data flow moving from an external entity into a process (b) A data flow moving from a process into a data store (c) A data flow moving directly from one data store to another data store without passing through a process (d) A data flow moving from a data store into a process
10. **TEST 2 Q22** Which two of the following statements about Data Flow Diagrams are accurate? **WHICH TWO** (a) DFDs are purely logical models that contain no implementation details (b) A system may have multiple Level 0 DFDs, one for each department (c) DFDs must specify the exact programming language used to implement each process (d) There can be only one Level 0 DFD for a given system
11. **TEST 2 Q27** A process named "Add 1 to the counter" is considered problematic in DFD naming conventions primarily because it is which of the following? (a) Named using a noun rather than a verb (b) Procedural, describing how a step is implemented rather than what business function is performed (c) Missing a corresponding data store (d) Insufficiently detailed for a programmer to implement
12. **TEST 2 Q28** Which of the following best describes the relationship between a Level 0 DFD and a Level 1 DFD in the same process model? (a) A Level 1 DFD must contain fewer data flows than the Context Diagram (b) A Level 1 DFD replaces the need for a

Context Diagram entirely (c) A Level 1 DFD always precedes and determines the structure of the Level 0 DFD (d) A Level 1 DFD further decomposes one specific process from the Level 0 DFD into more detailed sub-processes, while conserving that process's original inputs and outputs

13. **TEST 2 Q8** Match each term with its correct definition. **MATCH**

ITEM	DEFINITIONS, SHUFFLED
A. Business process	1. A specific ordering of work activities across time and place, with a beginning, an end, and defined inputs and outputs
B. Process	2. A collection of activities designed to produce a specific output for a particular customer or market
C. Process modelling	3. A pictorial representation of reality
D. Model	4. A technique for organising and documenting a system's processes, inputs, outputs, and data stores

1. The two answers are **This practice is called balancing** and **Its purpose is to ensure consistency between the parent process and its decomposed sub-processes**. BALANCING is the rule that a child diagram must show exactly the same net inputs and outputs as the parent process it explodes. Nothing may appear from nowhere and nothing may vanish. It is what keeps a set of levelled diagrams describing ONE system rather than several, and it is the first thing an examiner checks when you hand in a Level 1 diagram.

- ✗ **This practice is called gray-holing.** Wrong. A gray hole is a DEFECT, a process whose outputs cannot be produced from its inputs, not a good practice.
- ✗ **Its purpose is to eliminate the need for a context diagram.** Wrong. The context diagram is the Level 0 boundary view and is still needed.
- ✓ **This practice is called balancing.** Correct. This is the definition of balancing.
- ✓ **Its purpose is to ensure consistency between the parent process and its decomposed sub-processes.** Correct. Consistency between parent and children is exactly what balancing guarantees.

2. The answer is **Data store**. Four symbols, and this pair of phrases separates two of them cleanly. A DATA FLOW is data in MOTION, an arrow carrying data from one place to another. A DATA STORE is data at REST, data held so it can be used later. If the data is going somewhere it is a flow; if it is sitting somewhere it is a store.

- ✗ **Data flow.** Wrong, and it is the exact opposite: a data flow is data in motion.
- ✗ **Process.** Wrong. A process TRANSFORMS data; it does not hold it.
- ✓ **Data store.** Correct. A store holds data at rest.
- ✗ **External agent.** Wrong. An external agent sits outside the system boundary and supplies or receives data.

3. The answer is **Because they capture what the organisation does and how it processes data without specifying implementation details such as programming languages or hardware**. A logical model says WHAT happens; a physical model says HOW it is built. A DFD names the processes, the data that moves between them and the stores it rests in, and says nothing about the language, the database product or the servers. That is deliberate: it lets you agree the business behaviour with users who neither know nor care what it will be written in.

- ✗ **Because they can only be drawn using computer-aided software engineering tools.** Wrong. The tool used to draw it has nothing to do with whether the model is logical.
- ✗ **Because they represent the physical wiring and network topology of the system's servers.** Wrong, and it is the opposite: wiring and topology are exactly the physical detail a DFD leaves out.
- ✓ **Because they capture what the organisation does and how it processes data without specifying implementation details such as programming languages or hardware.** Correct. What the organisation does, without implementation detail.
- ✗ **Because they must always be paired with a corresponding entity-relationship diagram before they can be considered valid.** Wrong. A DFD is valid on its own, though the two models are often drawn together.

4. The answer is **A black hole**. Three named process errors, and the pictures are worth remembering. A BLACK HOLE takes data in and lets none out, so data disappears. A MIRACLE produces output from no input, so data appears from nowhere. A GRAY HOLE has both, but the inputs are not enough to produce the stated outputs. Here data goes in and nothing comes out, so it is a black hole.

- ✗ **A balancing violation.** Wrong. Balancing concerns consistency BETWEEN levels, not one process on one diagram.
- ✗ **A gray hole.** Wrong. A gray hole does produce output, just not output the inputs could justify.
- ✗ **A miracle.** Wrong, and reversed: a miracle has output with no input.
- ✓ **A black hole.** Correct. Inputs but no outputs at all: data goes in and never comes out.

5. The two answers are **A process is needed to retrieve and present the data from the store to the external agent** and **This is an illegal data flow because an external agent cannot access a data store directly**. The rule is simple and the examiner leans on it: EVERY data flow must touch at least one process. A store cannot talk to an external agent, a store cannot talk to another store, and an agent cannot talk to another agent, all for the same reason: nothing would be doing any work. Something has to retrieve the data and present it, and that something is a process.

- ✗ **No correction is needed because data stores are considered a type of process in Gane-Sarson notation.** Wrong. A data store is not a process in any notation; they are separate symbols with separate rules.
- ✓ **A process is needed to retrieve and present the data from the store to the external agent.** Correct. A process must retrieve and present the data.
- ✓ **This is an illegal data flow because an external agent cannot access a data store directly.** Correct. Store to external agent with no process between is an illegal flow.
- ✗ **This is a legal data flow because data stores may output directly to any external agent that needs the information.** Wrong, and it is the misconception the question is testing.

6. The answer is **Data stores are usually excluded unless they are owned by systems or processes other than the one being documented**. A context diagram shows the system as ONE process and draws the boundary around it. Everything inside is hidden, so internal data stores do not appear. The exception is a store the system does not own, belonging to some other system, because that is genuinely outside the boundary and the flow to it has to be shown.

- ✗ **The Context Diagram must show every sub-process contained within the system.** Wrong, and it is the opposite: the whole system is a single process, so no sub-processes appear.
- ✗ **The Context Diagram is drawn only after all Level 1 DFDs have been completed.** Wrong. The context diagram comes FIRST and defines the scope the lower levels decompose.
- ✓ **Data stores are usually excluded unless they are owned by systems or processes other than the one being documented.** Correct. Stores are excluded unless owned elsewhere.
- ✗ **Data stores must always be shown explicitly, regardless of ownership.** Wrong. Internal stores are hidden at this level.

7. The pairing, in full. One rule generates all three corrections: EVERY data flow must touch a process. So whenever two non-process symbols are joined directly, the fix is always the same, insert a process. Only the wording of what that process does changes: it MOVES data between stores, EXCHANGES data between agents, or RETRIEVES and PRESENTS data from a store to an agent.

- **Data flowing directly from one data store to another** goes with A process must be inserted to move data from one data store to another
- **Data flowing directly from one external agent to another** goes with A process must be inserted to exchange the data flow between the two boundaries
- **Data flowing directly from a data store to an external agent** goes with A process must be inserted to present or retrieve the data from the store on the agent's behalf

8. The pairing, in full. Three failures, and the names are pictures. A BLACK HOLE swallows: data goes in, nothing comes out. A MIRACLE creates: output appears with no input to explain it. A GRAY HOLE is the subtle one, because it has both inputs and out-

puts, but the inputs could not possibly produce those outputs. Gray holes are the commonest real fault and the hardest to spot.

- **Miracle** goes with A process has outputs but no inputs, so data is produced from nothing
- **Gray hole** goes with A process's inputs are insufficient to logically produce its stated outputs
- **Black hole** goes with A process has inputs but no outputs, so data enters and then disappears

9. The answer is **A data flow moving directly from one data store to another data store without passing through a process**. Read the four options and notice that three of them touch a process. That is the whole rule: a legal flow always has a process at one end. Store to store has no process anywhere, so nothing is doing the moving, and the flow is illegal.

- ✗ **A data flow moving from an external entity into a process**. Legal. An external entity supplying a process is the normal way data enters the system.
- ✗ **A data flow moving from a process into a data store**. Legal. A process writing to a store is how data comes to rest.
- ✓ **A data flow moving directly from one data store to another data store without passing through a process**. Correct, this is the illegal one. No process at either end.
- ✗ **A data flow moving from a data store into a process**. Legal. A process reading from a store is routine.

10. The two answers are **DFDs are purely logical models that contain no implementation details** and **There can be only one Level 0 DFD for a given system**. Both correct statements protect the same idea: a DFD describes ONE system, logically. Purely logical means no language, no hardware, no database product. Only one Level 0 means there is a single agreed top-level view of the whole system, which every lower level then decomposes. Allow two Level 0 diagrams and you no longer have one system, you have two.

- ✓ **DFDs are purely logical models that contain no implementation details**. Correct. Logical, with implementation deliberately excluded.
- ✗ **A system may have multiple Level 0 DFDs, one for each department**. Wrong. Departments are not systems; one system gets one Level 0.
- ✗ **DFDs must specify the exact programming language used to implement each process**. Wrong, and it contradicts the first correct answer.
- ✓ **There can be only one Level 0 DFD for a given system**. Correct. Exactly one Level 0 for a given system.

11. The answer is **Procedural, describing how a step is implemented rather than what business function is performed**. This is the logical-model rule biting at the level of a single label. "Add 1 to the counter" is a line of code, not a business function. A DFD process should be named for what the BUSINESS does, in a verb plus object form such as Record Payment or Validate Order. If the name would still make sense to someone who has never seen a computer, it is at the right level.

- ✗ **Named using a noun rather than a verb**. Wrong, and it is the tempting one: the name does start with a verb, Add. The fault is not grammar, it is level.
- ✓ **Procedural, describing how a step is implemented rather than what business function is performed**. Correct. It describes implementation rather than business function.
- ✗ **Missing a corresponding data store**. Wrong. Not every process needs a data store.
- ✗ **Insufficiently detailed for a programmer to implement**. Wrong, and backwards: the problem is that it is too implementation-specific, not too vague.

12. The answer is **A Level 1 DFD further decomposes one specific process from the Level 0 DFD into more detailed sub-processes, while conserving that process's original inputs and outputs**. Two ideas in one sentence, and both carry

marks. DECOMPOSITION: a Level 1 diagram explodes ONE process from Level 0, not the whole diagram. CONSERVATION: the inputs and outputs of that parent process must all still be there, which is balancing. Levelling without balancing produces a set of diagrams that quietly contradict each other.

- ✗ **A Level 1 DFD must contain fewer data flows than the Context Diagram.** Wrong, and usually the reverse: decomposing reveals MORE detail, not less.
- ✗ **A Level 1 DFD replaces the need for a Context Diagram entirely.** Wrong. The context diagram sets the boundary and stays.
- ✗ **A Level 1 DFD always precedes and determines the structure of the Level 0 DFD.** Wrong direction. Level 0 comes first and Level 1 decomposes part of it.
- ✓ **A Level 1 DFD further decomposes one specific process from the Level 0 DFD into more detailed sub-processes, while conserving that process's original inputs and outputs.** Correct. One process decomposed, with its inputs and outputs conserved.

13. The pairing, in full. Two of these look interchangeable and are not, so take them in the manual's own words. A BUSINESS PROCESS is a collection of activities producing a specific output for a particular customer or market. Its emphasis is on HOW work is done in an organisation, in contrast to a product, which is about what. Making lunch is the example: the output is the lunch, and the activities inside it are the processes. A PROCESS is narrower and more precisely shaped: a specific ordering of work activities across time and place, with a beginning, an end, and clearly defined inputs and outputs. The manual calls it a structure for action. Assemble sandwich ingredients is a process; making lunch is the business process containing it. A MODEL is a pictorial representation of reality, and PROCESS MODELLING is the technique for organising and documenting a system's processes, inputs, outputs and data stores, whose main tool is the Data Flow Diagram. The short version: the business process is the WHOLE job for a customer, the process is one ordered step inside it with a beginning and an end.

- **Business process** goes with A collection of activities designed to produce a specific output for a particular customer or market
- **Process** goes with A specific ordering of work activities across time and place, with a beginning, an end, and defined inputs and outputs
- **Process modelling** goes with A technique for organising and documenting a system's processes, inputs, outputs, and data stores
- **Model** goes with A pictorial representation of reality

OBJECTIVE SECTION

Module Four, the Entity-Relationship Model

6 questions. Module Four. Part of compulsory Question Two. Cover the answers before you start.

1. **AUTHORED** A university system stores, for each student, a matriculation number, a name and a date of birth. Which TWO statements are correct? **WHICH TWO** (a) Student is an entity and matriculation number is one of its attributes (b) Student, matriculation number, name and date of birth are four separate entities (c) The matriculation number is a candidate identifier, because it is unique to one student (d) Date of birth is the natural primary key, because everyone has one
2. **AUTHORED** In a library, one member may borrow many books over time, and one book may be borrowed by many members over time. Which TWO statements are correct? **WHICH TWO** (a) The relationship between Member and Book is many to many (b) The

- relationship between Member and Book is one to many (c) A many-to-many relationship should be resolved into an associative entity such as Loan (d) A many-to-many relationship is drawn with a single bar at each end
3. **AUTHORED** On a crow's foot diagram, a line from Department to Employee ends at Employee with a crow's foot and a circle. What does that end mean? (a) Exactly one employee (b) Zero or many employees (c) One or many employees (d) Zero or one employee
4. **AUTHORED** The course text draws entity-relationship diagrams in Chen notation while the examination asks for the crow's foot style. Which TWO statements are correct? **WHICH TWO** (a) In Chen notation a relationship is drawn as a diamond on the line (b) In Chen notation attributes are drawn as ovals attached to the entity (c) Crow's foot notation writes cardinality as the numbers 1, M and N beside the line (d) Crow's foot notation cannot express an optional relationship
5. **AUTHORED** Which TWO statements about keys are correct? **WHICH TWO** (a) A primary key uniquely identifies one instance of an entity (b) A foreign key is an attribute that holds the primary key of a related entity (c) A foreign key must be unique within its own table (d) An entity may have several primary keys at once
6. **AUTHORED** A diagram shows Customer connected to Order with a bar and circle at the Customer end and a crow's foot and bar at the Order end. Which TWO readings are correct? **WHICH TWO** (a) Each order belongs to exactly one customer (b) Each customer has at least one order (c) A customer may exist with no orders (d) An order may exist with no customer

1. The two answers are **Student is an entity and matriculation number is one of its attributes** and **The matriculation number is a candidate identifier, because it is unique to one student**. An entity is the THING the business keeps data about; its attributes are the pieces of data kept. An identifier is the attribute that picks out exactly one instance, which is why uniqueness, not universality, is the test.

- ✓ **Student is an entity and matriculation number is one of its attributes.** Correct. Student is the thing; matriculation number, name and date of birth are facts about it.
- ✗ **Student, matriculation number, name and date of birth are four separate entities.** Wrong. Attributes are not entities. Making each attribute an entity gives a diagram with no structure and no relationships worth drawing.
- ✓ **The matriculation number is a candidate identifier, because it is unique to one student.** Correct. It is unique to one student, which is exactly what a candidate identifier means.
- ✗ **Date of birth is the natural primary key, because everyone has one.** Wrong. Everyone having one is not enough: many students share a date of birth, so it cannot identify a single instance.

2. The two answers are **The relationship between Member and Book is many to many** and **A many-to-many relationship should be resolved into an associative entity such as Loan**. Many to many is read in BOTH directions, and it is the one cardinality that cannot be implemented directly. It is resolved by an associative entity, which is where the attributes of the relationship itself, such as the date borrowed, finally have somewhere to live.

- ✓ **The relationship between Member and Book is many to many.** Correct. Many in each direction is the definition of many to many.
- ✗ **The relationship between Member and Book is one to many.** Wrong. One to many would mean a book could only ever be borrowed by one member, which the description contradicts.
- ✓ **A many-to-many relationship should be resolved into an associative entity such as Loan.** Correct. Loan holds the two foreign keys plus the borrow date and due date, which have no home on either original entity.
- ✗ **A many-to-many relationship is drawn with a single bar at each end.** Wrong. A single bar means exactly one. Many is the crow's foot.

3. The answer is **Zero or many employees**. Read the end symbols as two marks: the INNER one nearest the entity gives the maximum, and the OUTER one gives the minimum. A crow's foot is many, a bar is one, and a circle is zero.

- ✗ **Exactly one employee.** Wrong. Exactly one is two bars, not a foot and a circle.
- ✓ **Zero or many employees.** Correct. The foot gives many as the maximum and the circle gives zero as the minimum, so a department may have no employees at all.
- ✗ **One or many employees.** Wrong. One or many would be a crow's foot with a BAR, not a circle. The circle is what makes it optional.
- ✗ **Zero or one employee.** Wrong. Zero or one is a circle with a bar. There is no crow's foot in it.

4. The two answers are **In Chen notation a relationship is drawn as a diamond on the line** and **In Chen notation attributes are drawn as ovals attached to the entity**. The two styles say the same thing differently. Chen spends space: a diamond for each relationship and an oval for each attribute. Crow's foot is compact: attributes go inside the entity box and the cardinality is carried by the marks on the line ends.

- ✓ **In Chen notation a relationship is drawn as a diamond on the line.** Correct. The diamond holds the relationship name, with the two entities either side of it.
- ✓ **In Chen notation attributes are drawn as ovals attached to the entity.** Correct. Each attribute gets its own oval joined to the entity rectangle, which is why a Chen diagram of a real system covers a wall.

- ✗ **Crow's foot notation writes cardinality as the numbers 1, M and N beside the line.** Wrong. Writing 1, M and N beside the line is the CHEN convention. Crow's foot uses the bar, the circle and the foot.
- ✗ **Crow's foot notation cannot express an optional relationship.** Wrong. Optionality is exactly what the circle expresses, and expressing it is one of the reasons the style is preferred.

5. The two answers are **A primary key uniquely identifies one instance of an entity** and **A foreign key is an attribute that holds the primary key of a related entity**. The primary key identifies; the foreign key refers. That pair is how a set of separate entities becomes a connected model, and it is what a relationship line actually becomes when the design is built.

- ✓ **A primary key uniquely identifies one instance of an entity.** Correct. Unique identification of a single instance is the whole job of a primary key.
- ✓ **A foreign key is an attribute that holds the primary key of a related entity.** Correct. The foreign key is the copy of the other entity's identifier, and it is where the relationship physically lives.
- ✗ **A foreign key must be unique within its own table.** Wrong, and it is the common confusion. A foreign key repeats freely: many employees carry the same department number, which is what many to one MEANS.
- ✗ **An entity may have several primary keys at once.** Wrong. There may be several CANDIDATE keys, but exactly one of them is chosen as the primary key.

6. The two answers are **Each customer has at least one order** and **A customer may exist with no orders**. Read each end for the entity at the OTHER end. This is the single most common reading error on the paper, and the cure is to say the sentence out loud in both directions before writing anything.

- ✗ **Each order belongs to exactly one customer.** Wrong as a reading of the marks given. The Customer end carries a bar and a circle, which is zero or one, not exactly one.
- ✓ **Each customer has at least one order.** Correct. The Order end has a foot and a bar, which is one or many, so at least one order per customer.
- ✓ **A customer may exist with no orders.** Correct as the other half of the same pair: the circle at the Customer end makes the customer optional to an order.
- ✗ **An order may exist with no customer.** Wrong. Optionality on the Customer end says a customer need not be attached, not that an order floats free of the relationship entirely.

OBJECTIVE SECTION

Module Five, Object Orientation and UML

10 questions. Module Five. Compulsory Question Three. Cover the answers before you start.

1. **TEST 2 Q5** Which of the following correctly describes the arrow direction convention for an extends relationship, such as "Generate Warehouse Packing Order" extending "Place New Member Order"? (a) The arrow is drawn from the extension use case to the use case being extended, labelled with the extends stereotype (b) The arrow is drawn from the base use case to the extension use case, labelled with the extends stereotype (c) The arrow is drawn from the extension use case to the actor, labelled with the include stereotype (d) No arrow is used; extends relationships are shown only as dashed rectangles
2. **AUTHORED** Which TWO statements correctly distinguish a class from an object?
WHICH TWO (a) A class is a blueprint describing structure and behaviour (b) An object is an instance of a class, with actual values for its attributes (c) A class holds the

values and an object holds the definitions (d) There can be only one object of any given class

3. **AUTHORED** Which TWO statements about encapsulation are correct? **WHICH TWO** (a) It hides an object's internal data and exposes it only through its methods (b) It means one class inherits the attributes of another (c) It protects an object from being changed in ways its own methods do not allow (d) It allows the same method name to behave differently in different classes
4. **AUTHORED** Which TWO statements are correct? **WHICH TWO** (a) Inheritance lets a subclass reuse the attributes and methods of a superclass (b) Polymorphism lets the same message produce different behaviour in different classes (c) Inheritance is drawn as a filled diamond on the superclass end (d) Polymorphism removes the need for classes to define their own methods
5. **AUTHORED** A UML class is drawn as a rectangle split into three compartments. What do they hold, from top to bottom? (a) Name, attributes, operations (b) Name, operations, attributes (c) Attributes, name, operations (d) Name, attributes, relationships
6. **AUTHORED** A Department has Lecturers; if the department closes, the lecturers remain. A House has Rooms; if the house is demolished, the rooms cease to exist. Which TWO statements are correct? **WHICH TWO** (a) Department to Lecturer is aggregation, drawn with a hollow diamond (b) House to Room is composition, drawn with a filled diamond (c) Department to Lecturer is composition, because a lecturer belongs to a department (d) Both are drawn with a hollow triangle, because both are whole to part
7. **AUTHORED** In a use case diagram, which TWO statements are correct? **WHICH TWO** (a) An include relationship means the included use case ALWAYS runs as part of the base use case (b) An extend relationship means the extending use case runs only under a stated condition (c) An include relationship means the included use case is optional (d) The arrow of an include points from the included use case to the base use case
8. **AUTHORED** Which TWO statements about an activity diagram are correct? **WHICH TWO** (a) A diamond represents a decision, with a guard condition on each outgoing branch (b) A thick bar represents a fork into, or a join of, parallel flows (c) A diamond represents an action carried out by the system (d) A swimlane shows the order in which actions occur
9. **AUTHORED** Which TWO statements about a sequence diagram are correct? **WHICH TWO** (a) Time runs down the page, so the vertical order of messages is their order in time (b) A vertical dashed line under an object is its lifeline (c) Time runs left to right across the page (d) A sequence diagram shows the static structure of the classes involved
10. **AUTHORED** A class diagram shows Order 1 1..* OrderLine. What does it say? (a) One order has one or more order lines, and each order line belongs to exactly one order (b) One order has exactly one order line (c) One order may have no order lines (d) Many orders share the same order line

1. The answer is **The arrow is drawn from the extension use case to the use case being extended, labelled with the extends stereotype**. EXTENDS points from the extension BACK to the base, which feels backwards until you read it as a sentence: "Generate Warehouse Packing Order EXTENDS Place New Member Order". The arrow follows the sentence. Contrast INCLUDES, which points from the base to the thing it always uses: "Place New Member Order INCLUDES Check Credit". Get the two directions straight and this whole family of questions falls.

- ✓ **The arrow is drawn from the extension use case to the use case being extended, labelled with the extends stereotype.** Correct. From the extension to the base, with the extends stereotype.
- ✗ **The arrow is drawn from the base use case to the extension use case, labelled with the extends stereotype.** Wrong direction. That is how INCLUDES is drawn.
- ✗ **The arrow is drawn from the extension use case to the actor, labelled with the include stereotype.** Wrong twice: the arrow does not point at an actor, and the stereotype is extends, not include.
- ✗ **No arrow is used; extends relationships are shown only as dashed rectangles.** Wrong. It is a dashed arrow with a stereotype label, not a rectangle.

2. The two answers are **A class is a blueprint describing structure and behaviour** and **An object is an instance of a class, with actual values for its attributes**. The class is the definition and the object is one filled-in copy of it. The Contacts app is the standard image: one idea of what a contact is, and hundreds of saved contacts that are instances of it.

- ✓ **A class is a blueprint describing structure and behaviour.** Correct. It defines what every instance will have and be able to do.
- ✓ **An object is an instance of a class, with actual values for its attributes.** Correct. Instantiating the class produces an object with real values in its attributes.
- ✗ **A class holds the values and an object holds the definitions.** Wrong, and it is exactly backwards. The definitions are in the class; the values are in the object.
- ✗ **There can be only one object of any given class.** Wrong. A class exists precisely so that many objects can be made from it.

3. The two answers are **It hides an object's internal data and exposes it only through its methods** and **It protects an object from being changed in ways its own methods do not allow**. Encapsulation is data hiding plus controlled access. The two halves go together: hiding the data is pointless without a defined way in, and a defined way in is what lets the object keep itself valid.

- ✓ **It hides an object's internal data and exposes it only through its methods.** Correct. Attributes are private, methods are the public door.
- ✗ **It means one class inherits the attributes of another.** Wrong. That is inheritance, a different concept entirely.
- ✓ **It protects an object from being changed in ways its own methods do not allow.** Correct. Because every change goes through a method, the object can refuse an invalid one.
- ✗ **It allows the same method name to behave differently in different classes.** Wrong. That is polymorphism.

4. The two answers are **Inheritance lets a subclass reuse the attributes and methods of a superclass** and **Polymorphism lets the same message produce different behaviour in different classes**. Inheritance is about REUSE and is a relationship between classes. Polymorphism is about BEHAVIOUR and is what makes that re-

use worth having: the caller sends one message and each subclass answers it in its own way.

- ✓ **Inheritance lets a subclass reuse the attributes and methods of a superclass.** Correct. The subclass gets everything the superclass has and adds or overrides what it needs.
- ✓ **Polymorphism lets the same message produce different behaviour in different classes.** Correct. Draw() sent to Circle and to Square produces two different drawings from one call.
- ✗ **Inheritance is drawn as a filled diamond on the superclass end.** Wrong. Inheritance, or generalisation, is a hollow TRIANGLE pointing at the superclass. A filled diamond is composition.
- ✗ **Polymorphism removes the need for classes to define their own methods.** Wrong. Polymorphism depends on each class defining its own version. Without those definitions there is nothing to vary.

5. The answer is **Name, attributes, operations**. Name, then data, then behaviour. It is worth writing in that order every time, because an examiner scanning a page of class boxes reads the top line first and a box with no name is a box with no marks.

- ✓ **Name, attributes, operations.** Correct. The class name on top, its attributes in the middle, its operations or methods at the bottom.
- ✗ **Name, operations, attributes.** Wrong. Data comes before behaviour: what the thing HAS, then what it DOES.
- ✗ **Attributes, name, operations.** Wrong. The name is always the top compartment.
- ✗ **Name, attributes, relationships.** Wrong. Relationships are drawn as lines BETWEEN boxes, never inside one.

6. The two answers are **Department to Lecturer is aggregation, drawn with a hollow diamond** and **House to Room is composition, drawn with a filled diamond**. The test is LIFETIME, not ownership. If the part survives the whole it is aggregation and the diamond is hollow; if the part dies with the whole it is composition and the diamond is filled. The two symbols look almost identical and mean opposite things, which is exactly why the paper asks.

- ✓ **Department to Lecturer is aggregation, drawn with a hollow diamond.** Correct. The lecturer outlives the department, so the whole-to-part link is the weaker one.
- ✓ **House to Room is composition, drawn with a filled diamond.** Correct. A room cannot exist without its house, so the diamond is filled.
- ✗ **Department to Lecturer is composition, because a lecturer belongs to a department.** Wrong. Belonging is not the test. A lecturer belongs to a department and still survives it.
- ✗ **Both are drawn with a hollow triangle, because both are whole to part.** Wrong. A hollow triangle is generalisation, that is inheritance, which is a different relationship altogether.

7. The two answers are **An include relationship means the included use case ALWAYS runs as part of the base use case** and **An extend relationship means the extending use case runs only under a stated condition**. Include is compulsory and factors out shared behaviour; extend is conditional and adds behaviour in special cases. Both arrows are dashed with an open head, and the direction is what separates them: include points AT what is included, extend points AT what is being extended.

- ✓ **An include relationship means the included use case ALWAYS runs as part of the base use case.** Correct. Verify PIN is included by Withdraw Cash because it happens every time.
- ✓ **An extend relationship means the extending use case runs only under a stated condition.** Correct. Print Receipt extends Withdraw Cash because it happens only if the customer asks.
- ✗ **An include relationship means the included use case is optional.** Wrong. Optional is extend. Include is always.
- ✗ **The arrow of an include points from the included use case to the base use case.** Wrong. The include arrow points FROM the base use case TO the included one.

8. The two answers are **A diamond represents a decision, with a guard condition on each outgoing branch** and **A thick bar represents a fork into, or a join of, parallel flows**. The diamond BRANCHES, the bar PARALLELISES, and the rounded rectangle acts. A swimlane answers a different question: not when, but WHO.

✓ **A diamond represents a decision, with a guard condition on each outgoing branch.**

Correct. The guards must be mutually exclusive and cover every case, or the flow can stall or split.

✓ **A thick bar represents a fork into, or a join of, parallel flows.** Correct. One bar in and several out is a fork; several in and one out is a join.

✗ **A diamond represents an action carried out by the system.** Wrong. An action is a rounded rectangle. A diamond never does work.

✗ **A swimlane shows the order in which actions occur.** Wrong. A swimlane shows WHO or WHAT performs each action. Order is carried by the arrows.

9. The two answers are **Time runs down the page, so the vertical order of messages is their order in time** and **A vertical dashed line under an object is its lifeline**. A sequence diagram is a BEHAVIOURAL diagram: participants across the top, time down the side, messages as horizontal arrows between lifelines. Structure is what a class diagram is for.

✓ **Time runs down the page, so the vertical order of messages is their order in time.**

Correct, and it is the one rule that makes the diagram readable. Read it top to bottom like a script.

✓ **A vertical dashed line under an object is its lifeline.** Correct. The lifeline is the object's existence over time, with an activation bar drawn on it while it is actually doing something.

✗ **Time runs left to right across the page.** Wrong. The participants are arranged left to right; TIME is the vertical axis.

✗ **A sequence diagram shows the static structure of the classes involved.** Wrong. Static structure is the class diagram. A sequence diagram shows one scenario unfolding.

10. The answer is **One order has one or more order lines, and each order line belongs to exactly one order**. Multiplicity is written at the END NEAR the class it describes, so read across the line to find which number applies to which class. 1..* is one or more, 0..* is zero or more, and a bare 1 is exactly one.

✓ **One order has one or more order lines, and each order line belongs to exactly one order.**

Correct. 1..* at the OrderLine end is one or more lines; 1 at the Order end is exactly one order.

✗ **One order has exactly one order line.** Wrong. The star means many. Exactly one would be written 1.

✗ **One order may have no order lines.** Wrong. Zero would need 0..*, and the lower bound here is 1.

✗ **Many orders share the same order line.** Wrong. The 1 at the Order end forbids that: a line belongs to one order only.

OBJECTIVE SECTION

Module Six, Design, Architecture and Delivery

6 questions. Module Six. Objective section only: the course text does not carry it. Cover the answers before you start.

1. **AUTHORED** Which TWO statements about system installation strategies are correct?

WHICH TWO

(a) Direct cutover switches off the old system and starts the new one at once, which is cheapest and riskiest (b) Parallel running operates both systems together for a period and compares their output (c) Phased installation means installing

the whole system at one site before the others (d) Parallel running is the cheapest strategy because only one system is paid for

2. **AUTHORED** An organisation is choosing between developing a system in house and buying a commercial package. Which TWO statements are correct? **WHICH TWO** (a) Building in house gives a closer fit to the organisation's own processes (b) Buying a package usually delivers faster and at a lower initial cost (c) Buying a package guarantees a closer fit to the business processes (d) Building in house removes the need for a feasibility study
3. **AUTHORED** In a three-tier architecture, what are the three tiers? (a) Presentation, application logic, and data (b) Input, processing, and output (c) Hardware, software, and people (d) Analysis, design, and implementation
4. **AUTHORED** Which TWO statements about system maintenance are correct? **WHICH TWO** (a) Corrective maintenance fixes faults found after the system went live (b) Adaptive maintenance changes the system to suit a new environment or a new regulation (c) Perfective maintenance means removing features that are no longer used (d) Preventive maintenance is the same thing as corrective maintenance
5. **AUTHORED** Which TWO statements about scheduling a project are correct? **WHICH TWO** (a) A Gantt chart shows tasks as bars against a calendar, making durations and overlaps visible (b) The critical path is the longest chain of dependent tasks, and it sets the shortest possible project duration (c) The critical path is the chain of tasks that can slip without affecting the end date (d) A Gantt chart shows the dependencies between tasks better than any other tool
6. **AUTHORED** Which TWO principles of user interface design are correct? **WHICH TWO** (a) Be consistent, so the same action produces the same result everywhere in the system (b) Give feedback, so the user always knows what the system is doing and what happened (c) Maximise the amount of information on every screen so fewer screens are needed (d) Require the user to memorise codes, since experts work faster with codes

1. The two answers are **Direct cutover switches off the old system and starts the new one at once, which is cheapest and riskiest** and **Parallel running operates both systems together for a period and compares their output**. The four strategies trade cost against risk. Direct is cheap and dangerous, parallel is expensive and safe, phased brings the system in a part at a time, and pilot brings it in at one place at a time.

- ✓ **Direct cutover switches off the old system and starts the new one at once, which is cheapest and riskiest.** Correct. There is no fallback, which is exactly why it is the cheapest and the most exposed.
- ✓ **Parallel running operates both systems together for a period and compares their output.** Correct. Running both and comparing is what makes it safe, and paying for both is what makes it costly.
- ✗ **Phased installation means installing the whole system at one site before the others.** Wrong. Installing at one site first is the PILOT strategy. Phased brings the system in module by module.
- ✗ **Parallel running is the cheapest strategy because only one system is paid for.** Wrong, and it is the reverse: parallel running is the most expensive, because both systems and both sets of staff effort are paid for at once.

2. The two answers are **Building in house gives a closer fit to the organisation's own processes** and **Buying a package usually delivers faster and at a lower initial cost**. The trade is FIT against TIME and COST. Building fits exactly and takes longer; buying arrives sooner and asks the organisation to bend towards the software.

- ✓ **Building in house gives a closer fit to the organisation's own processes.** Correct. It is built to the requirements you gathered, so it matches them.
- ✓ **Buying a package usually delivers faster and at a lower initial cost.** Correct. The development has already been paid for by everyone else who bought it.
- ✗ **Buying a package guarantees a closer fit to the business processes.** Wrong, and it is the main argument against buying: the package embodies someone else's idea of the process.
- ✗ **Building in house removes the need for a feasibility study.** Wrong. Feasibility is assessed whichever route is chosen, and the choice itself is part of what feasibility examines.

3. The answer is **Presentation, application logic, and data**. The tiers separate what the user sees, what the system decides, and where the data lives. The point of the split is that any one tier can be changed or scaled without rewriting the others.

- ✓ **Presentation, application logic, and data.** Correct. Presentation at the front, business logic in the middle, the database behind.
- ✗ **Input, processing, and output.** Wrong. Input, process, output is the general model of any program, not an architecture.
- ✗ **Hardware, software, and people.** Wrong. Those are components of an information SYSTEM, not tiers of an architecture.
- ✗ **Analysis, design, and implementation.** Wrong. Those are phases of the life cycle.

4. The two answers are **Corrective maintenance fixes faults found after the system went live** and **Adaptive maintenance changes the system to suit a new environment or a new regulation**. Four kinds, and the examiner wants them separated: corrective fixes what is broken, adaptive responds to a changed environment, perfective improves what already works, and preventive reduces future failure.

- ✓ **Corrective maintenance fixes faults found after the system went live.** Correct. It is the reactive one: something is wrong, and it is repaired.

- ✓ **Adaptive maintenance changes the system to suit a new environment or a new regulation.** Correct. A new tax rate or a new operating system forces adaptive change though nothing is broken.
- ✗ **Perfective maintenance means removing features that are no longer used.** Wrong. Perfective maintenance IMPROVES performance or maintainability, or adds requested enhancements.
- ✗ **Preventive maintenance is the same thing as corrective maintenance.** Wrong. Preventive works on code that has not failed yet; corrective works on code that has.

5. The two answers are A Gantt chart shows tasks as bars against a calendar, making durations and overlaps visible and The critical path is the longest chain of dependent tasks, and it sets the shortest possible project duration. A Gantt chart shows WHEN; a network diagram shows WHAT DEPENDS ON WHAT. The critical path comes out of the second and is the sequence with no slack, so a day lost anywhere on it is a day lost on the project.

- ✓ **A Gantt chart shows tasks as bars against a calendar, making durations and overlaps visible.** Correct. It is the calendar view, and it is what a client is usually shown.
- ✓ **The critical path is the longest chain of dependent tasks, and it sets the shortest possible project duration.** Correct. It is longest and therefore binding: nothing can finish sooner than the critical path allows.
- ✗ **The critical path is the chain of tasks that can slip without affecting the end date.** Wrong, and it is exactly backwards. Tasks that can slip have FLOAT, and they are by definition not on the critical path.
- ✗ **A Gantt chart shows the dependencies between tasks better than any other tool.** Wrong. Dependencies are what a PERT or network diagram shows. A Gantt chart shows them poorly, which is why both tools exist.

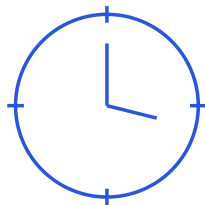
6. The two answers are Be consistent, so the same action produces the same result everywhere in the system and Give feedback, so the user always knows what the system is doing and what happened. Consistency and feedback are the two that appear on every list. Both reduce the load on the user's memory, which is the underlying goal of the whole subject.

- ✓ **Be consistent, so the same action produces the same result everywhere in the system.** Correct. Consistency means the interface can be learned once rather than relearned per screen.
- ✓ **Give feedback, so the user always knows what the system is doing and what happened.** Correct. A user with no feedback cannot tell a slow system from a broken one.
- ✗ **Maximise the amount of information on every screen so fewer screens are needed.** Wrong. Crowding a screen raises the error rate. The principle is the opposite: reduce what the eye must sort through.
- ✗ **Require the user to memorise codes, since experts work faster with codes.** Wrong. Recognition over recall is the principle. The system should show the options, not demand that the user remembers them.

MOCK EXAMINATION ONE

Sit This Under Exam Conditions

Three hours, no notes. Answer one question in Section A and any three in Section B, exactly as in the hall. These questions are ours, built to the exact shape of the 2025/2026 paper with fresh scenarios, so you practise the method and not a memorised answer. Attempt the whole paper before you turn to the answers.



Question Paper

Time: three hours. Section A: answer one. Section B: answer any three. Marks in brackets. These are practice questions written for this manual, not a past paper.

Section A · Answer only one question.

PRACTICE QUESTION

MOCK 1 · Q1

An online bookstore lets a customer browse books, place an order, and pay. Each order is placed by exactly one customer, but a customer may place many orders over time. Each order contains one or more books, and a book may appear on many orders. A courier delivers each paid order to the customer.

- Draw an ERD to model the relationships between the identified entities. (4.5 marks)
- Data Flow Diagram (DFD): (i) Draw a context diagram for the entire system. (1.5 marks) (ii) Draw a Level 0 DFD showing the major processes. (4.5 marks) (iii) Draw a Level 1 DFD for the payment process. (3 marks)
- State three reasons the project could succeed and three reasons it could fail. (3 marks)
- Read and interpret, using forward and backward chaining, a relationship in which a BOOK is joined to an ORDER with a crow's foot at each end. (1 mark)

PRACTICE QUESTION

MOCK 1 · Q2

A school library system lets a member borrow and return books. A member must be a registered student or a member of staff before borrowing. A librarian records each loan. A member can hold many loans, but each loan belongs to one member.

- Draw a class diagram for this scenario, showing multiplicity where necessary. (5 marks)
- Draw a use case diagram for the system. (5 marks)
- Draw an activity diagram for the borrowing process. (5 marks)
- Differentiate between an object and a class with an example. (2.5 marks)

Section B · Answer any three questions.

A neighbourhood laundry wants to automate its booking and collection business. They hire an analyst, Aisha, to guide them.

- a. Explain the three questions the team must answer during the feasibility analysis phase. (3 marks)
- b. Aisha's first step is to research and understand the problem. Outline six other steps she can take. (3 marks)
- c. How should Aisha explain to the owners what a model is? (4 marks)
- d. Besides people skills, explain one other skill Aisha needs to do a great job. (3 marks)
- e. What three activities will the support group participate in after the system is live? (3 marks)
- f. Mention three other job roles Aisha could fill. (1.5 marks)

A logistics company wants to acquire a fleet-tracking system quickly; it has no strategic software skills in-house.

- a. Which system acquisition strategy fits, and what three factors support your choice? (3 marks)
- b. Name two software architectural functions the system will need. (3 marks)
- c. Besides operational requirements, name two other types of hardware non-functional requirement. (3 marks)
- d. Outline three principles of interface design. (3 marks)
- e. Design and populate a relational database of Driver, Trip and Payment, with primary keys, at least three columns and five rows each. (5.5 marks)

A state university wants to replace its manual, paper-based student result processing with an integrated results and transcript system. Lecturers want speed, students want accuracy and quick access, and the registry insists on strict audit trails and reliability. The system must be delivered within one academic session.

- a. Briefly compare seven methodologies that could be used, saying why each is suitable or not. (7 marks)
- b. Identify the five activities in project planning. (5 marks)
- c. From the case, formulate at least: two business requirements, two user requirements, three functional requirements, two non-functional requirements, and two system requirements. (5.5 marks)

A hospital group wants to build a telemedicine platform so patients can consult doctors by video, share records, and receive e-prescriptions. At the start the requirements are unclear, and halfway through, management asks to add online payment and pharmacy delivery.

- a. Identify the weaknesses of the as-is system, the improvements in the to-be system, and the assumptions to validate before development. (3.5 marks)
- b. Design a requirements elicitation plan using at least four techniques; for each, give the stakeholders, the information collected, and why it suits this project. (4 marks)
- c. Write short notes on UML, stating its inventors and uses. (2 marks)
- d. Categorise UML diagrams into two broad categories, giving two diagrams and their function in each. (4 marks)
- e. With diagrams, explain inheritance, composition, aggregation and association. (4 marks)

Answers

Mark your own work honestly. Where an answer says "draw", check your diagram against the figure named in the relevant module.

ANSWER

MOCK 1 · Q1

- a. ERD.** Entities: CUSTOMER, ORDER, BOOK, COURIER. CUSTOMER 1:M ORDER (a customer places many orders, each order one customer); ORDER M:N BOOK (an order has many books, a book on many orders, resolved with an ORDER-LINE linking entity); COURIER 1:M ORDER (a courier delivers many orders). Crow's foot throughout.
- b. DFDs.** Context: one process "O Bookstore" between CUSTOMER and COURIER, with order and payment in, confirmation and delivery out. Level 0: 1.0 Place order, 2.0 Take payment, 3.0 Arrange delivery, plus stores D1 Orders and D2 Books. Level 1 of 2.0: 2.1 Validate card, 2.2 Charge, 2.3 Issue receipt, balancing 2.0.
- c.** Succeed: clear requirements, user involvement, realistic schedule. Fail: unclear requirements, no user involvement, weak management support.
- d.** A crow's foot at each end is many-to-many: forward, one BOOK appears on many ORDERS; backward, one ORDER contains many BOOKS.

ANSWER

MOCK 1 · Q2

- a. Class diagram.** Classes MEMBER, STUDENT, STAFF, LOAN, BOOK, LIBRARIAN. STUDENT and STAFF generalise to MEMBER (hollow triangle). MEMBER 1 to * LOAN; LOAN * to 1 BOOK; LIBRARIAN records * LOAN. Give each class two attributes and a method.
- b. Use case.** Actors MEMBER and LIBRARIAN; use cases Register, Borrow book, Return book, Record loan. "Borrow book" includes "Check membership".
- c. Activity.** Start, "Request book", decision "registered member?" (no ends, yes continues), "Check availability", "Record loan", "Issue book", end.
- d.** A class is a blueprint (Book); an object is an instance of it (the specific copy of "Systems Analysis" with barcode 0012).

- a.** Is it technically feasible (can we build it)? Economically feasible (worth the cost)? Operationally feasible (will it be used and fit the business)?
- b.** Verify benefit outweighs cost; define requirements; develop alternatives; recommend the best; detail the chosen solution; implement (then monitor).
- c.** A model is a simplified picture of the system that shows the important parts so everyone can agree them before building, like an architect's drawing.
- d.** Technical skills: understanding computers, networks and databases and the tools of building, so she can specify and communicate technical requirements. (Business knowledge is also acceptable.)
- e.** Maintain the system, enhance the system, support the users (help desk).
- f.** Project manager, software architect, product designer.

- a.** Buy a packaged solution or outsource, not build; factors: short time available, lack of in-house skills, the system is not strategic (also cost, package fit).
- b.** Any two of data storage, data access logic, application logic, presentation logic.
- c.** Performance and security (the fourth is cultural/political).
- d.** Consistency, simplicity/clarity, user feedback.
- e.** Three tables, each with a primary key: Driver(driverId PK, name, phone), Trip(tripId PK, driverId FK, distance), Payment(paymentId PK, tripId FK, amount), five dummy rows each; driverId and tripId are the linking foreign keys.

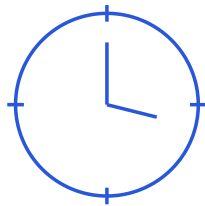
- a.** Compare waterfall (strong documentation, rigid), prototyping (good for unclear needs, can be mistaken for finished), RAD (fast, weak on large systems), spiral (risk analysis each cycle, heavy), incremental (early delivery, integration effort), agile (handles change, light documentation clashes with audit needs), V-shape (test pairing, reliable, rigid). For a results system needing audit trails and one-session delivery, an incremental or V-shape approach fits well.
- b.** Define the problem, produce the schedule, confirm feasibility, staff the project, launch it.
- c.** Business: cut result-processing time, guarantee an audit trail. User: a lecturer enters marks online, a student views results instantly. Functional: record marks, compute the grade, generate a transcript. Non-functional: 99.9% availability, encrypt results. System: store results as time-stamped records keyed by matriculation number, integrate with the student database.

- a.** As-is: no remote consultation, records on paper, no e-prescriptions. To-be: video consultation, shared records, e-prescriptions. Assumptions: patients have devices and connectivity; the platform can integrate with hospital records; video quality is adequate for diagnosis.
- b.** Interview doctors (clinical workflow; depth); questionnaire to patients (what they want; reaches many); observe a clinic (real process); review existing records (data and integration).
- c.** UML is a standard notation for modelling object-oriented systems, created by Booch, Rumbaugh and Jacobson; used to visualise, specify, construct and document a system.
- d.** Structural (class, object) model what the system is; behavioural (use case, activity, sequence) model what it does.
- e.** Inheritance (hollow triangle, is-a); composition (filled diamond, part dies with the whole); aggregation (hollow diamond, part survives); association (plain line).

MOCK EXAMINATION TWO

Sit This One After the First

Same rules, new scenarios. Three hours, one from Section A and any three from Section B. By now the routine should be automatic: read for nouns and verbs, draw boxes then lines then symbols, answer the verb in the instruction.



Question Paper

Time: three hours. Section A: answer one. Section B: answer any three. Practice questions written for this manual.

Section A · Answer only one question.

PRACTICE QUESTION

MOCK 2 · Q1

A hotel reservation system lets a guest book a room. Each booking is made by one guest, but a guest may make many bookings. Each booking is for one room, and a room may have many bookings over time. A receptionist confirms each booking and takes payment.

- Draw an ERD for the identified entities. (4.5 marks)
- DFD: (i) context diagram (1.5 marks); (ii) Level 0 (4.5 marks); (iii) Level 1 for the payment process (3 marks).
- Give three reasons the project could succeed and three it could fail. (3 marks)
- Interpret, forward and backward, a GUEST joined to a BOOKING with one at the guest end and many at the booking end. (1 mark)

PRACTICE QUESTION

MOCK 2 · Q2

A ride-hailing app connects a rider with a driver. A rider requests a trip; the system assigns one driver; each trip has one rider and one driver. A rider can be a standard rider or a premium rider. Payment is taken at the end of each trip.

- Draw a class diagram, showing multiplicity. (5 marks)
- Draw a use case diagram. (5 marks)
- Draw an activity diagram for requesting and completing a trip. (5 marks)
- State three drawbacks of multiple inheritance. (2.5 marks)

Section B · Answer any three questions.

PRACTICE QUESTION**MOCK 2 · Q3**

A bakery chain wants to automate ordering and delivery across its branches. They hire an analyst, Tunde, to lead the effort.

- a. Explain the three feasibility questions the team must answer. (3 marks)
- b. After researching and understanding the problem, outline six other steps Tunde can take. (3 marks)
- c. How should Tunde explain what a model is to the owners? (4 marks)
- d. Besides people skills, explain one other skill Tunde needs. (3 marks)
- e. What three activities will the support group participate in? (3 marks)
- f. Mention three other job roles Tunde could fill. (1.5 marks)

PRACTICE QUESTION**MOCK 2 · Q4**

A community bank wants to acquire a loan-management system fast; it lacks strategic software skills.

- a. Which acquisition strategy fits, and three supporting factors? (3 marks)
- b. Name two software architectural functions the system needs. (3 marks)
- c. Besides operational requirements, name two other hardware non-functional requirements. (3 marks)
- d. Outline three interface design principles. (3 marks)
- e. Design and populate a relational database of Account, Loan and Payment, with primary keys, at least three columns and five rows each. (5.5 marks)

PRACTICE QUESTION**MOCK 2 · Q5**

A courier company wants to replace its manual parcel-tracking with an integrated tracking and delivery system. Dispatchers want speed, customers want accurate live tracking, and management insists on reliability and audit trails. Delivery is required within nine months.

- a. Compare seven methodologies, saying why each suits this project or not. (7 marks)
- b. Identify the five project planning activities. (5 marks)
- c. Formulate at least two business, two user, three functional, two non-functional and two system requirements from the case. (5.5 marks)

A polytechnic wants to build an e-learning platform for lectures, assignments and grading. Requirements are unclear at first; midway, management asks to add live classes and a fee-payment portal.

- a. Identify the as-is weaknesses, to-be improvements, and assumptions to validate. (3.5 marks)
- b. Design an elicitation plan with at least four techniques, each with stakeholders, information and justification. (4 marks)
- c. Write short notes on UML with its inventors and uses. (2 marks)
- d. Categorise UML diagrams into two categories with two examples and their function each. (4 marks)
- e. With diagrams, explain inheritance, composition, aggregation and association. (4 marks)

Answers

Check each "draw" answer against the figure named in the relevant module.

ANSWER

MOCK 2 · Q1

- a.** Entities GUEST, BOOKING, ROOM, RECEPTIONIST. GUEST 1:M BOOKING; ROOM 1:M BOOKING; RECEPTIONIST 1:M BOOKING. Crow's foot throughout.
- b.** Context: one process "0 Reservation" between GUEST and RECEPTIONIST. Level 0: 1.0 Make booking, 2.0 Take payment, 3.0 Confirm, with stores D1 Bookings, D2 Rooms. Level 1 of 2.0: validate, charge, receipt, balanced.
- c.** Succeed: clear requirements, user involvement, realistic schedule. Fail: unclear requirements, no user involvement, weak management support.
- d.** One at the guest end, many at booking: forward, one GUEST makes many BOOKINGS; backward, one BOOKING is made by one GUEST.

ANSWER

MOCK 2 · Q2

- a.** Classes RIDER, STANDARDRIDER, PREMIUMRIDER, DRIVER, TRIP, PAYMENT. STANDARDRIDER and PREMIUMRIDER generalise to RIDER (hollow triangle). RIDER 1 to * TRIP; DRIVER 1 to * TRIP; TRIP 1 to 1 PAYMENT.
- b.** Actors RIDER, DRIVER; use cases Request trip, Assign driver, Complete trip, Pay. "Complete trip" includes "Take payment".
- c.** Start, "Request trip", decision "driver available?" (no ends, yes continues), "Assign driver", "Make trip", "Take payment", end.
- d.** The diamond problem (ambiguity), increased complexity, name clashes.

ANSWER

MOCK 2 · Q3

- a.** Technically feasible? Economically feasible? Operationally feasible?
- b.** Verify benefit outweighs cost; define requirements; develop alternatives; recommend the best; detail the solution; implement (then monitor).
- c.** A model is a simplified picture that shows the important parts so all can agree them before building, like an architect's drawing.
- d.** Technical or business knowledge and skills, with an explanation of how it helps him specify the system.
- e.** Maintain, enhance, support the users (help desk).
- f.** Project manager, software architect, product designer.

- a.** Buy a package or outsource; factors: short time, no in-house skills, not strategic.
- b.** Any two of data storage, data access logic, application logic, presentation logic.
- c.** Performance and security (fourth: cultural/political).
- d.** Consistency, simplicity/clarity, user feedback.
- e.** Account(accId PK, holder, balance), Loan(loanId PK, accId FK, amount), Payment(payId PK, loanId FK, amount); five dummy rows each; accId and loanId are the foreign keys.

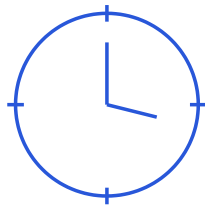
- a.** As in the manual: waterfall (documented, rigid), prototyping (unclear needs, mistaken for finished), RAD (fast, weak on large), spiral (risk analysis, heavy), incremental (early value, integration), agile (handles change, light docs), V-shape (test pairing, reliable, rigid). A tracking system needing audit trails and nine-month delivery suits incremental or V-shape.
- b.** Define the problem, schedule, confirm feasibility, staff, launch.
- c.** Business: faster delivery, full audit trail. User: a dispatcher assigns a parcel, a customer tracks live. Functional: register parcel, update status, notify customer. Non-functional: high availability, encrypt data. System: store status as time-stamped events keyed by parcel ID, integrate with the customer database.

- a.** As-is: no online lectures, manual grading, no analytics. To-be: online lectures, assignments, automated grading. Assumptions: students have connectivity, the platform integrates with existing records, the fee portal is available.
- b.** Interview lecturers (requirements, depth), questionnaire to students (needs, reach), observe a class (real use), review the existing system (data, integration).
- c.** UML: a standard object-oriented modelling notation by Booch, Rumbaugh and Jacobson; used to visualise, specify, construct and document.
- d.** Structural (class, object) and behavioural (use case, activity/sequence).
- e.** Inheritance (triangle, is-a), composition (filled diamond), aggregation (hollow diamond), association (plain line).

MOCK EXAMINATION THREE

One More, To Be Sure

The last rehearsal. If you can answer this paper cleanly, in time, with every diagram drawn without hesitation, you are ready for the hall. Three hours, one from Section A and any three from Section B.



Question Paper

Time: three hours. Section A: answer one. Section B: answer any three. Practice questions written for this manual.

Section A · Answer only one question.

PRACTICE QUESTION

MOCK 3 · Q1

A car rental system lets a customer rent a car. Each rental is by one customer, but a customer may have many rentals. Each rental is for one car, and a car may be rented many times. A staff member processes each rental and takes payment.

- Draw an ERD for the identified entities. (4.5 marks)
- DFD: (i) context diagram (1.5 marks); (ii) Level 0 (4.5 marks); (iii) Level 1 for the payment process (3 marks).
- Give three reasons the project could succeed and three it could fail. (3 marks)
- Interpret, forward and backward, a CUSTOMER joined to a RENTAL with one at the customer end and many at the rental end. (1 mark)

PRACTICE QUESTION

MOCK 3 · Q2

A hospital pharmacy system dispenses drugs against prescriptions. A pharmacist dispenses each prescription; a prescription belongs to one patient; a patient may have many prescriptions. A prescription can be an inpatient prescription or an outpatient prescription.

- Draw a class diagram, showing multiplicity. (5 marks)
- Draw a use case diagram. (5 marks)
- Draw an activity diagram for dispensing a prescription. (5 marks)
- Differentiate between an object and a class with an example. (2.5 marks)

Section B · Answer any three questions.

PRACTICE QUESTION**MOCK 3 · Q3**

A tailoring business wants to automate measurements, orders and delivery. They hire an analyst, Ngozi, to guide them.

- a. Explain the three feasibility questions the team must answer. (3 marks)
- b. After researching and understanding the problem, outline six other steps Ngozi can take. (3 marks)
- c. How should Ngozi explain what a model is to the owners? (4 marks)
- d. Besides people skills, explain one other skill Ngozi needs. (3 marks)
- e. What three activities will the support group participate in? (3 marks)
- f. Mention three other job roles Ngozi could fill. (1.5 marks)

PRACTICE QUESTION**MOCK 3 · Q4**

A supermarket wants to acquire a point-of-sale system quickly; it has no strategic software skills.

- a. Which acquisition strategy fits, and three supporting factors? (3 marks)
- b. Name two software architectural functions the system needs. (3 marks)
- c. Besides operational requirements, name two other hardware non-functional requirements. (3 marks)
- d. Outline three interface design principles. (3 marks)
- e. Design and populate a relational database of Product, Sale and Payment, with primary keys, at least three columns and five rows each. (5.5 marks)

PRACTICE QUESTION**MOCK 3 · Q5**

A government agency wants to replace its manual, paper-based tax filing with an integrated online tax system. Citizens want ease of use, the agency wants accuracy and strict audit trails, and security is critical. Delivery is required within one year.

- a. Compare seven methodologies, saying why each suits this project or not. (7 marks)
- b. Identify the five project planning activities. (5 marks)
- c. Formulate at least two business, two user, three functional, two non-functional and two system requirements from the case. (5.5 marks)

A university wants to build a smart-campus platform linking attendance, library and cafeteria services. Requirements are unclear at first; midway, management asks to add access control and mobile payments.

- a. Identify the as-is weaknesses, to-be improvements, and assumptions to validate. (3.5 marks)
- b. Design an elicitation plan with at least four techniques, each with stakeholders, information and justification. (4 marks)
- c. Write short notes on UML with its inventors and uses. (2 marks)
- d. Categorise UML diagrams into two categories with two examples and their function each. (4 marks)
- e. With diagrams, explain inheritance, composition, aggregation and association. (4 marks)

Answers

Check each "draw" answer against the figure named in the relevant module. If any answer here surprises you, go back to the module it comes from before the exam.

ANSWER

MOCK 3 · Q1

- a.** Entities CUSTOMER, RENTAL, CAR, STAFF. CUSTOMER 1:M RENTAL; CAR 1:M RENTAL; STAFF 1:M RENTAL. Crow's foot throughout.
- b.** Context: one process "0 Rental" between CUSTOMER and STAFF. Level 0: 1.0 Make rental, 2.0 Take payment, 3.0 Return car, stores D1 Rentals, D2 Cars. Level 1 of 2.0: validate, charge, receipt, balanced.
- c.** Succeed: clear requirements, user involvement, realistic schedule. Fail: unclear requirements, no user involvement, weak management support.
- d.** One customer, many rentals: forward, one CUSTOMER has many RENTALS; backward, one RENTAL belongs to one CUSTOMER.

ANSWER

MOCK 3 · Q2

- a.** Classes PATIENT, PRESCRIPTION, INPATIENTPRESCRIPTION, OUTPATIENTPRESCRIPTION, PHARMACIST, DRUG. INPATIENT and OUTPATIENT generalise to PRESCRIPTION (hollow triangle). PATIENT 1 to * PRESCRIPTION; PHARMACIST dispenses * PRESCRIPTION; PRESCRIPTION * to * DRUG (via a line item).
- b.** Actors PATIENT, PHARMACIST; use cases Present prescription, Dispense drug, Record. "Dispense" includes "Check stock".
- c.** Start, "Receive prescription", decision "in stock?" (no ends, yes continues), "Dispense", "Record", end.
- d.** A class is a blueprint (Drug); an object is an instance (the specific batch of Paracetamol with batch number B21).

ANSWER

MOCK 3 · Q3

- a.** Technically feasible? Economically feasible? Operationally feasible?
- b.** Verify benefit outweighs cost; define requirements; develop alternatives; recommend the best; detail the solution; implement (then monitor).
- c.** A model is a simplified picture that shows the important parts so all can agree them before building, like an architect's drawing.
- d.** Technical or business knowledge and skills, explained.
- e.** Maintain, enhance, support the users (help desk).
- f.** Project manager, software architect, product designer.

- a.** Buy a package or outsource; factors: short time, no in-house skills, not strategic.
- b.** Any two of data storage, data access logic, application logic, presentation logic.
- c.** Performance and security (fourth: cultural/political).
- d.** Consistency, simplicity/clarity, user feedback.
- e.** Product(prodId PK, name, price), Sale(saleId PK, prodId FK, qty), Payment(payId PK, saleId FK, amount); five dummy rows each; prodId and saleId are the foreign keys.

- a.** Waterfall (documented, rigid), prototyping (unclear needs, mistaken for finished), RAD (fast, weak on large), spiral (risk analysis, heavy), incremental (early value, integration), agile (handles change, light docs), V-shape (test pairing, reliable, rigid). A tax system needing security and audit trails suits V-shape or a disciplined incremental approach.
- b.** Define the problem, schedule, confirm feasibility, staff, launch.
- c.** Business: cut filing time, ensure audit trails. User: a citizen files online, an officer reviews a return. Functional: accept a return, compute tax due, issue a receipt. Non-functional: high availability, strong encryption. System: store returns as time-stamped records keyed by tax ID, integrate with the payment gateway.

- a.** As-is: separate, manual campus services, no single view, no analytics. To-be: one platform linking attendance, library and cafeteria. Assumptions: students carry compatible cards or phones, services can integrate, the network covers campus.
- b.** Interview service heads (rules, depth), questionnaire to students (needs, reach), observe a service point (real use), review existing systems (data, integration).
- c.** UML: a standard object-oriented modelling notation by Booch, Rumbaugh and Jacobson; used to visualise, specify, construct and document.
- d.** Structural (class, object) and behavioural (use case, activity/sequence).
- e.** Inheritance (triangle, is-a), composition (filled diamond), aggregation (hollow diamond), association (plain line).